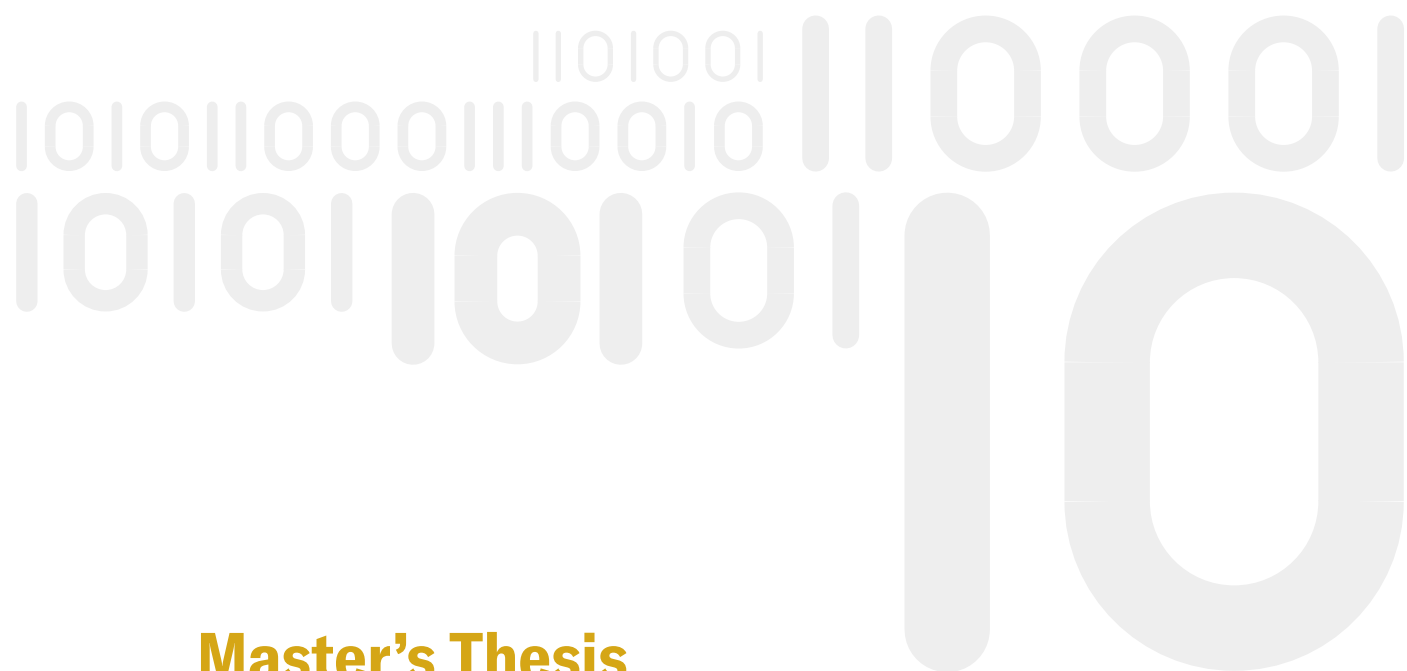




FACULTY OF APPLIED SCIENCES
UNIVERSITY OF WEST BOHEMIA
IN PILSEN

DEPARTMENT OF
COMPUTER SCIENCE
AND ENGINEERING

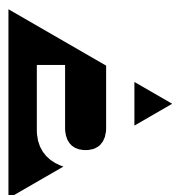


Master's Thesis

Distributed Metasolver

Jonáš Dufek





**FACULTY OF APPLIED SCIENCES
UNIVERSITY OF WEST BOHEMIA
IN PILSEN**

**DEPARTMENT OF
COMPUTER SCIENCE
AND ENGINEERING**

Master's Thesis

Distributed Metasolver

Bc. Jonáš Dufek

Thesis advisor

doc. Ing. Tomáš Koutný, Ph.D.

© 2026 Jonáš Dufek

All rights reserved. No part of this document may be reproduced or transmitted in any form by any means, electronic or mechanical including photocopying, recording or by any information storage and retrieval system, without permission from the copyright holder(s) in writing.

Citation in the bibliography/reference list:

DUFEK, Jonáš. *Distributed Metasolver*. Pilsen, Czech Republic, 2026. Master's Thesis. University of West Bohemia, Faculty of Applied Sciences, Department of Computer Science and Engineering. Thesis advisor doc. Ing. Tomáš Koutný, Ph.D.

ZÁPADOČESKÁ UNIVERZITA V PLZNI

Fakulta aplikovaných věd

Akademický rok: 2025/2026

ZADÁNÍ DIPLOMOVÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Bc. Jonáš DUFEK
Osobní číslo:	A24N0096P
Studijní program:	N0613A140039 Distribuované výpočetní systémy
Specializace:	Distribuované systémy
Téma práce:	Distribuovaný metasolver
Zadávací katedra:	Katedra informatiky a výpočetní techniky

Zásady pro vypracování

1. Seznamte se s problematikou solverů založených na populacích (evoluční a genetické algoritmy).
2. Navrhněte metasolver pracující v ostrovním režimu, ve kterém bude na každém ostrově použit jiný evoluční/genetický algoritmus – lze využít existující knihovny SmartCGMS a PaGMo.
3. Ostrovní model implementujte jako distribuovaný výpočet.
4. Funkcionalitu ověřte jako aplikační solver pro SmartCGMS.
5. Demostrujte výsledky optimalizace a zhodnotte provozní charakteristiky celého řešení.

Rozsah diplomové práce:	doporuč. 50 s. původního textu
Rozsah grafických prací:	dle potřeby
Forma zpracování diplomové práce:	tištěná/elektronická
Jazyk zpracování:	Angličtina

Seznam doporučené literatury:

dodá vedoucí diplomové práce.

Vedoucí diplomové práce:	doc. Ing. Tomáš Koutný, Ph.D. Katedra informatiky a výpočetní techniky
--------------------------	--

Datum zadání diplomové práce:	10. září 2025
Termín odevzdání diplomové práce:	14. května 2026

L.S.

doc. Ing. Miloš Železný, Ph.D.
děkan

doc. Ing. Přemysl Brada, MSc., Ph.D.
vedoucí katedry

V Plzni dne 30. září 2025

Declaration

I hereby declare that this Master's Thesis is completely my own work and that I used only the cited sources, literature, and other resources. This thesis has not been used to obtain another or the same academic degree.

I acknowledge that my thesis is subject to the rights and obligations arising from Act No. 121/2000 Coll., the Copyright Act as amended, in particular the fact that the University of West Bohemia has the right to conclude a licence agreement for the use of this thesis as a school work pursuant to Section 60(1) of the Copyright Act.

Plzeň, on 14 May 2026

.....

Jonáš Dufek

The names of products, technologies, services, applications, companies, etc. used in the text may be trademarks or registered trademarks of their respective owners.

Abstract

Many modern applications, such as artificial intelligence, machine learning, or medical informatics rely on specialized computational models that are optimized using solver software.

The aim of this thesis was to create a distributed metasolver utilizing the island-based model of work distribution, where each island represents a single worker node in a distributed system, with a centralized element being responsible for coordination tasks such as allocating work or distributing optimization problems.

The distributed metasolver was implemented as a C++ library and its performance was evaluated on several standardized optimization problems.

The results have demonstrated satisfactory performance regarding reliability and distribution of work, although the load-balancing mechanism proved to be less efficient in certain scenarios.

Abstrakt

Mnoho moderních oblastí výzkumu, jako například umělá inteligence, strojové učení nebo medicínská informatika, využívá specializované numerické modely optimalizované pomocí tzv. solverů.

Cílem této diplomové práce bylo implementovat distribuovaný metasolver pracující v ostrovním režimu, kde každý ostrov představuje jeden výpočetní uzel. Součástí tohoto distribuovaného systému je také centrální koordinační prvek zodpovědný za úlohy, jako je rozdělování práce mezi uzly nebo distribuce optimalizačních problémů.

Výsledný software byl implementován jako knihovna v jazyce C++ a jeho provozní charakteristiky byly ověřeny na několika standardizovaných optimalizačních úlohách.

Naměřená data ukázala uspokojivé vlastnosti systému z hlediska spolehlivosti a distribuce práce, oproti tomu algoritmy pro vyvážení zátěže mezi uzly v některých případech vykazovaly nižší efektivitu.

Keywords

distributed systems • metasolver • optimization • distributed computing • evolutionary algorithms • population-based algorithms

Acknowledgement

I would like to express my sincere gratitude to my supervisor doc. Ing. Tomáš Koutný, Ph.D., whose helpful feedback and guidance helped with the successful completion of this thesis.

Contents

1	Introduction	4
2	Optimization via Population-Based Solvers	6
2.1	The Fundamentals	6
2.2	Population-Based Algorithm	8
2.3	Population-Based Solver	9
2.4	Types of Population-Based Algorithms	10
2.4.1	Evolutionary Algorithms	11
2.4.2	Swarm Intelligence	19
2.5	The Number of Objective Functions	21
2.5.1	Single-Objective optimization	21
2.5.2	Multi-Objective optimization	21
2.6	Metasolving	22
2.6.1	The Island Model	23
2.6.2	Overview of a Metasolver	24
3	Distributed Computing	25
3.1	Parallel Virtual Machine	25
3.1.1	Architectural Components	26
3.1.2	Communication Model	26
3.1.3	Allocation of Work	28
3.1.4	Discovery of Executable Code	29
3.2	Active Networks	29
3.2.1	Architectural Components	30
3.2.2	Communication Model	30
3.2.3	Discovery of Executable Code	31
3.2.4	Reliability	32
4	Solver as a Distributed Application	33
4.1	Architectural Overview	33
4.2	Communication	34

4.3	Problem Discovery and Work Allocation	36
4.4	Reliability of the Cluster	37
5	Networking Libraries	38
5.1	BSD Sockets	38
5.2	Boost.Asio	41
5.3	ZeroMQ	45
6	Optimization Libraries	48
6.1	Pagmo	48
6.2	SmartCGMS	56
7	Architecture and Design of the Distributed Metasolver	59
7.1	Technological Stack	59
7.2	The Metasolver Library	60
7.3	Example Usage	62
8	The Network Protocol	64
8.1	Message Structure	64
8.2	Message Exchange	66
9	Implementation of the Metasolver Library	67
9.1	Logic	68
9.2	Messages	78
9.3	Sockets	79
9.4	Dynamic	81
9.5	Discovery	83
9.6	Utils	86
9.7	Benchmark	87
9.8	Unit Tests	87
10	Implementation of the SCGMS Wrapper	91
10.1	Overview	91
10.2	Usage	92
10.3	Implementation Details	93
11	Results	95
11.1	Simple Distributed Scenario	95
11.2	Three-Node Distributed Scenario	100
11.3	Distributed Metasolving Scenario	105
11.4	Local Metasolving Scenario	110
11.5	Local Single-Objective Metasolving	115

11.6 Local Multi-Objective Metasolving	118
12 Discussion	122
13 Conclusion	126
A Build Instructions	127
A.1 GNU Linux	127
A.2 MS Windows	129
B Controller Programs	132
B.1 Simple Distributed Scenario	132
B.2 Distributed Metasolving Scenario	133
Bibliography	135
List of Figures	142
List of Tables	147
List of Listings	148

Introduction

1

As the first general-purpose digital computers appeared after the Second World War, the most widespread form of hardware and software was sequential in nature.

The Von Neumann architecture introduced the **Fetch-Decode-Execute** cycle, in which the hardware fetched instructions of a sequential program, decoding and executing them “step-by-step” in accordance with the Program Counter (PC) register.

This sequential paradigm remained to be dominant for most of the 20th century, however, as early as 1958 [1] researchers began to explore the viability of implementing **parallel execution** of computer programs. Shortly after, early symmetric multiprocessor (SMP) systems appeared (Burroughs D825, 1962), followed by instruction pipelining (IBM 7030 Stretch, 1961) and SIMD vector processing (Cray-1, 1976). [2]

One of the more recent breakthroughs in terms of accelerating the execution time of a program was **distributed computing**. The introduction of the TCP/IP networking stack (1984), together with the advancement of personal computing, enabled the creation of economically efficient large scale computer clusters.

The goal of a distributed program, such as the **distributed metasolver** developed in this thesis, is therefore to distribute a workload among many nodes in a cluster, efficiently utilizing the CPU, GPGPU, or memory resources of each node.

To provide an example, a distributed solver is initialized with a **problem** and an initial **population**, this initial population is split into smaller portions which are then distributed to the various nodes of our cluster, each of the nodes then performs a computation using an **optimization algorithm**, and sends its results back to a centralized controller node.

The difference between a *solver* and a *metasolver* is that the latter uses multiple different optimization algorithms, in hopes of improving execution speed, diversity of solutions or some other metric.

Such solvers are useful for a wide range of optimization tasks, for example in **Machine Learning** or **Artificial Intelligence** they can be used for distributed training of *Neural Networks* or *Large Language Models* (LLMs). Another applica-

tion is estimating the parameters of scientific or medical models, for example the SmartCGMS software framework [3] discussed in this thesis focuses on *continuous glucose monitoring* models.

Apart from what was already described in the paragraphs above, the main goals and research questions of this thesis are:

- Designing and implementing a distributed metasolver using the “island-model” approach, existing software libraries may be utilized for the underlying functions
- Demonstrating the implemented solution as an application solver for the SmartCGMS framework
- Evaluating the metasolver on a set of exemplary problems, together with a discussion of the achieved results

The theoretical foundation upon which the metasolver operates is laid out in the following chapters. Later chapters of this thesis are concerned primarily with the development process and problems which were solved along the way.

Optimization via Population-Based Solvers

2

2.1 The Fundamentals

Before delving into further details, it should be explained that a distributed meta-solver is a specific case of **mathematical optimization** software.

Optimization is a technique which allows us to approximate the best **parameters** of some engineering or scientific system.

The **objective function** is what evaluates the “quality” of our parameters, it takes multiple parameters as an input and produces a single number. This number can for example represent the amount of used energy, the weight of an object, or the amount of spent time. Depending on the nature of our problem, we want to either maximize or minimize the objective function. [4, p. 2]

Additionally, the parameters of an objective function can be **constrained**. Constraints are a set of conditions which limit what values are the parameters allowed to take. The most common types are equality and inequality constraints.

The objective function, together with its parameters and constraints, forms what is called a mathematical **model** of a problem. After modeling our problem we can use an **optimization algorithm** to approximate its solution, this is where software tools brought the biggest benefit, many practical computational models (for example weather prediction models [5]) were developed long before the advent of digital computers, yet their usage was infeasible due to the high computational complexity.

For a formal definition of an optimization problem, please refer to equation 2.1. In this equation x is the vector of parameters, f is the objective function we want to minimize (or maximize) and c_i are the individual constraint functions. The variables $\mathcal{I}$ and $\mathcal{E}$ denote the sets of indices for equality and inequality constraints, respectively. [4, p. 3]

$$\begin{aligned} \min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad & c_i(x) = 0, \quad i \in \mathcal{E}, \\ & c_i(x) \geq 0, \quad i \in \mathcal{I}. \end{aligned} \tag{2.1}$$

The next question that comes to mind is: “How can an optimization algorithm find the extremum of an objective function?”. One of the possible answers is Fermat’s Theorem, which states that if a function f has a local extremum (a local maximum or minimum) at a point x_0 , and if $f'(x_0)$ exists, then:

$$f'(x_0) = 0.$$

In other words, if we have reached a (possibly local) extremum, the derivative of the objective function is equal to zero. Several classical optimization algorithms such as the Newton’s method or Gradient Descent (see fig. 2.1) were developed upon this assumption, Gradient Descent in particular to this day remains in widespread use in the field of Machine Learning.

However, as might already be obvious, these algorithms cannot be used when the objective function **isn’t differentiable**, moreover, simpler algorithms such as the Newton’s method only reliably find the local extremum, not the global one.

To address these issues, heuristic methods such as **population-based algorithms** were developed.

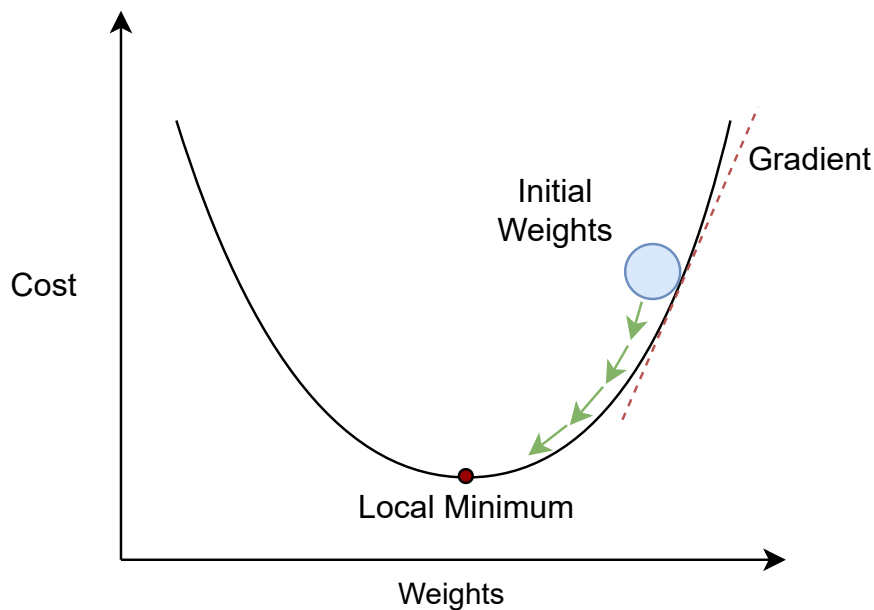


Figure 2.1: An illustration of gradient descent [6], a classical iterative optimization algorithm commonly used in Machine Learning. Notice that at each iteration, the derivative of the objective function defines a descent direction, only a single “candidate solution” is maintained. Population-based methods don’t follow this concept, instead they operate on a “population” of many different “candidate solutions”.

2.2 Population-Based Algorithm

As was already mentioned at the end of the previous section, population-based algorithms differ from classical methods in the fact that they utilize a “population” of “candidate solutions” of an optimization problem, rather than trying to iteratively improve a single one.

Moreover, population-based algorithms usually don’t rely on calculus-based mathematical approaches, rather than that they often are inspired by naturally occurring phenomena such as **biology** or **animal behavior**.

To illustrate this idea, we could for example mention algorithms like the *Grey Wolf Optimizer* (GWO) [7] inspired by the leadership hierarchy and hunting mechanism of grey wolves, *Ant Colony Optimization* (ACO) [8] inspired by the foraging behavior of ants, or the *Artificial Bee Colony algorithm* (ABC) [9] imitating the behavior of a bee swarm.

In simple terms, all population-based algorithms are a **loop**, inside of which individuals are **evaluated** (their fitness is computed using the objective function), the best individuals are then **selected** using a selection mechanism, and finally the selected individuals are used to produce new individuals (high fitness), which are then used to **replace** some of the old (low fitness) individuals. This repeats until a stopping criterion is reached, this criterion is usually a number of **generations** (iterations of the loop).

For a pseudocode description of a generalized population-based algorithm (inspired by [10, p. 26]), see listing 2.1. A summary of key terms associated with population-based algorithms is provided in table 2.1.

Source code 2.1: A generalized population-based algorithm

```
1 population = initialize with random individuals
2 for (i = 0; i < GENERATIONS_COUNT; i++) {
3     1) use objective function to calculate fitness of each individual
4     2) select the "best" individuals
5     3) produce new individuals
6     4) update the population, some old individuals are replaced
7 }
```

Table 2.1: Basic terminology of population-based algorithms

Term	Definition
Problem	An objective function together with its parameters and constraints.
Objective function	A function which assigns a fitness to an individual, i.e. the quality of an individual.
Individual	A single “candidate solution” to our problem.
Population	A set of individuals upon which the algorithm operates, the population size is fixed and does not change between iterations.
Generation	One iteration of the algorithm, the total number of generations is usually used as the stopping criterion.
Search space	A set of all possible solutions (individuals), it contains every individual that could possibly exist.
Diversity	A measure of how spread out individuals are across the search space. The more diverse the population, the less likely the algorithm is to get stuck in a “local extremum trap”.
Local extremum	A local maximum or minimum, it does not represent the best overall solution.
Global extremum	The goal we want the algorithm to reach, it represents the best solution across the entire search space.
Exploration	The tendency of the algorithm to search in new, unexplored areas of the search space, it can lead to improved diversity.
Exploitation	The tendency of the algorithm to keep “exploiting” already known good solutions. It keeps the algorithm in the same region of the search space.

2.3 Population-Based Solver

The algorithm itself, as described in section 2.2, handles the most computationally demanding part of the problem-solving process, however, it doesn’t provide us with all the necessary infrastructure that we might need.

The **solver** is a tool that can automatically choose an algorithm for our problem (or multiple algorithms, in the case of metasolving), then run the algorithm and return the best individual. It thus serves as a unified interface, improving ease of use for the end user.

The figure 2.2 shows us a block diagram of how a typical population-based solver might be implemented.

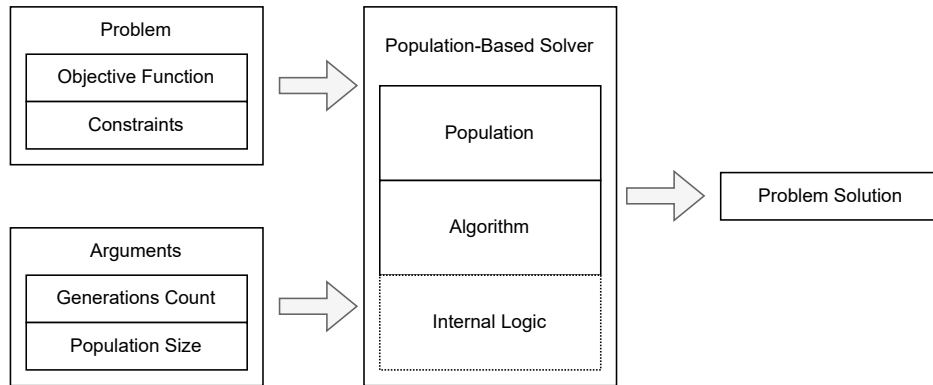


Figure 2.2: Example of a population-based solver, the solver has two types of inputs: the problem containing both the objective function and the constraints, and an arguments structure, containing the generations count and population size. The arguments structure can possibly include other fields, such as for example initial population members. After running the solver, the best population member is chosen and returned back to the user.

2.4 Types of Population-Based Algorithms

What all population-based algorithms have in common is that they're **metaheuristic**. Heuristic meaning that they produce acceptable solutions to an optimization problem through trial and error, the algorithm achieves this by introducing randomness, thus population-based algorithms are stochastic, rather than deterministic. [11] [12, p. 7]

The word “meta” could be interpreted as “high-level” or “general”, that is, metaheuristic methods are applicable to a wide range of different problems, instead of being closely specialized to solve only a specific class of problems. This is accomplished by efficiently exploring the problem’s search space while balancing the ratio between exploration and exploitation (see table 2.1). [13]

While all population-based algorithms share these common characteristics, there are several key aspects in which they differ from each other, namely: representation of the individual, the strategy that’s used to create new individuals, or the selection strategy used to replace old individuals.

Based on these differences, we can classify population-based algorithms into two main categories: **Evolutionary Algorithms** and **Swarm Intelligence**.

2.4.1 Evolutionary Algorithms

EAs are a class of algorithms based on the theory of **Darwinian evolution**, individuals of the population (parents) produce offspring (children), **natural selection** then pushes this system to evolve better individuals by ensuring that only the strongest individuals (of high fitness) survive until the next generation.

There are two variation operators that allow us to produce offspring, the first is a binary operator called **crossover**¹, which in a predetermined way combines parts of both parents to produce offspring. The second is a unary operator called **mutation**, this operator randomly changes some part of the individual, it improves population diversity and prevents the algorithm from getting stuck in a local minimum.

The algorithm uses a **selection strategy** to select individuals from which offspring will be produced. There are many different types of selection strategies, the most common ones are: roulette wheel, tournament and elitism.

Roulette wheel is a strategy where individuals are assigned a probability of being chosen (their “slice” of the roulette) depending on their fitness, this approach thus gives all individual a chance to reproduce, albeit only in proportion to their fitness.

Tournament selection randomly chooses a small subset of the population, the fittest individuals in this subset are then selected to produce offspring.

In contrast to the two previous strategies, **elitism** doesn’t allow low fitness individuals to reproduce, it only allows reproduction among the best individuals of the current generation.

The various subcategories of EAs differ primarily in the data structure they use to represent an individual of the population, other differences include which variation operators are used and how selection is performed.

Genetic Algorithms

GAs were first developed by John Holland in the 1970s [14], typical for GAs is that the individual, usually called a “chromosome”, is represented as a fixed-length binary string.

Among the first genetic algorithms was the so called **Simple Genetic Algorithm** (SGA), this algorithm thus forms the basis of any other GAs. [10, p. 99]

The SGA begins by creating a population of μ random binary strings, the fitness of each individual is then evaluated and a roulette wheel is used to select μ individuals into an intermediary population (duplicates are allowed). These individuals are shuffled into pairs and 1-point crossover is performed, where a random cut point is chosen with probability p_c . Each bit of every individual in the intermediary

¹Sometimes also called “recombination”

population is then flipped with probability p_m . This cycle repeats, discarding the old population and replacing it with the new one each generation, until the required number of generations is reached.

For pseudocode of the SGA see listing 2.2, to better visualize the operations performed on a chromosome refer to figures 2.3 and 2.4.

Source code 2.2: The Simple Genetic Algorithm (SGA)

```

1 population = MU random binary strings
2 for (i = 0; i < GENERATIONS_COUNT; i++) {
3     1) Evaluate fitness of each individual
4     2) Select MU individuals into intermediary population via roulette
       wheel (duplicates allowed)
5     3) Shuffle intermediary population into pairs
6     4) For each pair, choose a random cut point with probability P_C and
       perform 1-point crossover
7     5) For each individual, flip each bit with probability P_M
8     6) Replace old population with intermediary population
9 }

```

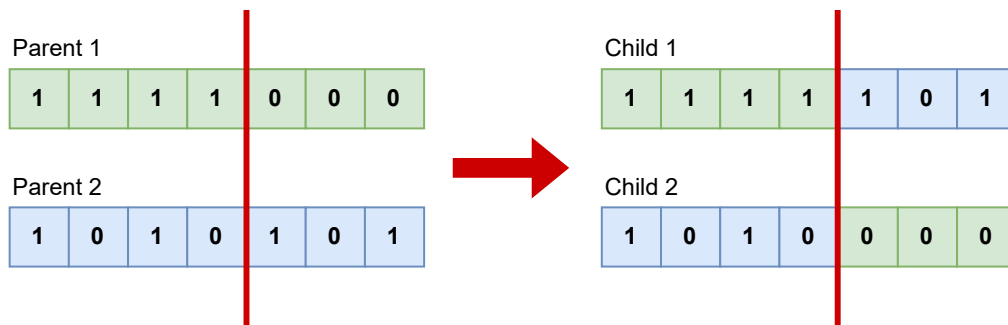


Figure 2.3: Illustration of a one-point crossover, the SGA performs this crossover with the probability p_c , otherwise the children stay the same as their parents. The crossover point is chosen randomly using a uniform RNG.

Evolutionary Strategies

Evolution strategies were invented by Hans-Paul Schwefel and Ingo Rechenberg in the 1960s, originally developed as an aerodynamic optimization method, optimizing the shapes of wings to reduce drag and increase thrust. Traditional optimization methods such as gradient-based techniques often failed in this context due to noisy measurements and the presence of many local minima. Schwefel's idea was inspired by the process of **biological mutation** and **selection**, where small random modifications are applied to the parameters of candidate solutions. Those performing better are then selected to form the next generation. [15]

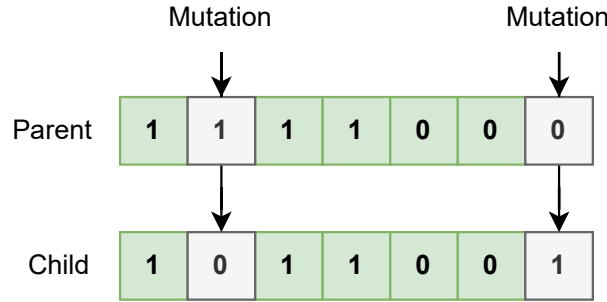


Figure 2.4: A mutation performed on two bits of the parent chromosome. SGA mutates each bit with the probability p_m .

So, the key features of ES are the usage of mutation as the primary variation operator and real-valued vectors² being used to represent the individual. A special feature of ES is the self-adaptation of mutation step sizes, the step size is stored in the individual's vector and is evolved as part of it.

Classical evolutionary strategies can be divided into four categories: $(1 + 1)$, $(1, 1)$, $(\mu + \lambda)$ and (μ, λ) . Each of these categories is explained in the following paragraphs. As for modern state-of-the-art evolutionary strategies, the CMA-ES algorithm [16] is worth mentioning.

$(1 + 1)$ is the simplest possible ES, its population consists of two individuals, the parent and the result of its mutation (offspring), the parent is replaced by the offspring only if the offspring's fitness is better. A modification of this approach is the $(1 + \lambda)$ ES, in this scenario λ offsprings are produced, the fittest individual is chosen among the offsprings and the current parent.

$(1, 1)$ is similar to the $(1 + 1)$ ES, except that the offspring **always** replaces the parent (even if it's worse). This strategy can be modified to form the $(1, \lambda)$ ES, where λ offsprings are produced from the parent, the fittest individual is then chosen **only** among these offsprings.

$(\mu + \lambda)$ ES maintains a population size of μ , from these μ parents, λ offsprings are produced using mutation. The next generation always consists of μ individuals, these are chosen among both μ (parents) + λ (offsprings), the same elitist strategy of selecting only the highest-fitness individuals is used.

(μ, λ) is the same as $(\mu + \lambda)$ ES, except that the next generation is selected only among the λ offsprings.

The main four ES categories are summarized in table 2.2, figures 2.5 and 2.6 visualize simple $(1 + 1)$ and $(2 + 4)$ evolutionary strategies.

²That is to say, an array of double-precision or single-precision floating point numbers

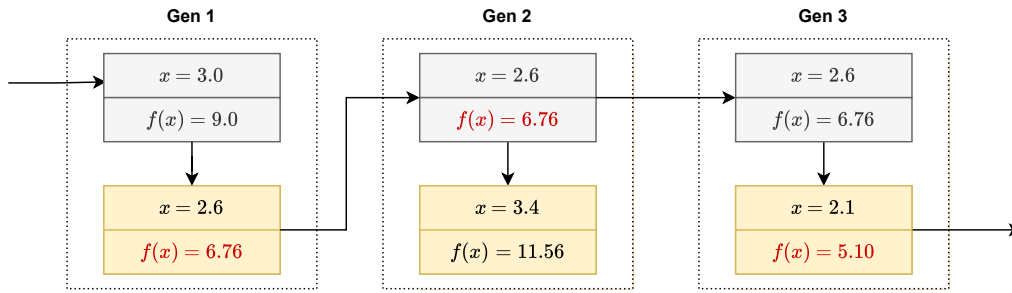


Figure 2.5: An (1 + 1) ES, parents are designated by gray background, offspring by yellow background. Individual with the highest fitness in each generation is highlighted in red.

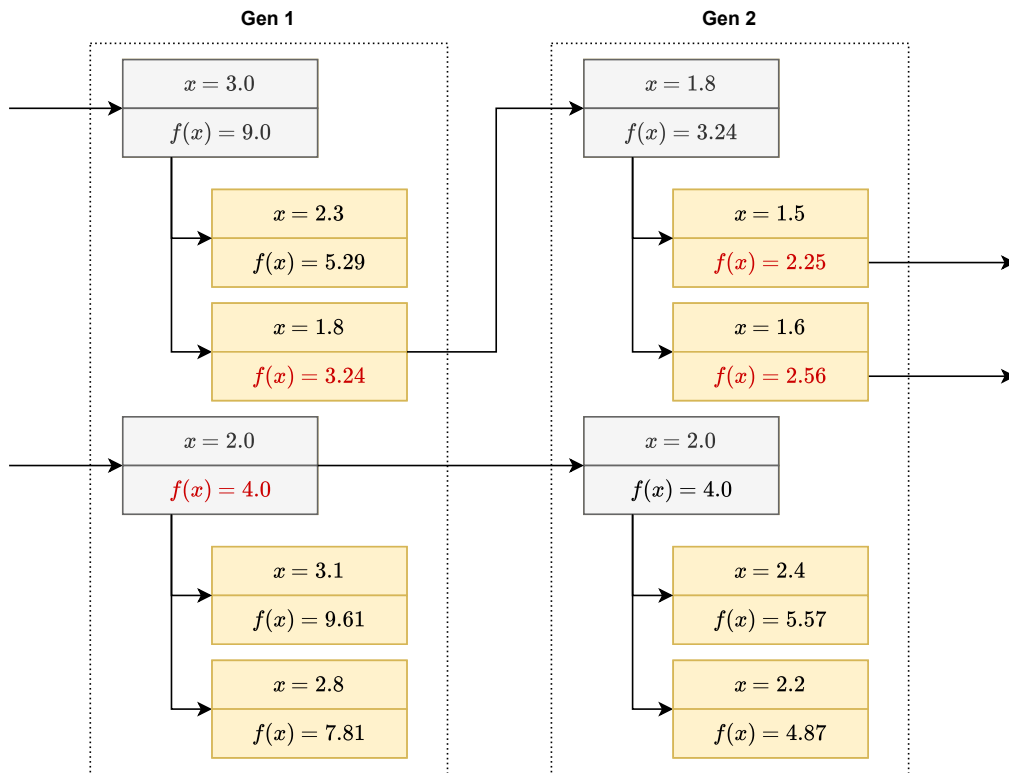


Figure 2.6: An (2 + 4) ES, so a variant of $(\mu + \lambda)$ ES. Notice the increased rate of convergence as opposed to the (1 + 1) ES example. It can also be seen that this type of algorithm is highly parallelizable, both the mutation step and the selection step can be parallelized across many GPGPU or CPU cores.

Table 2.2: Overview of Evolution Strategy types

Type	Population	Description
$(1 + 1)$	1 parent, 1 offspring	Offspring replaces parent only if fitter.
$(1, 1)$	1 parent, 1 offspring	Offspring always replaces parent regardless of fitness.
$(\mu + \lambda)$	μ parents, λ offspring	Next generation selected from parents and offspring combined.
(μ, λ)	μ parents, λ offspring	Next generation selected from offspring only, parents discarded.

Differential Evolution

DE is an algorithm that was introduced by Kenneth Price and Rainer Storn in a series of research papers published from 1995 to 1997. [17, 18] Like many other GAs, DE represents the population individuals as a vector of real numbers, the core difference is that DE uses a special form of the mutation operator called “differential mutation”.

Differential mutation is a self-adapting mechanism, similar to how ES “evolves” its mutation step, DE however doesn’t need to store any additional information to achieve this. Differential mutation is performed by randomly choosing three vectors: a base vector x_1 and additional two vectors x_2, x_3 . The mutant vector v is then produced by adding the scaled difference of x_2 and x_3 to the base vector x_1 (see equation in step 2 below). This ensures that **exploration** is preferred when population is spread out across the search space (big difference between vectors = big mutation step), the algorithm then naturally switches to **exploitation** once all of the individuals cluster in a promising region, this happens because all of the individuals are close to each other, producing a small difference between x_2 and x_3 .

As was noted in the 1997 paper [18, p. 2], this bears similarity to the simulated annealing algorithm, which at first makes big changes to the individual, and then slowly reduces the amount of these changes as time increases. This serves as an efficient mechanism preventing the algorithm from being stuck in a local minimum.

DE requires only three input parameters: NP (Population size), F (Scaling factor) and CR (Crossover probability). The algorithm starts by initializing a population of NP random vectors, then the algorithm’s inner loop performs the following four steps **for each individual in the population**, until the required number of generations is reached:

1. Randomly select three distinct parents x_1, x_2 and x_3

2. Perform differential mutation, creating a mutant vector v_i like so:

$$\mathbf{v}_i = \mathbf{x}_1 + F \cdot (\mathbf{x}_2 - \mathbf{x}_3)$$

3. Perform a uniform (binomial) crossover creating a trial vector³ u_i like so:

$$u_{i,j} = \begin{cases} v_{i,j} & \text{if } \text{rand}_j[0, 1] \leq CR \text{ or } j = j_{\text{rand}}, \\ x_{i,j} & \text{otherwise} \end{cases}$$

4. Perform an elitist one-to-one replacement, the child u_i is only compared to its parent x_i :

$$\mathbf{x}_i \leftarrow \begin{cases} \mathbf{u}_i & \text{if } f(\mathbf{u}_i) \leq f(\mathbf{x}_i), \\ \mathbf{x}_i & \text{otherwise} \end{cases}$$

As we can see, this loop can be parallelized, because the mutation, crossover and replacement steps are independent for each individual.

Genetic Programming

GP is a branch of evolutionary algorithms introduced by John Koza in 1992 [19]. GP differs highly from other forms of GAs, the individuals are represented by **tree structures** of dynamic size, meaning that they can shrink or grow, these can be the parse trees of a computer program or a mathematical expression.

GP is given examples of inputs and desired outputs, for example “For input $x = 4$, produce output $y = 16$, for $x = 5$ produce $y = 20$, ...”, in this specific case the algorithm’s goal would be to find the parse tree of a program multiplying the input x by 4.

The typical workflow for other GAs would be to find the best input to a known objective function, GP on the other hand constructs a function (parse tree of a program) that maps inputs to desired outputs. This aligns GP more closely with machine learning, than mathematical optimization.

The following steps are a high level summary of how a typical GP algorithm operates:

1. Randomly initialize a population
2. Evaluate fitness of each individual (by e.g. checking semantics or executing)
3. Randomly select parents

³Randomly chosen index j_{rand} guarantees that at least one component of the trial vector comes from the mutant (so the trial vector never exactly duplicates the parent)

4. Perform crossover **or** mutation
5. Replace the current population
6. Repeat until the stopping criterion is met

As shown in step 4, unlike most evolutionary algorithms GP performs crossover *or* mutation, not crossover *and* then mutation. The chance of a mutation is usually set to small values such as 5% or in some cases mutations are disabled completely. [10, p. 106]

Crossover is thus the dominant variation operator in GP algorithms, it's performed by swapping sub-trees of two individuals. Mutations are performed by replacing a randomly chosen sub-tree by a newly generated random sub-tree or by replacing a single node of the tree by another one.

For this to function properly, GP algorithms need to maintain a relatively large population (in the sizes of thousands). One of the related problems is the so called “bloat” or “survival of the fittest” phenomena, where individuals of the population tend to grow unnecessarily large trees as the algorithm progresses, one of the possible solutions is to introduce a limit on the maximal allowed tree size. [10, p. 107]

Figure 2.7 shows parse tree of a computer program, interior nodes are the language's *non-terminal symbols*, whereas the leaves are the *terminal symbols*. The relationship between these symbols is defined by the language's grammar.

If we were to compare a GP algorithm to the translation stages of a programming language compiler, GP de facto replaces the **parsing** and **semantic analysis** stage. GP always produces a syntactically valid tree representation of the program, because it's variation operators directly use the language's context free grammar to create or modify the individual. To evaluate the fitness of each individual, the semantics of the program are checked, as an optional step, the GP algorithm can also interpret the program⁴ and see what output is produced.

As opposed to this concept, today's mainstream LLMs don't operate on the level of the language's grammar and syntax rules, they generate the program probabilistically purely based on what was learned through the training data. This means that GP algorithms are a more “formal” method of generating program code, as they by design enforce the syntactical and semantical correctness of the produced program.

As figure 2.8 shows, LLMs operate more like a real user would: producing possibly syntactically or semantically incorrect code and relying on the compiler infrastructure to detect any problems. Whereas GP algorithms bypass a large portion of the compiler infrastructure entirely, needing only the final phases of it such as the code generator, assembler and linker to produce an executable file.

For more information about formal language theory and compilers see: [20].

⁴Or fully compile it if we're using a compiled language

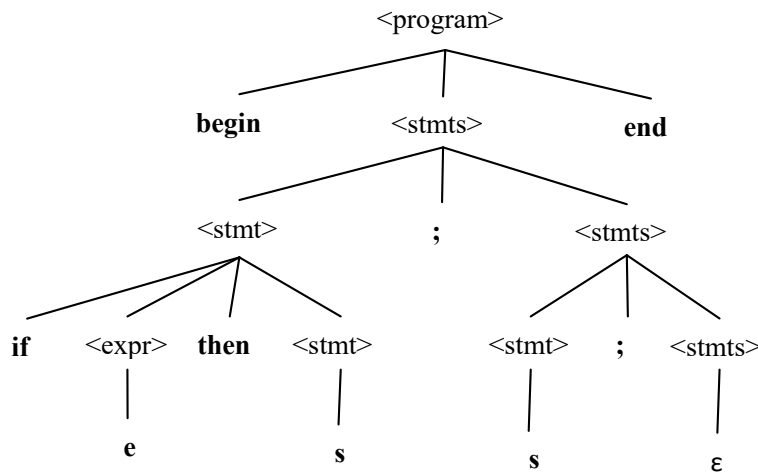


Figure 2.7: Example parse tree of a computer program. Source: [21]

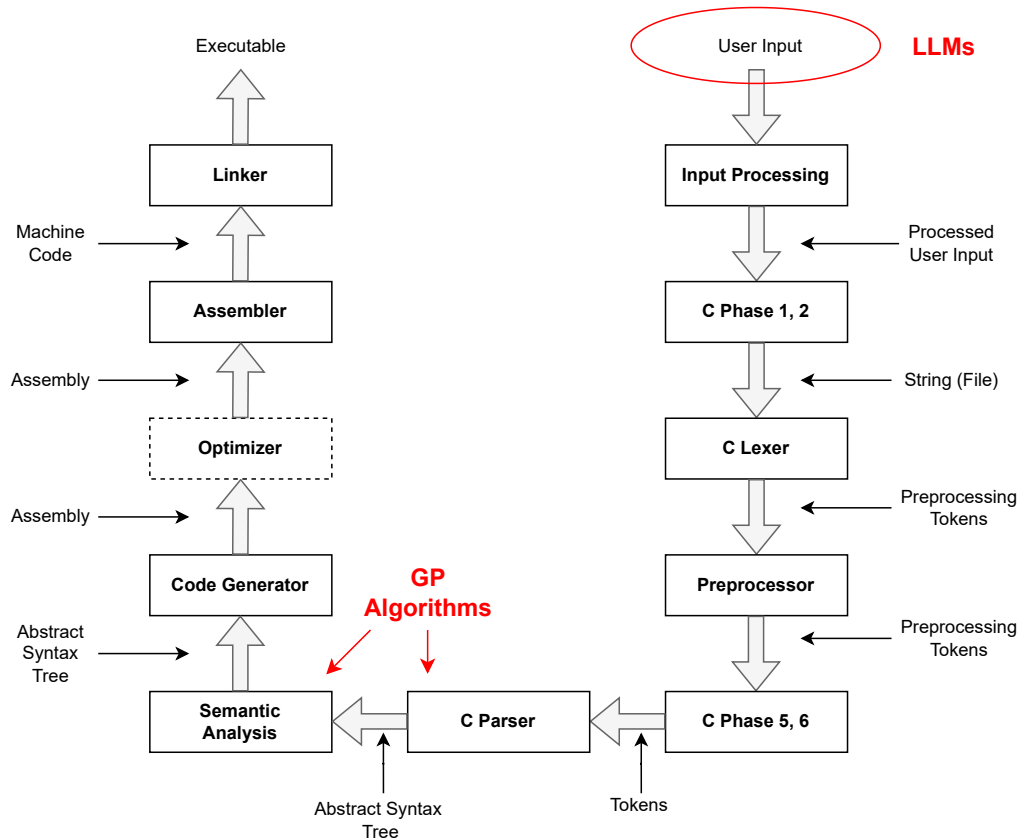


Figure 2.8: The translation phases of a typical C language compiler, in accordance with ISO/IEC 9899 (C99), for a detailed explanation of these phases see: [22, p. 9]. The image highlights at which stage of the translation process is genetic programming situated, as opposed to large language models.

2.4.2 Swarm Intelligence

Swarm intelligence (SI) is a specific form of population-based algorithms, unlike EAs, SI doesn't use the concept of evolution, meaning that SI algorithms usually maintain a static population of "agents" or "particles" where the individuals are never replaced, instead each iteration of the algorithm only updates them. This also means that there's no crossover or a selection mechanism. SI algorithms might additionally store global information (such as position of the best particle in PSO), this information allows the individuals to indirectly coordinate their actions.

It could simply be said that SI explores the search space by "performing adjustments on agents through their local interaction and shared information" whereas EAs explore the search space by a repeated "death and replacement cycle" of the individuals.

The following algorithms are typical examples of SI: *Particle Swarm Optimization* (PSO), *Ant Colony Optimization* (ACO), *Artificial Bee Colony* (ABC) or *Grey Wolf Optimizer* (GWO). This section will focus on providing a short explanation of the first two: PSO and ACO.

Particle Swarm Optimization

PSO was first introduced in a 1995 paper by James Kennedy and Russell Eberhart. The original paper states that PSO was initially developed as a simulation of collective animal behavior such as bird flocking. This model was later generalized into an optimization method based on "particles moving through a search space". [23]

PSO's population consists of many particles defined by their **position** and **velocity**. PSO additionally stores the **personal best position reached by each particle**, together with the **global best position**, this information is then used to adjust the particle's velocity and position, this is the key step which allows PSO to both explore the search space and at the same time attract the particles towards a known global minimum.

Formally the particle could be defined like so:

$$\text{Particle } i = (\mathbf{x}_i, \mathbf{v}_i, \mathbf{p}_i)$$

Where $\mathbf{x}_i$ represents the current position, $\mathbf{v}_i$ is the current velocity and $\mathbf{p}_i$ is the personal best position found by this particle.

The algorithm first initializes a population of particles with random starting positions and velocities, then the algorithm's main loop begins:

1. Fitness of each particle is evaluated
2. "Personal best" and "global best" positions are updated

3. The velocity formula is applied to calculate a new velocity for each particle (see eq. 2.2)
4. Each particle's position is updated according to their new velocity:

$$\mathbf{x}_i^{new} = \mathbf{x}_i^{old} + \mathbf{v}_i^{new}$$

5. Loop repeats until the stopping criterion is met

The equation below shows how the velocity of each particle is updated, it contains three hyperparameters which are specified by the user:

- w is the “inertia weight”, i.e. how much the particle resists changing its **current path**
- c_1 is the “cognitive coefficient”, higher number makes the particle be drawn towards its **personal best** ($\mathbf{p}_i$) position
- c_2 is the “social coefficient”, higher number makes the particle be drawn towards the **global best** ($\mathbf{g}$) position

r_1 and r_2 are two random components, adding non-determinism into the system. The new velocity is then computed as a sum of the particle's momentum, together with the *cognitive* (personal) and *social* (global) parts.

$$\mathbf{v}_i^{new} = \underbrace{w\mathbf{v}_i^{old}}_{\text{Momentum}} + \underbrace{c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^{old})}_{\text{Cognitive}} + \underbrace{c_2 r_2 (\mathbf{g} - \mathbf{x}_i^{old})}_{\text{Social}} \quad (2.2)$$

Ant Colony Optimization

First described in 1996 by Marco Dorigo [24], this algorithm is inspired by the collective foraging behavior of ants. Individuals cooperate on finding the best solution by depositing and following **pheromone trails**. Unlike PSO, it is primarily used to solve graph optimization problems, such as the Traveling Salesman Problem (TSP).

The following steps describe how the ACO algorithm could be used to solve a shortest path problem:

1. Each edge of the graph is assigned a small uniform amount of pheromone, and a population of ants is placed at the starting node
2. Each ant constructs a solution⁵ by moving from node to node, selecting the next node probabilistically based on pheromone levels and a problem-specific heuristic

⁵Keep in mind that in this context the term “solution” means any valid path from the starting node to the destination node

3. Once all ants have constructed a complete solution (complete path), the fitness of each solution is evaluated (determined by total path length)
4. Pheromone amounts are updated: all values are reduced by a predetermined evaporation factor, and better (high fitness) solutions receive additional pheromone
5. Repeat from step 2 until the stopping criterion is met

2.5 The Number of Objective Functions

So far we have only discussed problems (and algorithms) which have a single objective function. However, problems which have multiple possibly mutually conflicting objective functions also exist. This section explains the key differences between them.

2.5.1 Single-Objective optimization

Single objective (SO) problems have a single objective function that returns a real number, formally it can be written as:

$$f(x) \rightarrow \mathbb{R}$$

This has several consequences, perhaps most importantly, it is trivial to compare the fitness of two individuals, let's say we have individuals A and B , in this case we can simply compare $f(A) > f(B)$ to determine if the fitness of A is higher than that of B . That is to say that the output values of an objective function $f(x)$ have a **total order**.

An SO algorithm thus converges towards a single “best value”, always aiming to find the global optimum of an objective function, the solution is always a single point x .

The following section explains how this changes when additional objective functions are added.

2.5.2 Multi-Objective optimization

Multi objective (MO) problems use a “vector-valued” objective function $F(x)$, i.e. a function which returns a vector of real numbers, where each value is produced by an objective function $f_i(x)$, it can be written in the following form:

$$F(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \\ \vdots \\ f_n(x) \end{bmatrix} \in \mathbb{R}^n$$

With MO optimization the situation changes completely if we want to for example compare the results of $F(A) = (2, 1, 3)$ and $F(B) = (1, 3, 2)$, there is **no total ordering**. It is impossible to decide if $(2, 1, 3) > (1, 3, 2)$, elements 2 and 3 are bigger than the corresponding values in the second vector, however element 1 is smaller than in the second vector. Thus, the first vector is neither bigger, smaller or equal to the second one.

Now, let's consider the situation where $F(A) = (3, 2, 4)$ and $F(B) = (1, 2, 2)$, in this case we can say that $(3, 2, 4)$ is a superior solution to $(1, 2, 2)$ because each of A 's fitness values are better⁶ than B 's values. In MO terminology we'd say that the solution A **dominates** solution B .

This is the so called concept of **Pareto dominance**, formally, a solution A is said to dominate solution B if it satisfies the following conditions:

1. A is at least as good as B across all objectives
2. A is strictly better than B in at least one objective

Consequently, MO problems **don't** have a single "best" solution, objective functions can compete among each other, therefore there won't be a single solution vector that'd maximize all of them. Instead, there will be a set of solution vectors that are not dominated by any other feasible solution in the search space, these vectors are what is known as the **Pareto front**.

Therefore the goal of an MO algorithm is to find (or approximate) the problem's Pareto front. It is up to the user to inspect the trade-offs of each solution and select the one that best fits their application.

Population-based algorithms are particularly well suited to solve MO problems, precisely because they operate on a population of many candidate solutions, this allows them to iteratively evolve the entire population towards the Pareto front.

One of the most well known MO evolutionary algorithms is NSGA-II [25], this algorithm is discussed in later chapters of this thesis.

2.6 Metasolving

Metasolving can be summarized as an attempt at using multiple different optimization algorithms to solve a problem, as opposed to the traditional approach where only one specific algorithm is used.

The goal of a metasolver is to improve some performance or quality metric, such as diversity of solutions or the collective time it takes to approximate a satisfactory solution.

⁶In this example we're assuming that bigger number means better fitness value

To run multiple different algorithms at once, metasolvers are usually implemented as parallel or distributed applications, the most common design pattern in this regard is the **island model**.

2.6.1 The Island Model

The island model is a collection of “islands”, where each island maintains a population of individuals and executes an algorithm.

A **topology** describes how the islands are connected together, most common is the fully connected topology (see fig. 2.9), however other topologies such as ring or star can also be used. Each edge represents a communication channel, over which islands can **exchange population individuals**, this process is called **migration**.

Naturally, in a distributed island model each island corresponds to a single computational node of our cluster, in a parallel implementation each island would be mapped to a separate thread or a process. Distributed islands communicate by sending messages over a TCP/IP network, whereas (local) parallel islands communicate via shared memory or IPC.

Often, an additional centralized element called **archipelago** is added, its purpose is to perform tasks such as scheduling work or distributing global data to islands.

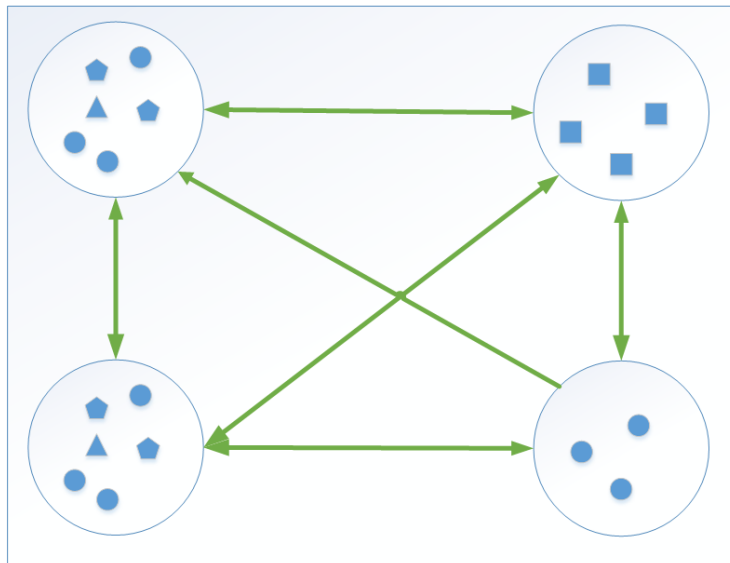


Figure 2.9: An example of a fully-connected island model, different shapes represent individuals migrating from one island to another. Source: [26]

2.6.2 Overview of a Metasolver

Based on sources [27] and [28] we can establish that a typical metasolver implementing the island model operates as a loop repeating the following steps:

1. Initialize each island with a starting population
2. Run an algorithm on each island
3. Migrate individuals between islands according to topology
4. Assess performance of islands, adjust their parameters as needed
5. If a stopping criterion is met, collect the solutions of each island and determine the best one, otherwise continue with step 2

As highlighted by [27], migration is an important part of this process, because it increases the diversity of individuals in each island and thus allows the algorithms to **cooperate**. Without migration, running the algorithm which works the best for our specific problem on each island will always outperform a metasolver system where each island runs a different algorithm.

Some of the disadvantages of the island model are the added overhead of communication, sensitivity to parameter changes (such as migration rate) and a need to dynamically adjust the island's parameters depending on performance of the algorithm and hardware.

This means that the solver becomes inefficient if migration rate is too high, as the overhead of sending and waiting for messages will become higher than the time spent performing useful computations.

Benefits of metasolving include the high probability that a global minimum will be found, each algorithm explores the search space differently, so it's unlikely that all of them will get trapped in the same local minimum. Another benefit is that a metasolver can provide a satisfactory solution to almost any kind of an optimization problem, this is because even if one of the algorithms is ill-suited to solve this particular problem, the other algorithms might still perform well.

Distributed Computing

3

In this chapter, we'll evaluate several historical concepts in the field of distributed computing, namely **Parallel Virtual Machine** (PVM) and **Active Networks**. The goal of this chapter is to explore different ways of how a distributed application can be implemented, it serves as a stepping-stone for chapter 4, where we provide a theoretical outline of what approaches should be taken to implement a solver as a distributed application.

3.1 Parallel Virtual Machine

PVM was originally created by Oak Ridge National Laboratory (ORNL) in 1989, it remained in internal use at ORNL until February 1991, when the software was rewritten and publicly released by the University of Tennessee. [29]

The era when PVM began development (mid 80s to early 90s) can be characterized by the increasingly widespread adoption of the Personal Computer and the rising capabilities of consumer-grade hardware.

To put this into perspective, the Intel 386 microprocessor released in 1985 consisted of 275 thousand transistors and was capable of reaching clock speeds of 12 to 40 MHz. By the mid-1990s, CPUs such as the Intel Pentium Pro were reaching counts of 5.5 million transistors and clock speeds of up to 200 MHz. This was an outstanding **20x increase in transistor density** and about **10x increase of clock speeds** in a single decade. [30]

Educational and research institutions (such as the ORNL), thus suddenly became in possession of a large amount of network-connected PCs, which made researchers think about the possibilities of utilizing these shared computational resources by forming distributed computer clusters. PVM was one of the first attempts at implementing this idea.

3.1.1 Architectural Components

PVM consisted of four primary components: PVM Daemon (PVMD), libpvm, PVM Console and XPVM. [31]

PVMD is a daemon running on every computer that is currently a part of the cluster. PVM used a **master-slave** architecture (see fig. 3.1), meaning that the first started PVMD instance became a master, it then used remote execution (e.g. SSH) to start slave PVMD instances on other computers in the network.

The master thus acted as a centralized control element, its features included managing a global hosts table, adding and deleting hosts, providing centralized task scheduling and collecting logs from all running tasks.

Libpvm was a C (or Fortran) library against which every program utilizing PVM was linked. It acted as a user-side component allowing applications to interact with the locally running PVM Daemon, allowing them to use:

- Message send / receive commands (`pvm_send`, `pvm_recv`)
- Process management commands (`pvm_spawn()`, `pvm_kill()`, ...)
- Configuration commands (`pvm_addhosts()`, `pvm_delhosts()`)
- And many more such as: fault notification, group communication, ...

PVM console (and its GUI equivalent XPVM) were standalone tools allowing the user to manage the cluster. It allowed the user to start the cluster by using console commands such as `pvm myhosts.txt` where `myhosts.txt` was a file containing the addresses of computers on which we want to start the PVMD.

3.1.2 Communication Model

As mentioned in the section above, the master daemon kept track of which computers are currently part of the cluster in a **hosts table**. This table was shared with all slaves and any changes to this file were broadcasted to the network.

This meant that slaves could communicate peer-to-peer, as they knew the address of each other, eliminating the need for master to proxy communication between them, this peer-to-peer communication was done via TCP. PVM allowed for either synchronous or asynchronous send / receive communication.

Additional UDP protocol (see fig. 3.2) was used for management purposes, it included a keep-alive mechanism which informed the master of any slaves which might have unexpectedly disconnected.

Serialization or deserialization of messages was handled by libpvm, providing the user application with pack and unpack commands.

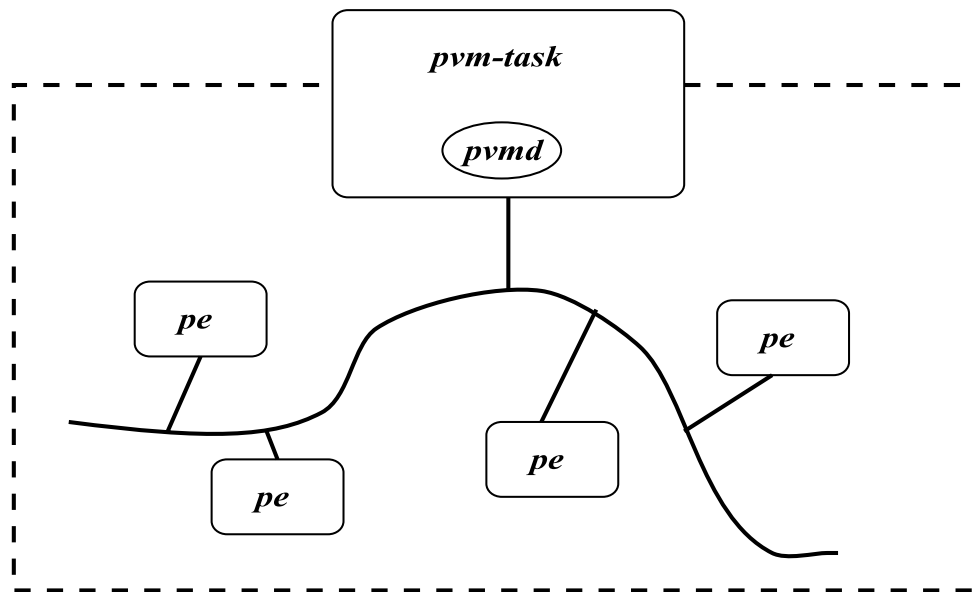


Figure 3.1: PVM architecture depicted as a number of *processing elements* (“*pe*”) interconnected over a network. The master PVMD has the ability to spawn slave PVMDs on any of the “processing elements”. Source: [32]

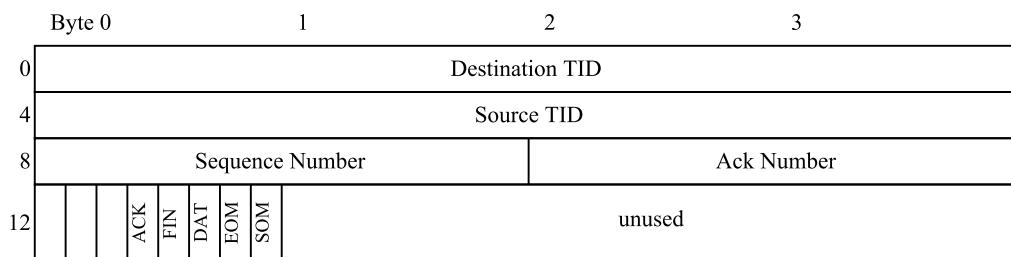


Figure 3.2: Header of the PVMD UDP protocol, using Task IDs for identification and additional fields to ensure the reliable transfer of messages. Source: [31, p. 113]

3.1.3 Allocation of Work

Work in PVM is represented as **tasks**, master spawns a slave using `pvm_spawn()`, this call returns a Task ID (TID) uniquely identifying the slave node. Master can then assign work to a specific worker using the `pvm_send(tid, content)` command.

A commonly utilized paradigm in PVM is the so called **pool-of-tasks**, where the master divides work into small portions, and then assigns them to workers as soon as they send back the completed piece of previous work. This provides a primitive form of load-balancing, as faster nodes will by design be allocated more pieces of work than slower nodes. [31, p. 35]

A “hello world” PVM master-slave program is shown in listings 3.1 and 3.2.

Source code 3.1: Example of a PVM master, which starts a single slave and then blocks until a “hello” message is received

```

1  /* hello.c – Master program PVM example */
2  #include <stdio.h>
3  #include "pvm3.h"
4
5  int main() {
6      int cc, tid, msgtag;
7      char buf[100];
8
9      /* Obtain this task's unique identifier (TID) from PVM */
10     printf("I'm task %t\n", pvm_mytid());
11
12     /* Spawn one instance of the 'hello_other' slave program */
13     cc = pvm_spawn("hello_other", (char **) 0, 0, "", 1, &tid);
14
15     if (cc == 1) {
16         /* If successful, wait for a message from the slave */
17         msgtag = 1;
18         pvm_rcv(tid, msgtag); /* Blocking receive */
19         pvm_upkstr(buf);      /* Unpack the received string */
20
21         /* Print the message and the slave's task ID */
22         printf("From %t: %s\n", tid, buf);
23     } else {
24         printf("Can't start hello_other\n");
25     }
26
27     /* Disconnect this task from the PVM virtual machine */
28     pvm_exit();
29     return 0;
30 }

```

Source code 3.2: A PVM slave, which receives its parent TID, constructs a message, and sends it back to the master

```
1 /* hello_other.c – Master program PVM example */
2 #include "pvm3.h"
3 #include <string.h>
4 #include <unistd.h>
5
6 int main() {
7     int ptid, msgtag;
8     char buf[100];
9
10    /* Get the task ID (TID) of the parent (the master) */
11    ptid = pvm_parent();
12
13    /* Create a message string containing the host name */
14    strcpy(buf, "hello,_world_from_");
15    gethostname(buf + strlen(buf), 64);
16
17    /* Send the message back to the master */
18    msgtag = 1;
19    pvm_initsend(PvmDataDefault); /* Initialize send buffer */
20    pvm_pkstr(buf);                /* Pack the string into the buffer */
21    pvm_send(ptid, msgtag);        /* Send the message to master */
22
23    /* Detach this task from PVM and exit */
24    pvm_exit();
25    return 0;
26 }
```

3.1.4 Discovery of Executable Code

As we can see in listing 3.1, master specifies the name (or path) of a slave program in the `pvm_spawn` command. The executable file must be present in the slave's filesystem for this to work, as PVM doesn't provide any facilities for transfer or management of executable code.

By default PVM looks for executables in `$HOME/pvm3/bin/$PVM_ARCH`, alternatively we can re-configure this path and use e.g. NFS to make executable files available to slaves over the network.

3.2 Active Networks

“Active Networks” was a research program initiated by DARPA in the mid 1990s [33, p. 1], the core goal of which was to make networks more flexible by **encapsulating** a packet's data together with the code that's then used to process this packet.

This approach would've allowed rapid deployment of new network protocols, as each network device would be able to on-the-fly receive the protocol's implementation. This is in contrast to traditional IP networks, where each device must already support the underlying protocol before being able to accept its packets. Active networks would for example never run into the IPv4 and IPv6 incompatibility problem, as each device would be able to instantly "learn" a new version of the protocol. [34]

Another idea behind active networks was the ability of networking devices to process or perform computations on packet data as it's transmitted over the network. Each device along the path could, for example, transcode part of the packet's data, so that it was already in the correct format by the time it reached the destination. This was essentially a primitive form of distributed computing. [34]

Active networks as a concept ultimately failed due to security concerns and poor performance, however they did serve as an inspiration for modern areas of research such as Software-Defined Networking (SDN).

3.2.1 Architectural Components

Each implementation varies slightly in the naming and functionality of its components, however generally they consist of the four foundational parts described below. [33, 35, 36, 34, 37]

Capsule is the data unit used for communication in active networks, it consists of two primary parts: data and a code identifier (or a fragment of the code).

Execution Environment is the environment which runs the code to process each capsule, it was most commonly implemented as an interpreter of Java bytecode.

NodeOS is an operating system of the networking device (e.g. router, switch, server, ...) it can be either a general-purpose OS such as GNU Linux or MS Windows, or a specially written operating system (like for example Cisco IOS in IP networking). It contains a sandbox in which the execution environment runs and provides an API layer for it to access the node's hardware.

Active Applications are a set of long-running processes which inject capsules into the network, receive capsules or register new protocols. It could be said that they act as the "end-user" system.

3.2.2 Communication Model

Communication in active networks is handled purely by passing capsules from node to node, each node upon reception performs the actions specified by the capsule's code. The program code can be either directly embedded in the capsule, or it can only contain a hash of the code, which can then be used by the node to request a copy of it.

The steps which each node performs upon capsule reception can be summarized as:

1. Link layer receives the capsule, passes it to the node runtime
2. The node checks its local cache for a program code matching the hash that's specified in the capsule, if it doesn't have this program available locally, it initiates a code distribution request
3. The node creates a temporary sandboxed execution environment for the capsule
4. Node evaluates the capsule according to the program
5. Evaluation produces zero or more capsules for transmission to other nodes
6. The execution environment is destroyed

3.2.3 Discovery of Executable Code

It is obvious that it's inefficient to send the protocol's code in each capsule if the protocol doesn't change often. For that reason, most active network implementations use a **code distribution protocol**, which allows nodes to download code on demand.

Each node contains a cache in which executable code is stored, this cache is indexed by the code's hash. The cache can be implemented as LRU (Least Recently Used), to prevent flood attacks exhausting the node's disk space.

A typical active networks code distribution protocol (see fig. 3.3) consists of the following steps:

1. Node receives capsule, extracts **code hash** from the capsule's header
2. If code is present in cache proceed with execution, otherwise continue with step 3
3. Node sends a **load request** to the sender of the capsule (previous node)
4. Sender node returns a **load response**
5. Node verifies that the received code matches hash
6. Code is stored in cache
7. Node resumes suspended capsule(s), and executes their code
8. If load request fails or times out, the capsule is discarded

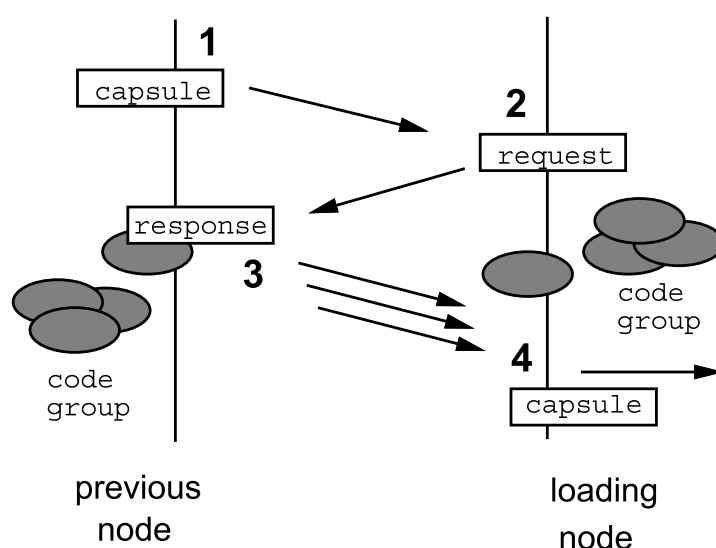


Figure 3.3: A typical pattern of code distribution in active networks, each node maintains a “code group” (cache), if the node doesn’t have the required code to process this capsule, it can request it from the previous node. Source: [36, p. 4]

As we can see, active network nodes are designed to request code from the **sender** of the capsule, as the sender **must already have** the required code in its cache, otherwise it wouldn’t have been able to process and send the capsule in the first place. This greatly increases the efficiency of the network, the workload of handling code load requests is spread among all nodes participating in transmission of the capsule.

3.2.4 Reliability

Active networks by default don’t ensure end-to-end reliability, i.e. if a capsule is lost, it’s up to the application to use its own re-transmission mechanism.

Only reliability features used in active networks are related to denial-of-service attacks, such as limiting the amount of CPU time a capsule’s program is allowed to take, or introducing a TTL field (see fig. 3.4) into the capsule’s header to prevent infinite send-receive loops.

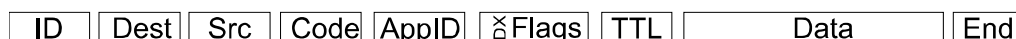


Figure 3.4: Example structure of an active networks capsule. Source: [34]

Solver as a Distributed Application

4

This chapter serves as a theoretical introduction to what underlying technologies and paradigms can be used to implement a distributed (meta)solver, for an exact description including the implementation-specific details of the finalized version of the software developed in this thesis, see chapters 7 and 9.

The following sections focus primarily on the networking and distributed computing aspects of the solver, utilizing knowledge and findings from chapter 3.

Chapters 5 and 6 can be seen as a continuation of this chapter, useful libraries which the solver could utilize are discussed there.

4.1 Architectural Overview

As was explained in section 2.6.1, from a high-level perspective, the model we want our solver to utilize is the “Island” or “Island-Archipelago” model.

Such island model allows the user to define a topology by which the individual islands are connected, it can be for example a mesh topology, or a ring topology where each node is connected only to its two neighbors. Naturally, this has to be implemented on the software level, as today’s IP networks are almost always a star topology, i.e. end-user devices connected to a switch.

In the terminology of Software-Defined Networking (SDN), we can think of the island model as an **overlay** network (see fig. 4.1). In this context, the **underlay** (physical) network consists of nodes (computers) which are able to communicate with each other and thus form our distributed cluster.

Based on these requirements, it appears as a good solution to adopt a “master-slave” architecture (as was explored in section 3.1), which inherently splits our system into two different types of nodes: the **controller node** (master) and the **worker node** (slave).

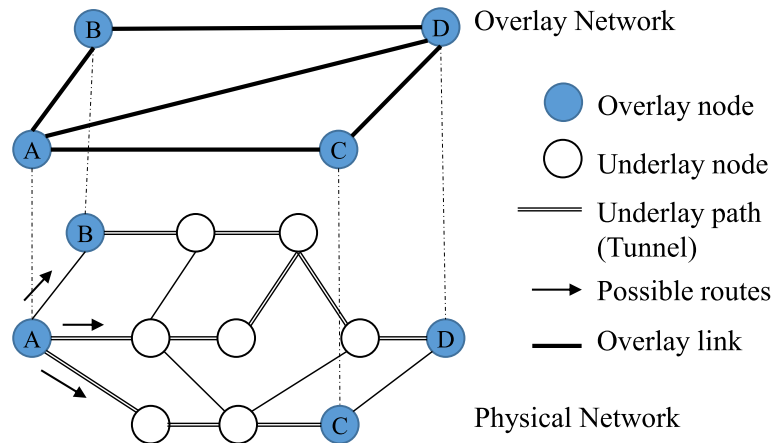


Figure 4.1: Overlay network example. Source: [38]

The controller would act as an “archipelago”, i.e. a centralized element mediating the communication and exchange of information between islands. Similar to the PVM system explained in the previous chapter, the controller would create **tasks** and then assign them to **workers**.

The task is a “work unit” of our cluster, it represents an optimization problem (together with its parameters, initial population, etc.) which gets solved by a worker node’s optimization algorithm. So the task must transfer both data and a problem (executable code / program), this is remarkably similar to the concept of capsules in active networks.

As for the internal structure of the controller and worker nodes, it’s clear that they’ll need to depend on at least two core libraries, that is: an optimization algorithm library, and a networking library. The optimization library will provide worker with algorithms to solve a problem, and controller with the necessary tools to manage an island-archipelago model (i.e. performing migrations and exchanging individuals between islands). A networking library would then provide both with language-level constructs for exchanging messages.

Please refer to figure 4.2 for a high-level visualization of this concept.

4.2 Communication

A custom protocol will need to be designed in order to facilitate the communication between controller and workers. The communication itself should be asynchronous, i.e. the controller should still be responsive when waiting for the workers to respond. The types of messages that this protocol should utilize can be described like so:

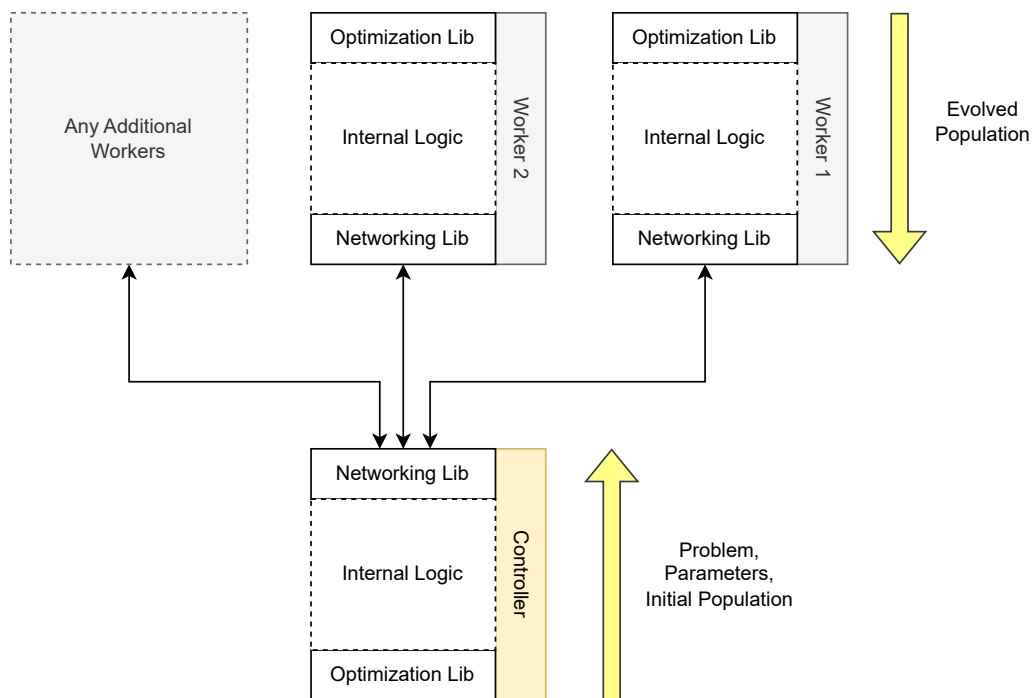


Figure 4.2: A computational cluster where each worker node uses a population-based optimization algorithm to evolve a population of candidate solutions to an optimization problem received from the controller, results sent back by workers accumulate at the controller. It can be seen that in this schema workers don't connect to each other, instead they only connect to the controller, this is intentional as it allows us to centralize all decision making logic (such as migration of individuals between islands) to the controller, thus this is where logic of the “overlay network” described above will be implemented.

- Work related messages
 - e.g.: `WORK_DATA`, `RESULTS_DATA`
 - Controller should send `WORK_DATA` containing the problem identifier, parameters, etc.
 - Worker returns the evolved population inside `RESULTS_DATA` back to controller
 - See section 4.3
- Management messages
 - e.g. `JOIN`, `LEAVE`
 - Workers use these messages to inform controller of their presence

- The join message could contain fields with additional information about the worker such as the worker’s OS, hardware specifications, etc.
- Code distribution messages
 - e.g. `PROBLEM_REQUEST`, `PROBLEM_RESPONSE`
 - Messages for sending and receiving executable code (problem)
 - See section 4.3
- Diagnostic messages
 - e.g. `PING`
 - Messages for ensuring reliability of the cluster
 - See section 4.4

An appropriate networking library and serialization method should be chosen to implement this communication pattern.

4.3 Problem Discovery and Work Allocation

As we have already established, our “task” which gets allocated to a worker consists of two parts: **data** and **executable code** (optimization problem).

It was shown by active networks (see sec. 3.2.3), that it doesn’t make sense to re-send this executable code each time if it doesn’t change. It’d thus be best for our worker nodes to use a similar mechanism.

We can therefore define the structure of `WORK_DATA` as the following:

1. Optimization problem identifier (hash)
2. Initial population
3. Additional parameters (optional)

Using a hash as the identifier of an optimization problem brings another benefit, this system by design ensures that there are **no versioning conflicts** as different versions of the same problem will automatically be stored under a different hash. This wouldn’t be the case if we for example used hard-coded string names to identify a problem.

The worker will have a local cache indexed by the problem’s hash. If the worker receives a `WORK_DATA` message and finds out that it doesn’t have this problem cached, it’ll send a `PROBLEM_REQUEST` message to the controller, which will then respond with `PROBLEM_RESPONSE` containing the executable code.

Lastly, it'd also be appropriate to mention that the controller should implement some form of load balancing when it comes to allocation of work to workers, as some workers could have weaker hardware than other ones. This can be achieved by measuring performance metrics and tweaking the amount of work allocated to each node (by e.g. reducing the population size).

4.4 Reliability of the Cluster

The worker should send a PING or KEEPALIVE message to the controller at some fixed time interval, that way the controller can detect dead workers and remove them from the work allocation process.

The fact that a worker node can unexpectedly fail introduces another complication, we need to be able to re-allocate work to another node if that happens. This could be achieved by storing the `WORK_DATA` in a buffer on the controller node and removing it only after we've received the corresponding `RESULTS_DATA`.

Networking Libraries

5

This chapter explores various networking libraries that can potentially be used for the communication of nodes in a distributed system. Our focus will be on highlighting the pros and cons of each library, example code will be provided when adequate.

5.1 BSD Sockets

BSD (Berkeley Software Distribution) Sockets are perhaps the most well known software library in the field of networking. First released in 1983 as part of 4.2BSD Unix, today the BSD Sockets API is standardized as part of POSIX (IEEE Std 1003.1), this includes all the header files and required system calls [39, p. 549].

As mentioned above, the API consists of several header files which can be directly used in a C or C++ program, other languages usually provide their own wrappers of this API.

Programs using BSD sockets are safely portable across all modern Unix-based (or Unix-like) operating systems such as macOS, FreeBSD or GNU Linux, compatibility with MS Windows is however not guaranteed, as the Windows Sockets API in a significant way deviates from the standard.

Usage

After including `<sys/socket.h>`, it becomes possible to create a socket by calling `int socket(int domain, int type, int protocol);` the return value of this call is a **file descriptor** identifying the socket, this is the typical “everything is a file” model used by Unix-like operating systems.

Once a socket is created, we can use it either as a **client** or a **server**. In the case of a client socket, calling `int connect(int s, const struct sockaddr *name, socklen_t namelen);` will connect it to a server.

We can then use standard file manipulation calls like `write()`, `read()` or `close()`, the socket behaves just like a normal file would.

Source code 5.1: BSD Sockets client in C++, see listing 5.2 for the corresponding server code.

```
1 #include <iostream>
2 #include <vector>
3 #include <stdexcept>
4 #include <sys/socket.h>
5 #include <netinet/in.h>
6 #include <arpa/inet.h>
7 #include <unistd.h>
8
9 int main()
10 {
11     constexpr auto ADDRESS = "127.0.0.1";
12     constexpr uint16_t PORT = 3000;
13
14     // Set up socket (AF_INET = IPv4)
15     const int fd = socket(AF_INET, SOCK_STREAM, 0);
16     if (fd <= 0)
17     {
18         throw std::runtime_error("Failed_to_create_socket");
19     }
20
21     // Try to connect to the server
22     sockaddr_in addr{.sin_family = AF_INET, .sin_port = htons(PORT), .
23     sin_addr = {inet_addr(ADDRESS)}};
24     if (connect(fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) !=
25     0)
26     {
27         throw std::runtime_error("Cannot_connect_to_server");
28     }
29     std::cout << "Successfully_connected_to_server" << std::endl;
30
31     // Try to receive data from the server
32     std::vector<char> buffer(256);
33     int n = recv(fd, buffer.data(), buffer.size(), 0);
34     if (n < 0)
35     {
36         throw std::runtime_error("Failed_to_receive_data_from_server");
37     }
38     if (n == 0)
39     {
40         throw std::runtime_error("Server_closed_connection_unexpectedly");
41     }
42     std::cout << "Received:_" << std::string(buffer.data(), n) << "\n";
43     close(fd);
44 }
```

To create a server socket we must first **bind** the socket to an IP and a port, then start **listening**, and only after that is the socket able to **accept** incoming connections from clients (see listing 5.2).

Keep in mind that a real server would have the accept call inside a loop to make connections from multiple clients possible. The easiest way to then simultaneously read and write to all client sockets is to spawn a new thread for each. However, this approach is rather inefficient due to the overhead of running a large amount of threads, real-world solutions typically use high-performance I/O multiplexing such as the Linux **epoll** API. [40]

Source code 5.2: BSD Sockets server in C++

```
1 #include <iostream>
2 #include <stdexcept>
3 #include <sys/socket.h>
4 #include <netinet/in.h>
5 #include <unistd.h>
6
7 int main()
8 {
9     constexpr uint16_t PORT = 3000;
10    // 1) Create a socket
11    const int serverFd = socket(AF_INET, SOCK_STREAM, 0);
12    if (serverFd <= 0)
13    {
14        throw std::runtime_error("Failed_to_create_server_socket.");
15    }
16
17    // 2) Bind socket to address (INADDR_ANY = 0.0.0.0)
18    sockaddr_in addr{.sin_family = AF_INET, .sin_port = htons(PORT), .
19    sin_addr = {INADDR_ANY}};
20    if (bind(serverFd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr))
21    != 0)
22    {
23        throw std::runtime_error("Failed_to_bind_on_this_port");
24    }
25
26    // 3) Start listening on bound address
27    if (listen(serverFd, 1) != 0)
28    {
29        throw std::runtime_error("Failed_to_start_listening");
30    }
31    std::cout << "Server_listening_on_port_" << PORT << "..." << std::
32    endl;
33
34    // 4) Start accepting client connections
35    int clientFd = accept(serverFd, nullptr, nullptr);
```



```
33     if (clientFd <= 0)
34     {
35         throw std::runtime_error("Failed_to_accept_incoming_connection");
36     }
37     std::cout << "Client_connected" << std::endl;
38
39     // 5) Client connected, send a hello message
40     const std::string msg = "server_hello";
41     if (send(clientFd, msg.c_str(), msg.length(), 0) < 0)
42     {
43         throw std::runtime_error("Failed_to_send_data_to_client");
44     }
45
46     close(clientFd);
47     close(serverFd);
48 }
```

Key Findings

BSD Sockets are a reliable and future-proof way of implementing network servers and clients on Unix-like platforms. The main benefit is that we don't need any external dependencies, the standardized API is provided to us directly by the operating system. This is why software such as HTTP servers often utilize BSD Sockets directly.

On the other hand, a significant problem of BSD Sockets is the limited compatibility with non-Unix operating systems such as MS Windows, this forces us to write OS specific code by e.g. adding preprocessor directives such as `#ifdef _WIN32`, or to instead use a high-level library which handles OS specifics for us.

Furthermore, BSD Sockets is purely a sockets library, therefore it doesn't provide us with any I/O multiplexing, which means we're forced to use OS specific APIs such as for example the Linux `epoll` API mentioned above.

5.2 Boost.Asio

Boost.Asio is a cross-platform library targeting low-level network and I/O programming. It was first released as "Asio" in 2003 by Christopher M. Kohlhoff, later integrated into the Boost library collection. [41]

According to official documentation, Boost.Asio is portable across all commonly used operating systems, scalable to support thousands of concurrent connections, efficient in terms of memory copy operations, and its interface is based on already established APIs such as BSD sockets. [42]

Apart from providing TCP or UDP sockets, the library also includes other related tools such as “execution management” via the `io_context` object, timers which can schedule functions to execute at specific time intervals, access to I/O objects such as serial ports or files, and it also provides buffers such as `boost::asio::streambuf`.

The library in particular focuses on asynchronous **callback style** programming, allowing us to for example register callbacks on socket receive events, etc. Asio internally uses OS-specific I/O multiplexing APIs to achieve this, namely: Windows IOCP, macOS kqueue and Linux epoll or `io_uring`. See fig. 5.1 for an example of this approach.

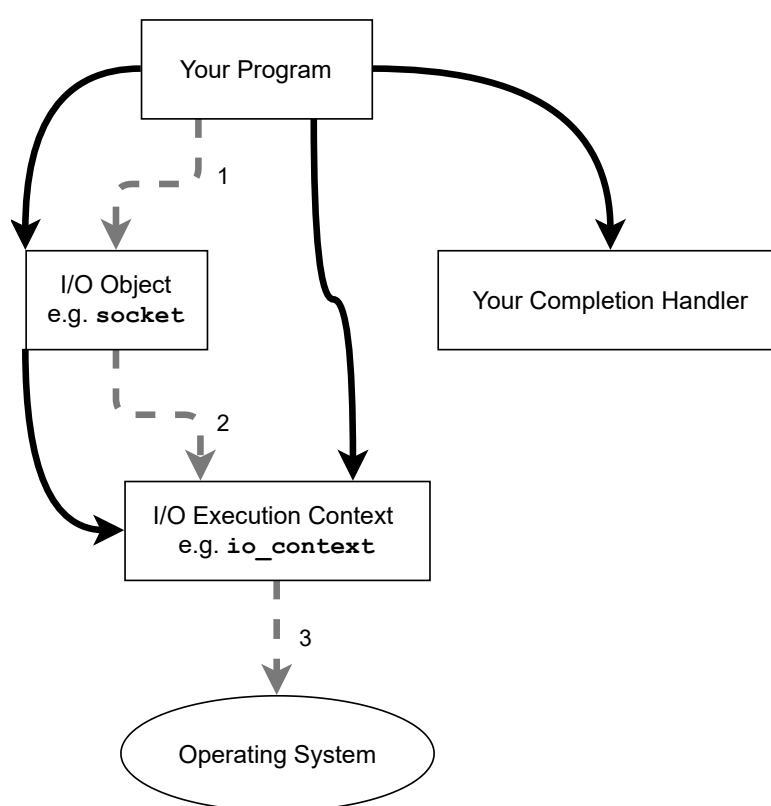


Figure 5.1: Boost.Asio callback style asynchronous model, the numbers signify: 1) User program initiates a connect operation by calling the I/O object (socket), 2) The I/O object forwards this request to the execution context (io_context), 3) Execution context signals the underlying OS API to start an asynchronous connect operation. The function “Your Completion Handler” is then notified once an event occurs on the socket. (Inspired by: [43])

Usage

A TCP client using Boost.Asio can be created by making an instance of `tcp::socket`, passing a `tcp::endpoint` object to its `connect` member function, and then reading a message from the server using the `read_some` member function.

Listing 5.3 shows us the complete code, notice the use of objects and easier error handling than in the BSD Sockets example.

Source code 5.3: Boost.Asio TCP client

```
1 #include <asio.hpp>
2 #include <iostream>
3
4 using asio::ip::tcp;
5
6 int main()
7 {
8     asio::io_context io;
9     tcp::socket socket(io);
10    socket.connect(tcp::endpoint(asio::ip::make_address("127.0.0.1"),
11                                3000));
12
13    std::string buffer(256, '\0');
14    asio::error_code ec;
15    size_t len = socket.read_some(asio::buffer(buffer), ec);
16
17    if (!ec)
18    {
19        std::cout.write(buffer.data(), len);
20        std::cout << std::endl;
21    }
22 }
```

To make a TCP server, we need a `tcp::acceptor` object which accepts incoming client connections. The complete code can be seen in listing 5.4, notable in this example are the member functions `on_accept` and `on_write` which act as callback functions for Asio objects.

Source code 5.4: Boost.Asio “hello world” server

```
1 #include <asio.hpp>
2 #include <ctime>
3
4 using asio::ip::tcp;
5
6 class server
7 {
8 public:
```

```
9     server(asio::io_context& io) : _acceptor(io, tcp::endpoint(tcp::v4(),
10         3000)), _socket(io)
11     {
12         start_accept();
13     }
14 private:
15     tcp::acceptor _acceptor;
16     tcp::socket _socket;
17     std::string _msg;
18
19     void start_accept()
20     {
21         _acceptor.async_accept(_socket, [this](std::error_code ec)
22             {
23                 on_accept(ec);
24             });
25     }
26
27     void on_accept(const std::error_code& ec)
28     {
29         // Runs when acceptor accepts client
30         if (!ec)
31         {
32             _msg = "hello_from_server";
33
34             asio::async_write(_socket, asio::buffer(_msg),
35                 [this](std::error_code, std::size_t)
36                 {
37                     on_write();
38                 });
39         }
40         _socket = tcp::socket(_acceptor.get_executor());
41         start_accept();
42     }
43
44     void on_write()
45     {
46         // runs when async_write finishes
47     }
48 };
49
50 int main()
51 {
52     asio::io_context io;
53     server s(io);
54     io.run();
55 }
```

Key Findings

Boost.Asio is a highly performant C++ networking library, it achieves portability by providing a unified interface utilizing APIs of all common operating systems.

The primary benefit of Boost.Asio is its usage of the asynchronous callback pattern, greatly improving performance and simplifying usage.

The age of this library can however be considered a disadvantage, as of the time of writing, Boost.Asio is more than 20 years old. Another disadvantage can possibly be the fairly “low-level” approach to networking, the library provides only “raw” TCP or UDP sockets without providing serialization or any of such supporting features.

5.3 ZeroMQ

ZeroMQ started development in 2007 as a reaction to the flaws of broker-based messaging protocols such as AMQP. It consists of both a **networking library** (`libzmq`) and a **transport protocol** called ZMTP (*ZeroMQ Message Transport Protocol*), it can therefore be considered a fairly high-level library as opposed to for example Boost.Asio. [44]

The creators of ZeroMQ aimed to retain the features of traditional broker-based architectures, such as for example the publisher-subscriber communication pattern, while at the same time removing the single point of failure and increased latency which broker-based architectures suffer from. Hence, the “Zero” in “ZeroMQ” stands for “**Zero broker**”.

The fact that ZeroMQ doesn’t use a centralized broker means that there are peer-to-peer connections between nodes, each node then has its own message queue (**MQ**) and internally handles the delivery of messages to its peers by utilizing the `libzmq` library.

Internally `libzmq` maintains a receive (Rx) queue and a transmit (Tx) queue for each connected peer. We initialize the library by creating a `zmq::context_t` object, which spawns an I/O thread that’s responsible for writing and reading from the Tx and Rx queues. All instances of `zmq::socket_t` accept `zmq::context_t` in their constructor, so the sockets themselves only interact with the Tx and Rx queues without handling any of the actual send or receive network operations. This separation is important, because it allows the implementation of the transport protocol.

The *ZeroMQ Message Transport Protocol* can work over many different underlying protocols such as TCP, PGM or EPGM, even IPC or *In-process* communication can be used. That is to say, we can easily switch from different underlying protocols (e.g. from TCP to IPC) without having to rewrite any existing code.

ZMQ is not a “stream of bytes” like TCP, instead it’s a **message-oriented protocol** where each message can contain multiple **frames**. Another feature is the added reliability, messages are kept in the Tx queue and automatically re-sent at pre-determined intervals if their delivery fails. This also means that our peer doesn’t have to be online at the time we’re sending a message, the message will simply wait in the queue until the peer comes online. [45]

ZeroMQ uses different socket types depending on the communication pattern we want to use, the socket type enforces a specific communication pattern, so both peers have to use a compatible socket, e.g. *subscriber* peer can connect to a *publisher* peer, however it doesn’t make any sense to for example connect a *request* peer to a *subscriber* peer. The key ZeroMQ socket types are summarized in table 5.1.

Table 5.1: Overview of ZeroMQ socket patterns

Pattern	Sockets	Description
Request-Reply	REQ, REP	Synchronous blocking “HTTP-like” communication.
Publish-Subscribe	PUB, SUB	Asynchronous one-to-many broadcast, subscriber filters by message prefix.
Pipeline	PUSH, PULL	Used for distribution of tasks, PUSH load-balances tasks to connected workers, PULL queues tasks from multiple sources.
Asynchronous client-server	DEALER, ROUTER	Fully asynchronous non-blocking client-server pattern, DEALER acts as a client, ROUTER as a server, dealers are identified by a <i>routing ID</i> .
Exclusive Pair	PAIR	One-to-one bidirectional pipe, can be used for in-process communication.

Usage

The examples in this section show a router-dealer pattern, allowing asynchronous client-server communication. Listing 5.5 shows us how ZeroMQ differs from traditional libraries such as BSD Sockets, the ROUTER socket can accept connections from multiple dealers at once, it’s a one-to-many pattern, the individual DEALER clients are identified by their routing ID.

Both examples 5.5 and 5.6 use the C++ header-only library `cppzmq`, this is an RAII wrapper around the C-style interface of `libzmq`.

Source code 5.5: ZeroMQ ROUTER server example

```
1 zmq::context_t context;
2
3 zmq::socket_t router(context, zmq::socket_type::router);
4 router.bind("tcp://*:5555");
5
6 // Receive client routingId + type + payload (Three different message frames)
7 zmq::message_t routingId, type, payload;
8 router.recv(routingId);
9 router.recv(type);
10 router.recv(payload);
11
12 // Send reply: type + payload
13 router.send(routingId, zmq::send_flags::sndmore);
14 router.send(zmq::buffer("RESPONSE"), zmq::send_flags::sndmore);
15 router.send(zmq::buffer("Hello_from_server"), zmq::send_flags::none);
```

Source code 5.6: ZeroMQ DEALER client example

```
1 zmq::context_t context;
2
3 zmq::socket_t dealer(context, zmq::socket_type::dealer);
4 dealer.connect("tcp://localhost:5555");
5
6 // Send request: type + payload
7 dealer.send(zmq::buffer("REQUEST"), zmq::send_flags::sndmore);
8 dealer.send(zmq::buffer("Hi_server"), zmq::send_flags::none);
9
10 // Receive reply: type + payload
11 zmq::message_t type, payload;
12 dealer.recv(type);
13 dealer.recv(payload);
```

Key Findings

ZeroMQ is a high-performance library capable of being used in large distributed systems.

The key benefit of ZeroMQ is that it not only provides us with sockets, but also with a transport protocol which automatically handles features like reliable message delivery or being able to internally divide a message into multiple frames.

However, the level of abstraction and large amount of different features also makes the library harder to use and understand.

Optimization Libraries

6

6.1 Pagmo

Pagmo stands for “Parallel global multiobjective framework for optimization”, it is a C++ software library providing us with many different **population-based algorithms** (see sec. 2.4) and several optimization problem test suites. Pagmo also includes a local (non-distributed) implementation of the Island model, as will be discussed later in this section. [46]

Pagmo can be used to solve both Single-Objective (SO) and Multi-Objective (MO) problems, the differences of which have been explained on a theoretical level in section 2.5.

A benefit of Pagmo is that it can be easily extended by the user, the library exposes an interface for each of its primary object types, this permits us to create *User Defined Algorithms (UDA)*, *User Defined Problems (UDP)* or *User Defined Islands (UDI)*.

Another benefit is the so called “Batch Fitness Evaluator”, which allows us to define a member function called `batch_fitness` in our UDP, this function takes a vector of individuals as an input and evaluates them in a “batch”, returning a vector of fitness values. The body of `batch_fitness` is intended to utilize GPGPU, CPU vector instructions or an FPGA to compute a large amount of fitness values at once, significantly speeding up the execution time as opposed to a traditional sequential computation of fitness values on the CPU. Pagmo algorithms will automatically use the `batch_fitness` function if defined.

The algorithm can be used to evolve a population either by directly calling the `evolve` function (as shown in listing 6.1), or by being embedded inside of an island.

Source code 6.1: Running an optimization algorithm in Pagmo (no islands) [47]

```
1 #include <iostream>
2 #include <pagmo/algorithm.hpp>
3 #include <pagmo/algorithms/nsga2.hpp>
4 #include <pagmo/population.hpp>
```



```
5 #include <pagmo/problem.hpp>
6 #include <pagmo/problems/dtlz.hpp>
7
8 using namespace pagmo;
9
10 int main()
11 {
12     // 1 – Instantiate a pagmo problem constructing it from a UDP
13     // (user defined problem).
14     problem prob{dtlz(1, 10, 2)};
15
16     // 2 – Instantiate a pagmo algorithm
17     algorithm algo{nsga2(100)};
18
19     // 3 – Instantiate a population
20     population pop{prob, 24};
21
22     // 4 – Evolve the population
23     pop = algo.evolve(pop);
24
25     // 5 – Output the population
26     std::cout << "The population:\n" << pop;
27 }
```

Island Model Implementation

Pagmo implements an “Archipelago-Island-UDI” model, it consists of three classes, where the first two: `pagmo::archipelago` and `pagmo::island` are “wrappers” and the third class (UDI) is one of the User-Defined-Islands. Pagmo by default provides us with two UDIs¹: `pagmo::thread_island` and `pagmo::fork_island`.

We’ll begin our explanation with `pagmo::archipelago`, it acts as a container for `pagmo::island` objects and facilitates the migration of individuals between them. It also contains an instance of `pagmo::topology`, which specifies the “connections” between islands, among which individuals are exchanged.

The archipelago has multiple different constructors, the first constructor takes: {island count, algorithm, problem, population size}, in this constructor `island count` instances of `pagmo::island` are created, these islands get the algorithm, problem and population size passed to them.

The second most commonly used constructor of `pagmo::archipelago` is the default constructor with no parameters, it will create an empty archipelago,

¹Yes, even these two classes provided by the library itself are in Pagmo’s terminology called “User-Defined-Island”, because they’re an implementation of the island interface, not something that’d be a core part of the library.

we can then add instances of `pagmo::island` to it ourselves by calling `archipelago.push_back(island);`

The second class in this hierarchy is the `pagmo::island`, it purely acts as a wrapper around the UDI. `pagmo::island` stores all of the data such as the population and the problem, whereas the UDI is stateless.

UDI is the only object here which runs the algorithm and performs evolution, it is therefore the “**computational unit**” of this model. As was mentioned above, by default it can either perform the computation on a new thread (`pagmo::thread_island`) or perform it in a new process (`pagmo::fork_island`). This is what allows the island model to utilize multiple CPU cores, we can for example create 16 islands and each of them would contain an UDI running on a new thread.

As you might’ve already noticed, UDI is the only part of this model which needs to be replaced with a “Distributed UDI”, in order to make the system distributed across multiple nodes. More on that in later chapters.

See listing 6.2 for an example of how the archipelago operates from a user’s perspective, figure 6.1 then shows the implementation details we’ve talked about in the paragraphs above.

Source code 6.2: Basic example of archipelago in Pagmo [48]

```
1 #include <iostream>
2 #include <pagmo/algorithm.hpp>
3 #include <pagmo/algorithms/sade.hpp>
4 #include <pagmo/archipelago.hpp>
5 #include <pagmo/problem.hpp>
6 #include <pagmo/problems/schwefel.hpp>
7
8 using namespace pagmo;
9
10 int main()
11 {
12     // 1 – Instantiate a pagmo problem constructing it from a UDP
13     // (i.e., a user-defined problem, in this case the 30-dimensional
14     // generalised Schwefel test function).
15     problem prob{schwefel(30)};
16
17     // 2 – Instantiate a pagmo algorithm (self-adaptive differential
18     // evolution, 100 generations).
19     algorithm algo{sade(100)};
20
21     // 3 – Instantiate an archipelago with 16 islands having each 20 individuals.
22     archipelago archi{16u, algo, prob, 20u};
23
24     // 4 – Run the evolution in parallel on the 16 separate islands 10 times.
```

```

25     archi.evolve(10);
26
27     // 5 – Wait for the evolutions to finish.
28     archi.wait_check();
29
30     // 6 – Print the fitness of the best solution in each island.
31     for (const auto &isl : archi) {
32         std::cout << isl.get_population().champion_f()[0] << '\n';
33     }
34 }

```

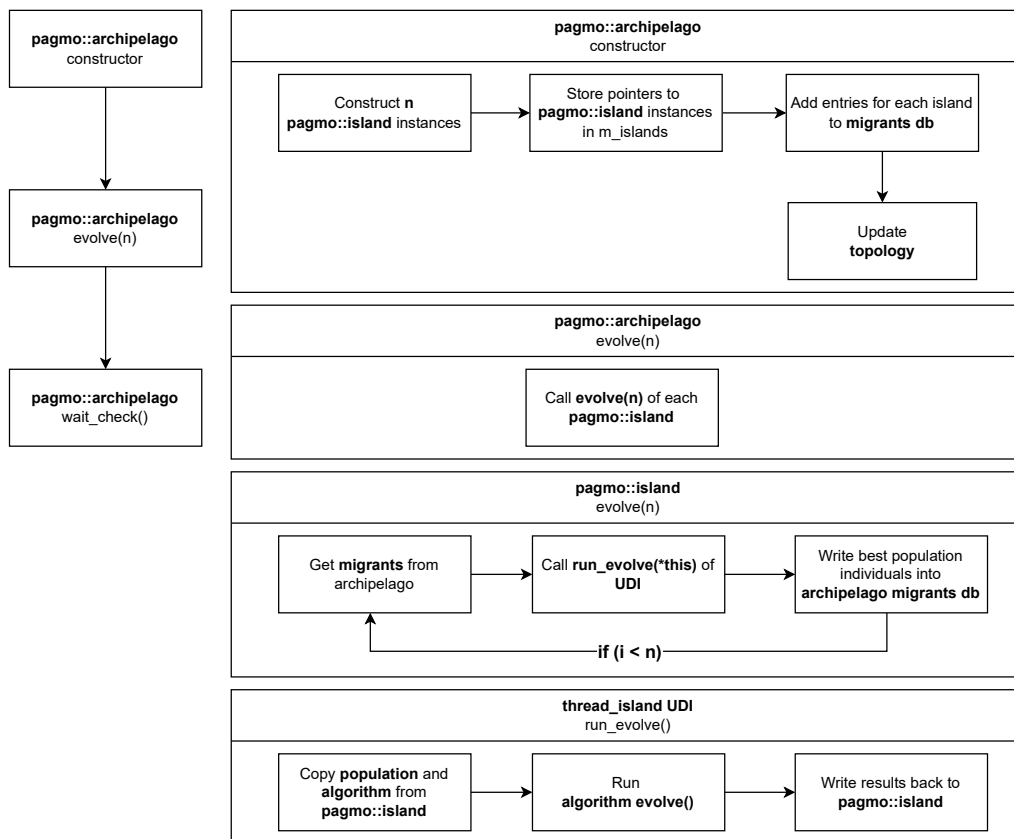


Figure 6.1: Internal details of how the “Archipelago-Island-UDI” model works. Left portion of the figure shows the program flow (see listing 6.2), right portion of the figure shows the internals of each function.

NSGA II Performance Analysis

It was noticed during the examination of this library that certain algorithms such as NSGA II show abnormally poor performance, especially when using the MSVC

Fixing greater_than_f

Listing 6.3 shows us the source code of this function, `std::isnan` is what's internally calling `_dclass`. Therefore, we need to reduce the amount of calls to `std::isnan` as much as possible.

Source code 6.3: Function `greater_than_f` inside of `pagmo/detail/custom_comparisons.hpp`

```

1 // Greater than compares floating point types placing nans after inf or before -inf
2 // It is a useful function when calling e.g. std::sort to guarantee a weak strict ordering
3 // and avoid an undefined behaviour
4 template <typename T, bool After = true>
5 inline bool greater_than_f(T a, T b)
6 {
7     static_assert(std::is_floating_point<T>::value, "greater_than_f can_
      be_used_only_with_floating_point_types.");
8     if (!std::isnan(a)) {
9         if (!std::isnan(b))
10             return a > b; // a > b
11         else
12             return !After; // a > nan
13     } else {
14         if (!std::isnan(b))
15             return After; // nan > b
16         else
17             return false; // nan > nan
18     }
19 }
```

Listing 6.4 shows a new version of this function that we have rewritten using the C++20 spaceship operator. This produces the exact same output as the original code, because the operator returns an instance of `std::partial_ordering` if any of its operands is NaN. It's very rare that NaN gets passed to this function, so in our updated code the function almost always exits by hitting the return statement in either the first, or the second if block. By doing this, we've achieved that `std::isnan` gets executed only in rare edge cases instead of being called every time the function's body executes.

This change alone has resulted in a **30% performance increase**, reducing the execution time of our test program from **33.9 second** to **23.61 seconds**. For illustration see figure 6.4, notice that the `_dclass` function visible in figure 6.2 has almost completely disappeared.

Source code 6.4: Optimized body of `greater_than_f`

```

1 auto cmp = a <=> b;
2 if (cmp == std::partial_ordering::greater) {
```

```

3     return true;
4 }
5 if (cmp == std::partial_ordering::less || cmp == std::
partial_ordering::equivalent) {
6     return false;
7 }
8
9 const bool a_nan = std::isnan(a);
10 const bool b_nan = std::isnan(b);
11
12 if (!a_nan && b_nan) {
13     return !After;
14 }
15 if (a_nan && !b_nan) {
16     return After;
17 }
18
19 return false;

```

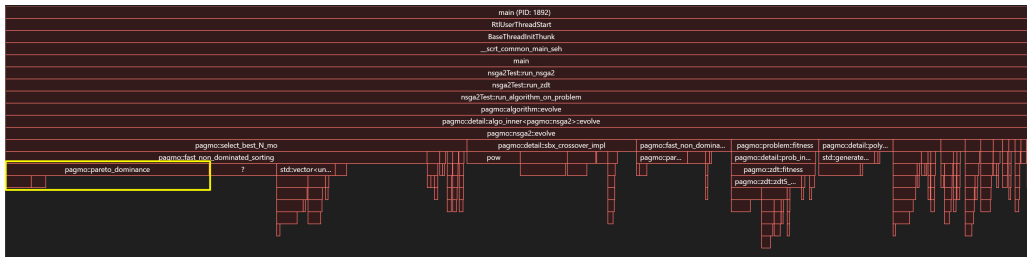


Figure 6.4: Flame graph after optimizing the `greater_than_f` function

Additional Fixes

The calls to function `pareto_dominance` were causing significant call overhead (stack operations, register management, parameter passing, possible branch prediction misses, etc.) in the for loop of a hot path function `fast_non_dominated_sorting()`.

The solution was to inline the `pareto_dominance` dominance function, which together with the previous fix improved execution time to **18.282 seconds** (46% improvement over baseline of **33.9 seconds**). See fig. 6.5 for details.

Another source of a significant slowdown were vectors getting allocated as on-stack variables by several of the hot path function. This caused the vectors to be repeatedly re-allocated, these allocations were taking up 7% of the total CPU time.

The simplest way to fix this was to pass the vector as a function parameter and re-use the already allocated memory each time the function is called. By doing this, we further improved the performance of the hot path function `fast_non_dominated_sorting`.

Results of this fix are visible in figure 6.6. The final execution time after applying all fixes was **16.783 seconds**, which is about **50% improvement** over the baseline of **33.9 seconds**.

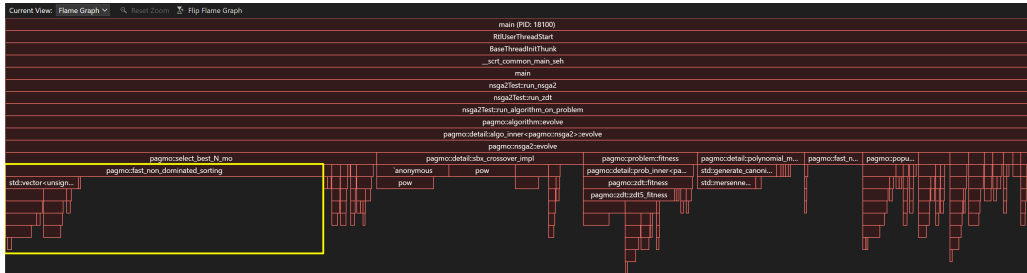


Figure 6.5: Flame graph after fixing `greater_than_f` and inlining `pareto_dominance`

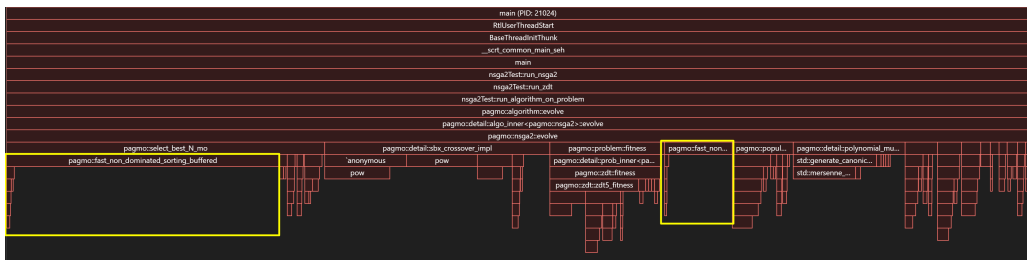


Figure 6.6: Final flame graph after applying all fixes

Effects on Other Compilers

After testing the exact same program compiled with Clang, the execution time before applying any of the fixes was **21.622 seconds**, this was reduced to **18.669 seconds** after applying all fixes, which is an improvement of **13.7%**.

Figure 6.7 shows us, that the biggest improvement was the reduction of useless `std::vector` re-allocations in hot path functions. This fix alone caused most of the performance gains.

Compiling with GCC has shown similar results, the execution time before applying any fixes was **12.416 seconds**, this was reduced to **10.496 seconds** after all fixes have been applied, which is an improvement of **15.5%**.

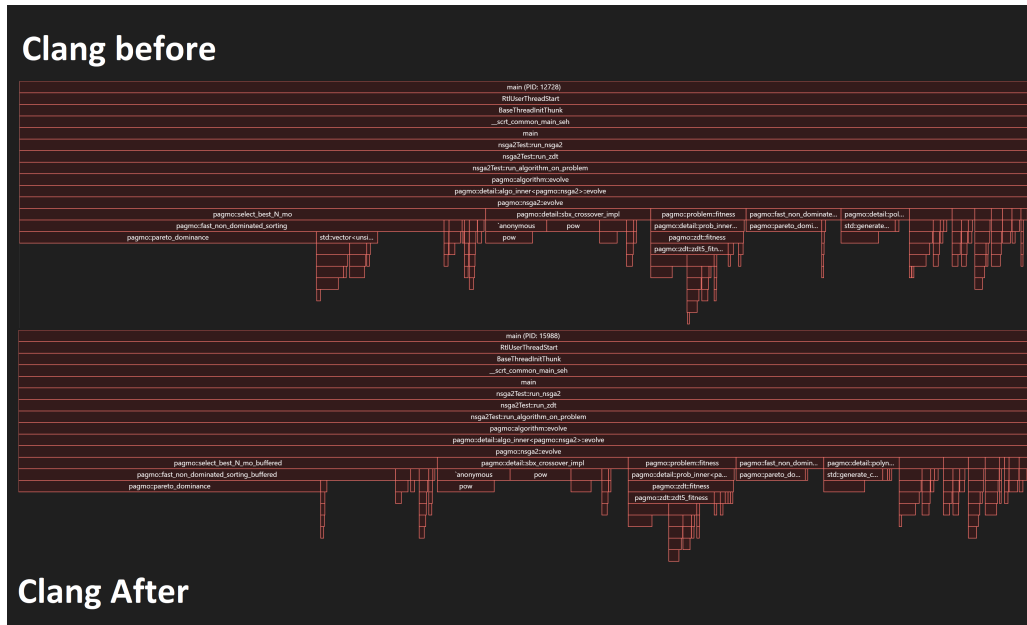


Figure 6.7: Before and after flame graph of the same application compiled with Clang instead of MSVC

The reason why other compilers have not shown as significant of an improvement as MSVC is that their implementation of `std::isnan` is superior to that of the MSVC Runtime Library, hence the spaceship operator fix had minimal to no effect.

6.2 SmartCGMS

SmartCGMS (Smart Continuous Glucose Monitoring System) is a software developed by doc. Tomáš Koutný at the Faculty of Applied Sciences, UWB. The purpose of SmartCGMS is to serve as a testbed for the development of blood-glucose level prediction models and algorithms. [3]

A diabetic patient's insulin pump needs to be regulated depending on the patient's blood glucose levels, a prediction model is used to predict future glucose levels to appropriately operate the insulin pump. The full “real world” workflow is shown in fig. 6.8.

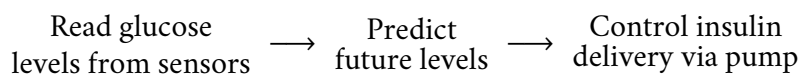


Figure 6.8: Closed-loop glucose control workflow

This is a closed-loop system, meaning that the insulin delivered by the pump causes a biological change in the patient influencing the glucose levels, this then feeds back into the sensors as updated glucose data.

The goal of SmartCGMS is to **replace** this biological feedback system with an in-silico model of the patient. [52]

See fig. 6.9 for a high-level overview of this workflow.

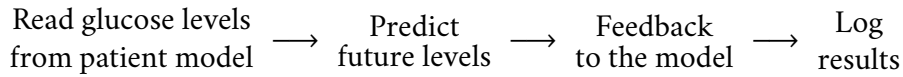


Figure 6.9: SmartCGMS workflow

Software Architecture

The “Predict future levels” step mentioned in fig. 6.9 internally utilizes a glucose prediction model, this model needs its parameters to be **optimized** in order to provide adequate results. The optimization step is what is handled by a **solver**, this can for example be the metasolver discussed in this thesis.

SCGMS provides by default several algorithms that can be used, like for example PSO or DE. However, additional solvers can be implemented by utilizing the solver interface (see next section).

As for the architecture itself, SCGMS utilizes the Component Object Model (COM), each object using the COM model implements a specific interface (“API”) and is identified by a GUID. Any object which adheres to this interface is compatible, i.e. it for example allows us to implement different parts of the software in different languages or to allow **runtime loading** of additional objects contained in dynamic libraries.

The main SCGMS workflow as shown in fig. 6.9 is internally implemented using the **uni-directional filter model**, this model can be described as a “pipeline” or a “chain” of filter objects connected together, each filter is a COM object which performs some processing of the data and passes its result to the next filter in the chain, until eventually the last filter is reached.

It thus allows us to integrate our metasolver into this system by creating a dynamic library implementing the **solver interface**, SCGMS will then be able to load this library at runtime.

Solver Interface

As was stated in chapter 1, one of the goals of this thesis is to demonstrate the metasolver as an application solver for SmartCGMS. To achieve this, a **wrapper library** around the metasolver will have to be created. The SCGMS interface that a solver library must implement consists of these two functions:

```
HRESULT IfaceCalling do_solve_generic(  
    const GUID*          solver_id,  
    solver::TSolver_Setup* setup,  
    solver::TSolver_Progress* progress  
);
```

Where `setup` is an object containing initial parameters of the solver such as objective function, parameter bounds, initial population, number of generations, etc. and `progress` is an optional parameter tracking the current progress of the computation.

```
HRESULT IfaceCalling do_get_solver_descriptors(  
    scgms::TSolver_Descriptor** begin,  
    scgms::TSolver_Descriptor** end  
);
```

Where pointers `begin` and `end` denote a list of `TSolver_Descriptor` objects, containing the name, id and description of each solver in the library.

The specific implementation of this wrapper library will be discussed in later chapters of this thesis.

Architecture and Design of the Distributed Metasolver

7

The sections in this chapter explain the architecture and design choices of the final implemented version of our distributed metasolver. It is expected that you understand the theoretical concepts from chapters 2 and 4, refer to those chapters if the theory of what is discussed in this chapter is unclear.

7.1 Technological Stack

The distributed metasolver is implemented as a C++ library, it utilizes the CMake build system and uses git submodules to manage its dependencies.

First dependency is the **Pagmo** library [46]. There were several reasons to choose Pagmo, firstly, it provides a satisfactory amount of both multi-objective and single-objective algorithms, which we can use in our metasolver. The second reason was that the Archipelago-Island-UDI model provided by this library can relatively easily be modified to support the allocation of work to worker nodes in our distributed cluster.

The second important dependency is the **ZeroMQ** library [53], along with its C++ wrapper **cppzmq** [54]. It was clear that choosing ZeroMQ would be a much better approach than using Boost.Asio or “raw” BSD Sockets, in particular because ZeroMQ is a high-level library providing us with a ready to use transport protocol and various other useful features such as the `zmq::active_poller_t` I/O multiplexing object. Thanks to ZeroMQ it was possible to implement our library’s network operations using a modern asynchronous event oriented pattern, without needing to use any OS-specific APIs.

The remaining dependencies are the **Catch2** unit testing framework [55] and dependencies of the Pagmo library: **Eigen3** [56], **Boost** [57] and **Intel TBB** [58].

All of the dependencies mentioned above, except for Boost and Intel TBB, are automatically downloaded as a git submodule and included in our CMake project using the `add_subdirectory()` command. Boost and Intel TBB must be installed manually by the user, including them as a git submodule wouldn't be practical due to the large size of these libraries and possible transitive dependencies.

7.2 The Metasolver Library

The most important portion of our library is the `./src/distributed/logic` subdirectory, inside of which the classes `distributed_solver`, `distributed_controller`, `distributed_worker` and `distributed_island` are located. The rest of this library consists of supporting features such as sockets, UDP (User Defined Problem) management, dynamic library loading, etc. more on that in chapter 9.

To utilize the metasolver in any existing application, we can simply link against this library and create the `distributed_solver` object. This object acts as a wrapper around `distributed_controller` and `pagmo::archipelago`, its only mandatory constructor parameter is an address on which the controller begins listening for worker connections.

The `distributed_controller` acts as a **server** to which worker nodes connect using the `distributed_worker` **client**. The worker nodes thus only need to know the server's address in order to connect and become ready to accept work.

After the required number of workers has connected, we can call the `evolve(problem, algorithm, populationSize)` member function of `distributed_solver`, this will begin the distributed evolution process.

The `distributed_solver` internally maintains an archipelago containing instances of `pagmo::island`, inside of which our UDI (User-Defined-Island) is contained (see chapter 6 for details on Pagmo's Island model).

The UDI in this case is the `distributed_island` class, each `distributed_island` represents a "work task", which needs to be allocated to a worker. The `distributed_island` internally communicates with the controller server using ZeroMQ IPC sockets, work tasks are sent to the controller over this socket, which then allocates these tasks to free workers. The island's thread is then blocked until the evolved population is received from controller via the same socket.

So the `distributed_island` de facto operates exactly the same as a `thread_island` would, only difference is that it offloads the evolution process to the controller (which then assigns it to a worker), instead of offloading it onto a new thread.

See figure 7.1, which visualizes what was described in the paragraphs above.

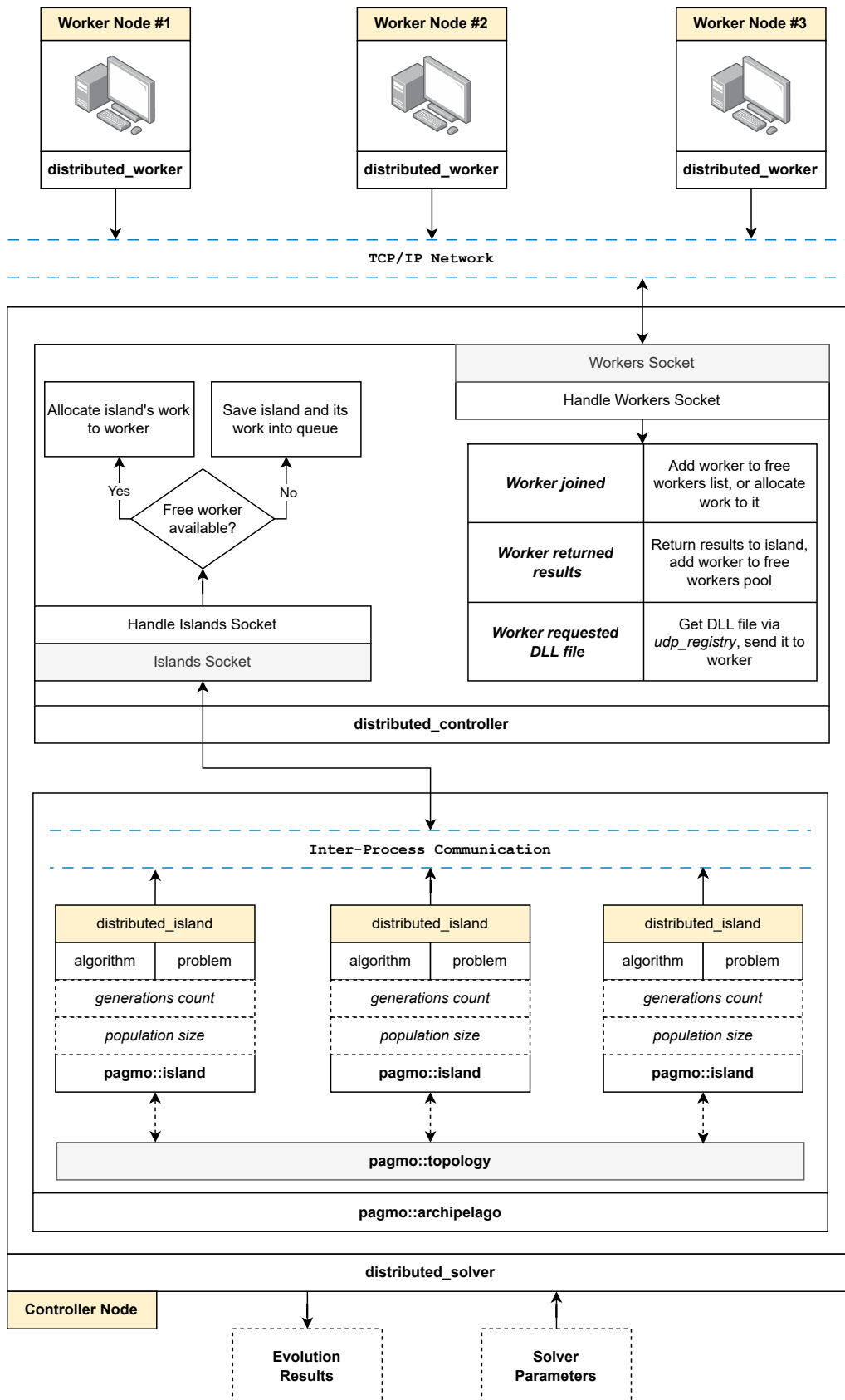


Figure 7.1: Architecture of the metasolver distributed system, showcasing three connected workers.

7.3 Example Usage

This section provides us with example code and a quick explanation regarding how this library is supposed to be used.

The `./src/run_example/main.cpp` file can be used to run the solver on a benchmarking suite or to start a worker, it is configurable by various command line parameters.

However, usually you'd want to create the `distributed_solver` object directly in your application and use it to solve your own optimization problems, listing 7.1 shows the minimal example code on how to create an instance of `distributed_solver` and use it to solve a problem.

Source code 7.1: A minimal controller node

```

1 #include "distributed_solver.h"
2 #include "udp_dll_wrapper.h"
3 #include <pagmo/problem.hpp>
4 #include <pagmo/algorithms/de.hpp>
5
6 int main()
7 {
8     // Optional: set directory containing our UDP DLLs
9     udp_registry::get().set_local_cache_dir("controller_cache");
10
11     // Construct your OWN problem here (using schwefel as an example)
12     unsigned dim = 2;
13     udp_dll_wrapper udp{"schwefel_udp", dim};
14     pagmo::problem problem{udp};
15
16     // Construct an algorithm (or multiple algorithms)
17     pagmo::algorithm algo{pagmo::de(1000)};
18
19     // Construct a distributed solver (controller starts internally)
20     distributed_solver solver{
21         "tcp://0.0.0.0:5000", // Controller address
22         8, // Expected number of workers
23     };
24
25     // Wait for workers to connect
26     solver.wait_until_workers_connect();
27     // Run distributed evolution, population size 1000
28     solver.evolve(problem, {algo}, 1000);
29     // Wait for completion and get the best individual
30     pagmo::vector_double best = solver.wait_until_completion();
31
32     return 0;
33 }

```

Listing 7.2 shows the minimal code to create a “worker node”, as opposed to the controller, we don’t need to collect any results or data in the worker’s program, so the exactly same behavior can be achieved by starting the worker via `./src/run_example/main.cpp`.

Source code 7.2: A minimal worker node

```
1 #include "distributed_worker.h"
2 #include "udp_registry.h"
3
4 int main()
5 {
6     // Construct worker (connects to controller)
7     distributed_worker worker{
8         "tcp://localhost:5000" // controller address
9     };
10
11     // Optional: where UDP DLLs are cached
12     udp_registry::get().set_local_cache_dir("worker_cache");
13
14     // Let worker get UDP DLLs from the controller when needed
15     udp_registry::get().register_udp_provider(
16         [&worker](const std::string& libName)
17         {
18             return worker.get_dll_from_controller(libName);
19         }
20     );
21
22     // Main worker loop (or use run_client() to run on background thread)
23     for (;;)
24     {
25         worker.client_loop();
26     }
27
28     return 0;
29 }
```

The Network Protocol

8

The sections in this chapter explain how the exchange of messages between the controller and worker is structured, without mentioning implementation details such as specifics of the sockets or how the messages are serialized. See chapter 9 if you're interested in these details.

8.1 Message Structure

Our protocol is built on top of the ZeroMQ “multi-part” messages [59] feature, meaning that each message is a **sequence of frames**, these frames are delivered atomically in-order as **one logical message**.

This is indeed very useful because each field of our custom protocol can be a single **frame** in the ZeroMQ message. These frames are automatically assembled into a message on the sender side, and then fragmented back into individual frames upon reception. We therefore don't need to implement any framing system such as protocol **field delimiters** or message **terminator sequence** as ZeroMQ already handles all of that for us.

The Controller-Worker message exchange protocol only uses two primary fields: **Message Type** and **Payload**, where **Payload** is an arbitrary-length object the type of which is determined by **Message Type**. Additional **Identity**¹ field is automatically added by ZeroMQ when sending messages from the worker to controller.

Figure 8.1 shows us how this works in practice.

¹“Identity” is a **Routing ID** which allows the controller to identify workers, IP addresses aren't used for identification because the ZeroMQ protocol is an “abstraction” which can work over different protocols than only TCP, e.g. over IPC communication channels.

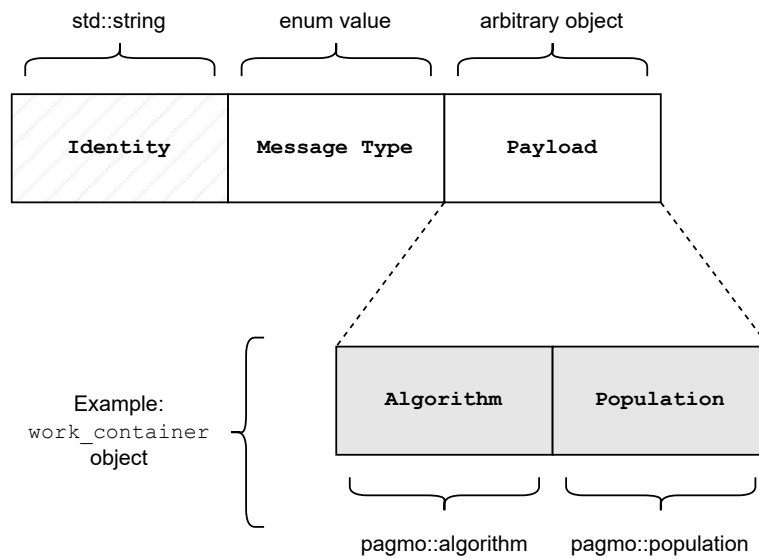


Figure 8.1: Structure of the protocol's messages. The Payload field can contain an arbitrary serialized object, the figure shows how the payload would look like with a `work_container` object inside of it.

MsgType	Direction	Payload Object	Purpose
WORKER_JOIN	W → C	—	Register a new worker with the Controller.
WORKER_LEAVE	W → C	—	Generated on worker disconnect.
ALLOCATE_WORK	C → W	work_container	Send algorithm, population, and other params.
WORK_RESULTS	W → C	work_container	Return evolved population together with the algorithm object.
GET_DLL	W → C	get_dll_request	Request a UDP DLL by name.
DLL_BINARY	C → W	dll_binary_container	Deliver the UDP DLL or empty optional if not found.

Table 8.1: Overview of the different messages sent between a worker (W) and the controller (C). The `WORKER_LEAVE` message is not actually sent by the worker itself, rather it's internally generated by ZeroMQ when the worker's connection is determined to be closed (no heartbeat signal received for a certain period of time).

8.2 Message Exchange

The typical Controller-Worker message exchange is shown in figure 8.2, a worker must first send the `WORKER_JOIN` message to controller, in order to join the cluster.

The controller tracks how many workers are connected, together with their details, in a thread-safe data structure which can be accessed by any other part of the application by calling the controller's `get_worker_info_repository()` member function.

Worker is allocated work as soon as controller receives the work task from a `distributed_island`, before continuing, the worker must first check if the UDP (User-Defined-Problem) received from the controller is loaded and present in the worker's file system, if it's not, then the worker must request this DLL library from the controller before proceeding further.

Finally, the worker evolves the population and sends it back to the controller. This loop repeats as long as there's work to be allocated.

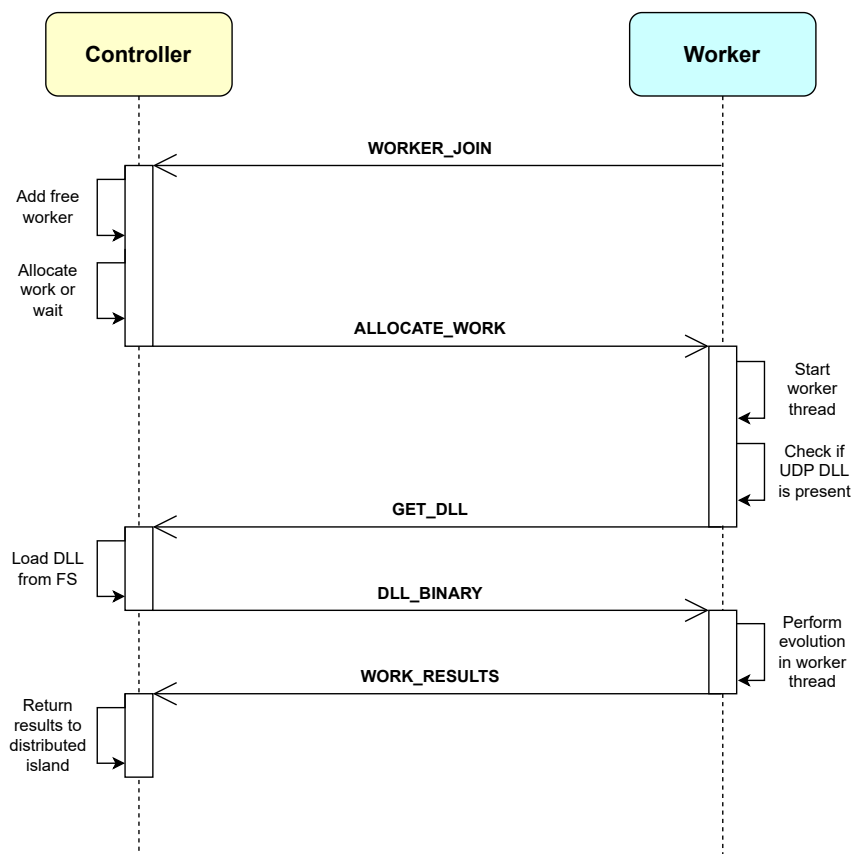


Figure 8.2: Sequence diagram showing the typical interaction between the controller and worker. The `GET_DLL` and `DLL_BINARY` steps are optional, they only happen if worker doesn't have the DLL cached.

Implementation of the Metasolver Library

9

In this chapter we'll explain each part of the library and its corresponding implementation files. Each of the following sections describes a single subdirectory in the library's source tree, see figure 9.1 for an overview.

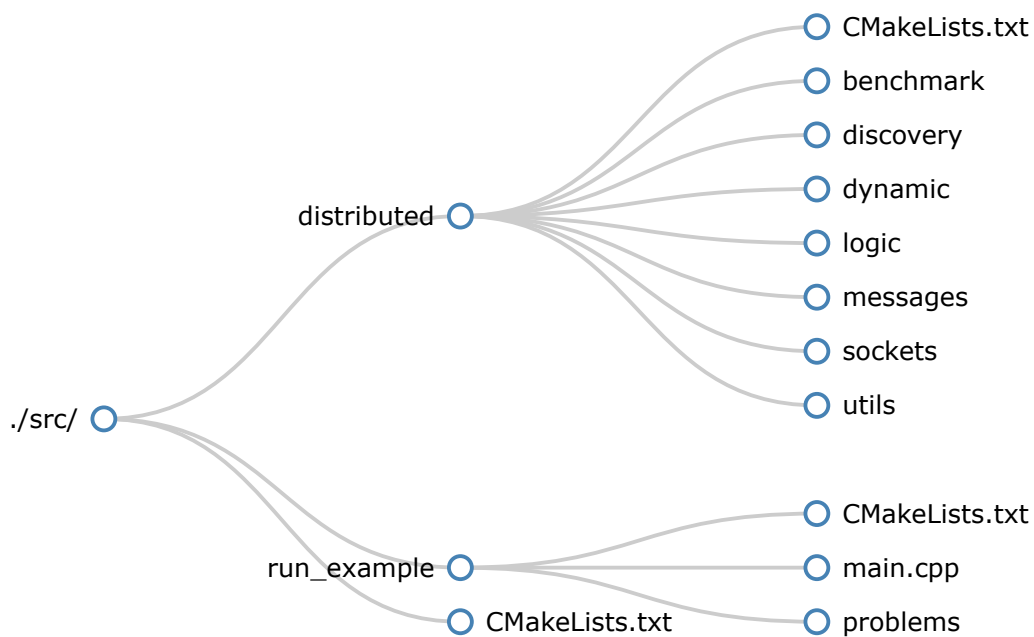


Figure 9.1: Subdirectory tree of the library, with root at the `./src/` directory.

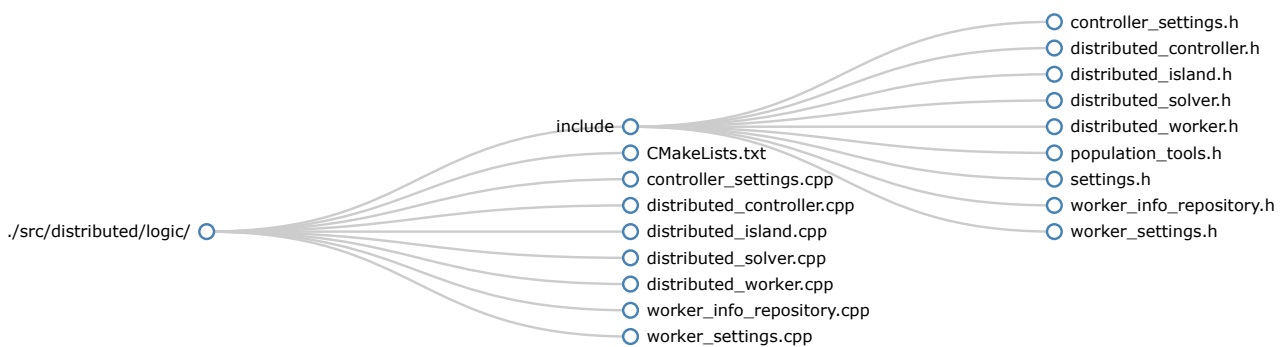


Figure 9.2: The logic subdirectory.

9.1 Logic

As was already discussed, this directory contains the library’s most important classes. All logic regarding work distribution, work allocation, load balancing or performing evolution is managed here.

Distributed_solver

The `distributed_solver` class serves as the primary way of creating a **controller node**, the features of this class are:

- Encapsulating and maintaining instances of `distributed_controller` and `pagmo::archipelago`
- Starting the distributed evolution process, filling archipelago with instances of `distributed_island`
- Collecting and returning results of the evolution
- Dividing work between workers (load-balancing)

It therefore acts as an “orchestration layer”, managing the underlying controller and archipelago, abstracting away these complexities from the user of this library.

We use the following constructor to make an instance of this class:

```
explicit distributed_solver(
    const std::string& controllerAddress,
    const size_t expectedWorkerCount = 1,
    load_balancing_strategy loadBalancingStrategy = BY_PERFORMANCE
);
```

`controllerAddress` is the address on which the controller server will start listening for workers.

`expectedWorkerCount` is an optional parameter which tells the class how many distributed islands to create, if not provided, only one island will be created by default, additional islands are then automatically created to match the amount of currently connected workers, this ensures that all workers will be utilized (one island per every worker).

`loadBalancingStrategy` is a useful parameter if we want to disable load balancing, all islands (and consequently workers) will have exactly the same population size if `load_balancing_strategy::ALL_ISLANDS_EQUAL` is passed here.

The constructor's body calls `_controller.run_server()`, starting the server and allowing workers to connect. At this stage, the `distributed_solver` class is idle until one of its member functions such as `wait_until_workers_connect()` or `evolve()` is called.

The `evolve()` function's signature looks like so:

```
void evolve(const pagmo::problem& problem,
           const std::vector<pagmo::algorithm>& algorithms,
           size_t populationSize,
           size_t cycleCount = 1,
           size_t minIslandPopSize = 80
);
```

Most of its parameters are self-explanatory, `algorithms` is a vector in order to allow metasolving (multiple algorithms), `cycleCount` determines how many *migration cycles* will be performed (i.e. exchanges of individuals between islands), this parameter is passed directly to the `archipelago`: `_archipelago.evolve(cycleCount)`.

The function's body performs the following steps:

1. If `archipelago` is busy, throw an exception (evolution is already running)
2. Reset `archipelago`'s state, i.e. delete any previous islands
3. Get current worker count from controller
4. Determine the island count by performing: `std::max(currentWorkerCount, _expectedWorkerCount)`;¹
5. Generate a “work plan” by calling `_generate_work_plan()` (ensures load balancing)
6. Create instances of `distributed_island` according to work plan, push them into `archipelago`

¹ If more workers connected than what was expected, it doesn't make sense to keep these workers idle, so we increase the island count to match the currently connected amount of workers.

7. Start the evolution by calling: `_archipelago.evolve(cycleCount);`

What enables load balancing in the `evolve()` function is the private member function `_generate_work_plan()`, its signature is:

```
std::vector<std::tuple<size_t, size_t, std::string, const pagmo::
    algorithm>> _generate_work_plan(
    const size_t islandCount,
    size_t populationSize,
    const std::vector<pagmo::algorithm>& algorithms, size_t
    minIslandPopSize
);
```

So, it takes mostly the same parameters as `evolve()`, with the addition of `islandCount`. The function returns a vector of tuples, where each tuple is in the format of: `[populationSize, cycleCount, preferredWorkerId]`.

These tuples are the **parameters of our islands**, the function determines the population size, cycle count and preferred worker for each island based on the **worker's performance**. The function's workflow can be summarized like so:

1. Get `worker_info_repository` from the controller, containing worker statistics
2. If there are more islands than connected workers, divide work between islands equally²
3. If $workerCount \geq islandCount$ perform for each island:
 - a) Get current algorithm (round-robin selection)
 - b) Find best worker in total, together with best worker for this algorithm
 - c) Choose a worker: the best for this algorithm is chosen, unless the best in total is at least 2 times better
 - d) Remove the chosen worker from the selection set
4. Calculate total performance of the worker cluster (from metrics calculated in the step above)
5. Calculate the percentage of performance that each worker represents in the cluster
6. Population size of each worker is determined in proportion to its percentage of performance
7. Return results

²This is an edge case, we cannot do "load balancing" if we don't know which workers will connect to these islands.

Distributed_island

The class `distributed_island` implements the Pagmo UDI (User-Defined-Island) interface, allowing it to be used inside of `pagmo::island`. It serves as a “proxy layer” between `pagmo::island` (contained within `pagmo::archipelago`) and the controller.

Archipelago assigns an algorithm and a population to `pagmo::island`, which then calls `distributed_island`’s `run_evolve()` function. The body of this function does the following:

1. Load population and algorithm from parent `pagmo::island`
2. Connect to controller server via IPC socket
3. Send an `ALLOCATE_WORK` message to the controller:

```
_dealerSocket->send(MsgType::ALLOCATE_WORK,
                    work_container{
                        initial_algo,
                        initial_pop,
                        _preferredWorkerId,
                        _cycleCount
                    });
```

4. Wait until controller returns results from worker
5. Pass results back to the parent `pagmo::island`

The controller has a private member variable: `distributed::router_socket _islandsSocket` which allows it to communicate with the individual distributed islands. More on that in the next section.

Distributed_controller

The main purpose of the controller is to act as a server to which workers connect. There are two sockets on which the controller listens for messages:

```
distributed::router_socket _workersSocket;
```

and

```
distributed::router_socket _islandsSocket;
```

An instance of `zmq::active_poller_t` is then used as an I/O multiplexing mechanism to allow asynchronous handling of messages from both sockets:

```

_poller.add(_workersSocket.get_socket(), zmq::event_flags::pollin,
            [this](zmq::event_flags e)
            {
                _handle_Workers_Socket_Msg();
            });

_poller.add(_islandsSocket.get_socket(), zmq::event_flags::pollin,
            [this](zmq::event_flags e)
            {
                _handle_Islands_Socket_Msg();
            });

```

The controller utilizes reliability features of the ZeroMQ protocol, this can be configured through constructor parameters:

```

explicit distributed_controller(
    const std::string& controllerAddress,
    int heartbeatInterval = 1000,
    int heartbeatTimeout = 3000,
    int reconnectIvlMax = 1000,
    const std::filesystem::path& settingsFilePath = "./
    controller_settings.xml"
);

```

All of the parameters except for `controllerAddress` are optional, `heartbeatInterval` specifies the interval between “ping” messages sent from the controller to each of its workers. A `WORKER_LEAVE` message is automatically generated if worker doesn’t respond to the heartbeat within `heartbeatTimeout` seconds. `reconnectIvlMax` specifies the maximal TCP SYN packet interval if the worker is not reachable.

`settingsFilePath` specifies a “settings” file into which controller saves statistics about the workers, further explanation regarding this is provided in a separate section.

The controller’s primary activity is to allocate “work tasks” provided by `distributed_island` to any free workers, for this it uses three member variables: `_islandsWaitingForAlloc`, `_freeWorkersPool`, `_workAllocationMap` and three member functions: `_add_free_worker()`, `_allocate_island_work()` and `_allocate_worker_to_island()`. The work allocation process can be summarized as:

1. `ALLOCATE_WORK` message is received from `distributed_island`
2. `_allocate_island_work(islandId, workData)` is called
3. If there are no free workers (`_freeWorkersPool` is empty) emplace `(islandId, workData)` into `_islandsWaitingForAlloc`, otherwise continue with the next step

4. `_allocate_worker_to_island(islandId, workData)` is called
5. Find worker which can accept this work task, choose `workData.preferredWorkerId` or a random worker if not available
6. Erase chosen worker from `_freeWorkersPool`
7. Store the `workerId`, `workData` and `islandId` in `_workAllocationMap`, this ensures that we can re-allocate the `workData` if the worker disconnects
8. Send `ALLOCATE_WORK` message to worker
9. Wait until worker completes work
10. Return results back to `distributed_island`
11. Call `_add_free_worker()`, this function either allocates a new work task to this newly freed worker, or keeps the worker waiting in `_freeWorkersPool`

Apart from what was described above, the controller also logs statistics about workers into its `_settings` object, this includes measuring the time it took each worker to finish its work task or tracking the amount of currently connected workers.

Distributed_worker

The worker acts as a network **client**, which connects to the controller **server** and waits until work (`pagmo::algorithm` and `pagmo::population`) is allocated to it.

The worker then proceeds to evolve this population using the provided algorithm, a new thread is spawned to perform this work (`_workerThread`).

The `distributed_worker` class uses two sockets, the first one is used for communication with the controller:

```
distributed::dealer_socket _workerSocket;
```

and the second one is used for communication with the worker thread:

```
distributed::router_socket _threadSocket;
```

Similarly to the controller, a `zmq::active_poller_t _poller` is used to asynchronously handle incoming messages from both of these sockets. This shows the benefit of ZeroMQ sockets, they can be used for both inter-thread communication and communication over the network, without adding any external synchronization or signaling mechanism.

The worker's constructor is similar to that of the controller:

```

explicit distributed_worker(
    const std::string& controllerAddress,
    worker_mode workerMode = worker_mode::ARCHIPELAGO_BASED,
    size_t minIslandPopSize = 80,
    int heartbeatInterval = 1000,
    int heartbeatTimeout = 300000,
    int reconnectIvlMax = 10000,
    const std::filesystem::path& settingsFilePath = "./worker_settings.
xml"
);

```

The difference here is that `heartbeatTimeout` is by default set to 300000 milliseconds (5 minutes), which means that the worker will attempt to send a TCP SYN packet to the controller (establishing a TCP connection) roughly every 5 minutes. ZeroMQ internally decreases the value of `heartbeatTimeout` if the controller suddenly disconnects (so we can quickly re-connect), `heartbeatTimeout` returns back to the original value if the controller is unavailable for an extended time period, to prevent flooding the network with TCP SYN packets.

The next parameter is `workerMode`, which specifies how will the population be evolved. The `ARCHIPELAGO_BASED` worker mode utilizes Pagmo's standard parallel Archipelago, allowing parallel evolution across all of the worker's CPU cores, whereas the `SINGLE_THREADED` mode simply calls `algo.evolve(pop)` performing the evolution on a single thread. The single threaded mode can be useful if we're testing multiple workers on a single machine.

See the steps below for an example of how an `ARCHIPELAGO_BASED` worker operates:

1. `ALLOCATE_WORK` message is received from the controller
2. `_start_worker_thread(workData)` is called
3. Worker thread starts
4. `workData` is deserialized, if the `pagmo::population` contains an instance of `udp_dll_wrapper`, then the UDP DLL is loaded or requested from the controller if it doesn't exist
5. `_archipelago_based_worker(algo, pop, cycleCount)` is called
6. Create socket for communicating with the parent thread
7. Determine the optimal island count based on the node's core count
8. Split the population into equally sized portions for each island
9. Initialize parallel archipelago with islands

10. Run evolution on all islands in parallel
11. Merge population from all islands into a single population
12. Choose best individuals out of this merged population (so the output population size is the same as the input)
13. Send `WORK_RESULTS` containing the output to the main thread
14. Main thread sends `WORK_RESULTS` to the controller

The purpose of step 12 is to make the population the same size as what was passed to us by controller, this is because each island has a **minimal population size**, step 8 will adjust population sizes for each island to respect this and fill the rest with random individuals.

Settings

The `settings` class was created to allow saving and loading of different data structures from a file in the local filesystem. This is achieved by using Boost XML serialization.

As was already mentioned in the worker and controller sections, there are two structures: `worker_settings` and `controller_settings` which are saved as an XML file. The `settings` class is what provides us with the loading and saving functionality, it's a template class parameterized by `SettingsStruct`.

To better understand this, let's look at the signature of this class:

```
template <typename T>
concept Initializable = requires(T t)
{
    { t.initialize() } -> std::same_as<void>;
};
```

```
template <Initializable SettingsStruct>
class settings
{
    // Location of the settings file
    std::filesystem::path _settingsPath;
    SettingsStruct _settings{};
    bool _autoSave;

    // ... rest of the class continues here ...
};
```

It accepts any class satisfying the concept `Initializable` as its template parameter, and keeps an instance of it stored in the `_settings` member variable.

The two public functions of this class are:

```

void save()
{
    _save_to_file();
}

SettingsStruct& operator ()()
{
    if (_autoSave)
    {
        // makes sure that the previous version is always saved
        _save_to_file();
    }
    return _settings;
}

```

Allowing either manually saving the structure into a file or having it automatically saved each time the overloaded () operator is invoked.

The initialize() function is used to generate default values which are put into the settings file upon creation, for example the worker_settings struct automatically generates an workerId:

```

struct worker_settings
{
    std::string workerId;

    void initialize() {
        workerId = "worker_" + uuid::v4::UUID::New().String();
    }
};

private:
    friend class boost::serialization::access;

    template <typename Archive>
    void serialize(Archive& ar, unsigned)
    {
        ar & BOOST_SERIALIZATION_NVP(workerId);
    }
};

```

The controller_settings struct is defined similarly, only difference is that it stores an instance of worker_info_repository instead of a string.

Other Files

The remaining two files in the subdirectory are population_tools and worker_info_repository.

`population_tools` contains helper functions simplifying the process of modifying a `pagmo::population`.

`worker_info_repository` contains the classes `work_stats`, `worker_info` and `worker_info_repository`:

```
struct work_stats
{
    uint64_t workTime = 0;
    uint64_t processedPopulation = 0;
    // ... boost serialization code omitted ...
}

struct worker_info
{
    work_stats totalStats{};
    std::unordered_map<std::string, work_stats> statsByAlgorithm{};

    // Place to save a timestamp when worker_started_work() is called
    std::chrono::high_resolution_clock::time_point lastWorkStartTime;
    // ... boost serialization code omitted ...
}

class worker_info_repository
{
    // Protect members of this class
    std::mutex _mtx{};

    // Notifying when the number of connected workers changes
    std::condition_variable _cv;
    std::unordered_set<std::string> _connectedWorkers;
    std::unordered_map<std::string, worker_info> _workerRecords;

    // ... public member functions continue here ...
}
```

The simplest of these classes is `work_stats`, it stores the information about how much time has worker spent performing computations, and how many population members it has processed during that time. Dividing `processedPopulation` by `workTime` gives us a good estimation of the worker's processing power.

We're assuming that `processedPopulation` and `workTime` will never overflow, because the maximal `uint64_t` value is: 18 446 744 073 709 551 615, which if we're using milliseconds translates to 584 942 417 years.

`worker_info` stores total `work_stats` (for every algorithm) and then separate `work_stats` for each algorithm that the worker has used. These statistics are updated by the `distributed_controller` and then used by `distributed_solver` to perform load balancing.

`worker_info_repository` then stores an instance of `worker_info` for each worker together with a set of connected worker IDs. All public member functions of this class are thread-safe, allowing the safe modifications of these structures from multiple different threads.

9.2 Messages

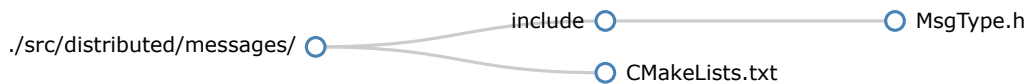


Figure 9.3: The messages subdirectory.

This subdirectory consists of a single header file `MsgType.h`, it contains definitions of various classes which are used as message payloads, and a definition of the `MsgType` enum:

```
enum class MsgType
{
    WORKER_JOIN,
    WORKER_LEAVE,
    ALLOCATE_WORK,
    WORK_RESULTS,
    GET_DLL,
    DLL_BINARY
};
```

This enum specifies every message type used by our protocol, the `distributed_controller` and `distributed_worker` use these values to identify which payload object is contained within the message, see the following snippet of code:

```
auto [workerId, type, binary] = _workersSocket.receive();
switch (type)
{
    case MsgType::WORK_RESULTS:
    {
        const auto workResults = vector_deserialize<work_container>(
            binary);
        // ... function body continues here ...
    }
}
```

The payload classes are: `work_container` used to transfer the algorithm and population, `get_dll_request` used by worker to request an UDP DLL library of specific name, and `dll_binary_container` used to transfer the DLL file as a vector of bytes.

These three classes are enough for everything that our protocol needs to accomplish, the same `work_container` class is reused for both `ALLOCATE_WORK` and `WORK_RESULTS` messages.

9.3 Sockets

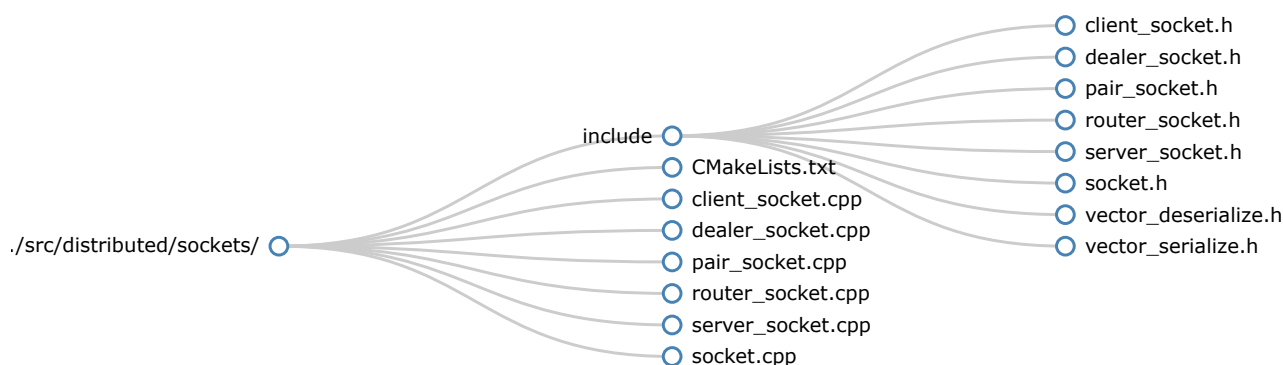


Figure 9.4: The sockets subdirectory.

The purpose of the sockets subdirectory is to create a **wrapper** around standard ZeroMQ sockets, this was done for two reasons:

1. The protocol consists of multiple message frames, the `send()` member function implemented by our wrapper classes always ensures that a valid message is sent. Correspondingly, the `receive()` member function always ensures that a valid protocol message is received.
2. ZeroMQ by default doesn't provide any serialization, so we add our own serialization code by creating a wrapper around the ZeroMQ socket.

Socket Types

An OOP hierarchy was created to correctly implement the different types of sockets, see fig. 9.5.

The `socket` class acts as a base from which all other sockets inherit, it encapsulates the `zmq::socket_t` object as a protected member variable, allowing any derived classes to access it. The two public member functions of this class are `bind(address)` and `connect(address)`, these are the same for every socket regardless of its type.

The hierarchy then splits into a `server_socket` and a `client_socket`, to illustrate the difference between them, let's look at the following pseudocode:

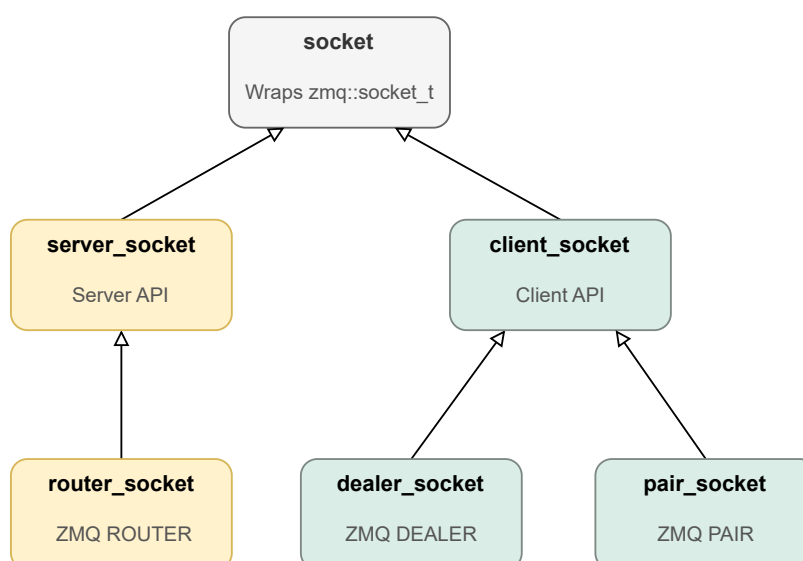


Figure 9.5: OOP inheritance hierarchy of the socket classes.

```
// server_socket API
send(id, msgType, payload)
receive() -> (id, msgType, payload)
```

```
// client_socket API
send(msgType, payload)
receive() -> (msgType, payload)
```

As we can see, the server's API needs to include an **ID**, which identifies the destination client. Naturally, a client socket doesn't need this because it's connected to a single server.

This aligns with what was discussed in chapter 8, the `router_socket` sends a message in the format of (Identity, Message Type, Payload), whereas the `dealer_socket` sends only (Message Type, Payload).

An additional `pair_socket` was added to facilitate point-to-point inter-thread communication, this socket ended up being unused in the final version of this library, and is thus reserved for future use.

Serialization

The Boost Serialization library is used for serialization and deserialization of the message payload, the two key files here are `vector_serialize.h` and `vector_deserialize.h`.

For an example, let's look at the `serialize` function:

```
template <typename Serializable>
```



```

std::vector<std::byte> vector_serialize(const Serializable& payload)
{
    std::vector<std::byte> serialized;
    vector_streambuf buf(serialized);
    std::ostream os(&buf);
    boost::archive::text_oarchive oa(os);
    oa << payload;
    os.flush();
    return serialized;
}

```

It's a template function allowing us to serialize any serializable object into a byte vector. If the object isn't serializable (doesn't contain Boost serialization functions), then a compile time error is produced. All of our sockets call `vector_serialize` in their `send()` function, serialization thus becomes a part of the message sending process abstracting it away from the programmer.

The `vector_deserialize` function is implemented similarly, the difference here is that this function is called directly in the `distributed_controller` and `distributed_worker` classes, as was already shown in previous sections.

A decision has been made to use `boost::archive::text_oarchive` i.e. a text-based serialization format, this reduces the possibility of incompatibilities between different platforms and standard library implementations. If needed, switching to `boost::archive::binary_oarchive` is trivial, as it only requires changing a single line of code in the `serialize` and `deserialize` functions.

9.4 Dynamic

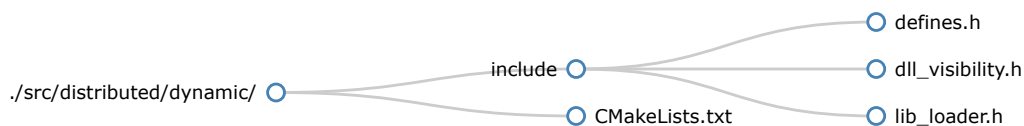


Figure 9.6: The dynamic subdirectory.

To allow the runtime loading of UDPs (User Defined Problems) from dynamic libraries, instead of forcing them to be statically linked to every worker, it was first needed to implement an OS-independent system which would be able to load these dynamic libraries into the process' address space and then create an instance of the encapsulated UDP.

This is what the class `lib_loader` does, it's able to load and create an instance of any library which implements the following interface:

```

extern "C" DLL_PUBLIC T* allocator(const std::any& params);
extern "C" DLL_PUBLIC void deleter(T* ptr);
extern "C" DLL_PUBLIC T* cloner(const T* other);
// Optional
extern "C" DLL_PUBLIC void run_after_load();

```

The typical usage is to first create an instance of this class, for example:

```
lib_loader<udp_base> loader{"/my_lib.dll"};
```

Only mandatory constructor parameter here is the path of the library we want to load, then we can proceed with opening the library and creating an instance:

```

loader.open_lib();
std::shared_ptr<udp_base> udp = loader.get_instance(<Your params here>);

```

The smart pointer is instantiated using the dynamic library's `allocator()` function, after the reference count reaches zero, the `deleter()` function is called. Additionally, we can also clone the smart pointer using the `loader.clone_instance()` function, this internally calls the `cloner()` function.

Portability

What ensures portability across different operating systems is the `defines.h` file, it contains wrapper functions which abstract away the OS-specific APIs, e.g.:

```

#if defined(WIN32)
    #include <Windows.h>
    using HANDLE_TYPE = HMODULE;
    inline HANDLE_TYPE portable_dlopen(const char* path)
    {
        return LoadLibraryA(path);
    }

    // ... other functions omitted ...
#elif defined(__linux__) || defined(__APPLE__)
    #include <dlfcn.h>
    using HANDLE_TYPE = void*;
    inline HANDLE_TYPE portable_dlopen(const char* path)
    {
        return dlopen(path, RTLD_NOW);
    }

    // ... other functions omitted ...
#endif

```

The `lib_loader` class can then simply call `portable_dlopen()`, `portable_dlsym()`, `portable_dlclosure()`, etc. without having to contain any OS-specific logic.

Similarly, the `dll_visibility.h` file defines a `DLL_PUBLIC` macro, which expands to the right OS-specific visibility macro at compile time.

9.5 Discovery

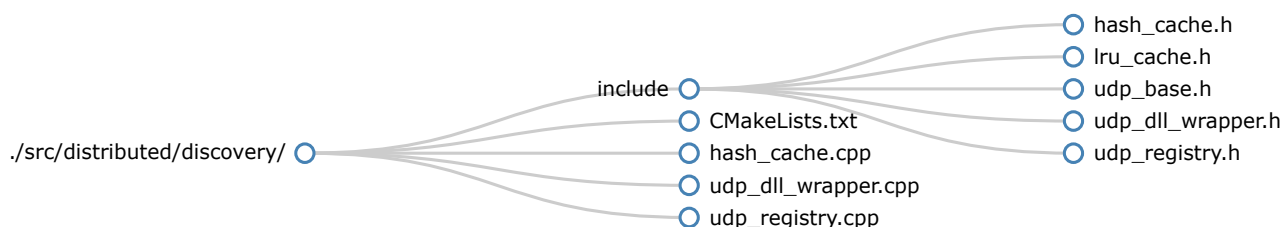


Figure 9.7: The discovery subdirectory.

Imagine that we have multiple UDPs (User Defined Problems), each of these problems is contained within a DLL file³, now, the way this system is designed, the DLL file might or might not be located in your local filesystem.

If it is in your local filesystem, then it might've already been loaded into your process by some previous activity, we need to keep track of this in order to prevent loading the library multiple times.

This is what the `udp_registry` class does, it's a centralized element which allows us to construct an UDP without worrying about the filesystem or any other underlying logic. It could be said that the `udp_registry` class acts as a **service discovery layer** (hence the name of this subdirectory).

"Service discovery" is something that's *globally unique*, think of e.g. how DNS domain names are globally unique. We need to achieve the same thing with UDP names, when requesting to construct an UDP, there should be one centralized element which decides what steps to perform to construct this UDP. This is why `udp_registry` is implemented as a thread-safe singleton object, allowing any part of the application to request an UDP while keeping the underlying logic centralized and conflict-free.

By following this design pattern, it was possible to keep the DLL management contained mainly within this subdirectory. This allowed great simplification of the `distributed_controller` and `distributed_worker`'s code, as the only thing these classes need to do is to transfer the DLL files between each other and then hand them over to the `udp_registry`.

³Or `.so` / `.dylib`, for simplicity we'll call any type of dynamic library a "DLL".

Key parts of the DLL management process are described in the following sub-sections.

udp_base

`udp_base` is the base abstract class from which every UDP is derived. Simply put, a Pagmo UDP must implement at least these two functions:

```
vector_double fitness(const vector_double &) const;
std::pair<vector_double, vector_double> get_bounds() const;
```

Apart from these, there are also many optional functions which a UDP might define. `udp_base` ensures that our UDPs follow this interface, this is highly useful because it allows `lib_loader` to load any UDP as an instance of `udp_base` without knowing its actual class name.

This is the only way a “DLL plugin system” can function, we need to have a base abstract class whose functions are then implemented by every “plugin”.

udp_dll_wrapper

Pagmo requires that each UDP must be copy constructible, Pagmo also doesn't allow passing a base class pointer as an UDP, for this reason it was needed to create the `udp_dll_wrapper` class:

```
class udp_dll_wrapper
{
    std::shared_ptr<udp_base> _udpPtr{};
    std::string _libFileName;
    std::string _libFileHash;
public:
    pagmo::vector_double fitness(const pagmo::vector_double& dv) const;
    std::pair<pagmo::vector_double, pagmo::vector_double> get_bounds()
    const;
    pagmo::vector_double::size_type get_nec() const;
    // ... rest of the class continues here ...
```

As we can see, this class encapsulates a pointer to `udp_base` and acts as a proxy for its functions (e.g. `fitness()` in `udp_dll_wrapper` calls `fitness()` in `udp_base`).

Apart from that it also handles serialization and deserialization, it serializes itself by calling:

```
pagmo::detail::to_archive(ar, libFileHash, _libFileName, _udpPtr);
```

And then during deserialization the following steps are performed:

```
// 1) Deserialize _libFileHash and _libFileName first
ar >> _libFileHash;
ar >> _libFileName;
```

```
// 2) Attempt to load the dynamic library of this UDP
udp_registry::get().initialize_udp(_libFileName, _libFileHash);
// 3) Deserialize UDP, lib containing the UDP class should now be loaded
ar >> _udpPtr;
```

This is the important part, because calling `udp_registry::get().initialize_udp()` will obtain the DLL from the controller if needed, and then load it, making it possible to construct the UDP. The “DLL discovery” is thus handled as part of the deserialization process, the worker class doesn’t need to do anything apart from initiating the deserialization - everything happens in the background.

Similarly, `_udpPtr = udp_registry::get().construct_udp()` is called in `udp_dll_wrapper`’s constructor, this is how the UDP library gets loaded on the controller side.

udp_registry

The primary objective of `udp_registry` is to store an instance of `lib_loader` for each loaded UDP, and then facilitate the construction or cloning of these UDPs. It also needs a way to obtain the UDP DLLs from the controller, which is what this function allows:

```
void register_udp_provider(const udp_provider& providerFunc);
```

After constructing our worker, we can use `register_udp_provider` to pass the `distributed_worker::get_dll_from_controller()` function as a `providerFunc` to the UDP registry.

Apart from what was already mentioned, the `udp_registry` also provides a public member function `get_lib_as_file()`, which allows the controller to load the DLL file into a byte vector before sending it to the worker.

This means that the user of this library only needs to interact with `udp_registry` (to configure provider) and `udp_dll_wrapper` (to construct an UDP), all the other required steps are handled internally.

hash_cache

As you might’ve already noticed, when a UDP is deserialized on the worker’s side, there’s a file hash being passed to the `udp_registry::initialize_udp()` function. Controller always sends the hash of the DLL file to worker, allowing the worker to check if its local version of the DLL is the same as the controller’s version. If the two hashes aren’t equal, a new version of the DLL is requested.

This means that the two hashes must be compared each time the worker receives a work allocation. To speed up this process a `hash_cache` was created, the hash is

computed only once instead of loading the DLL file into memory and re-computing it each time.

A LRU cache is internally used to store the hashes, meaning that old hashes (not recently used) will be evicted from the cache, preventing excessive memory usage on the worker node.

9.6 Utils

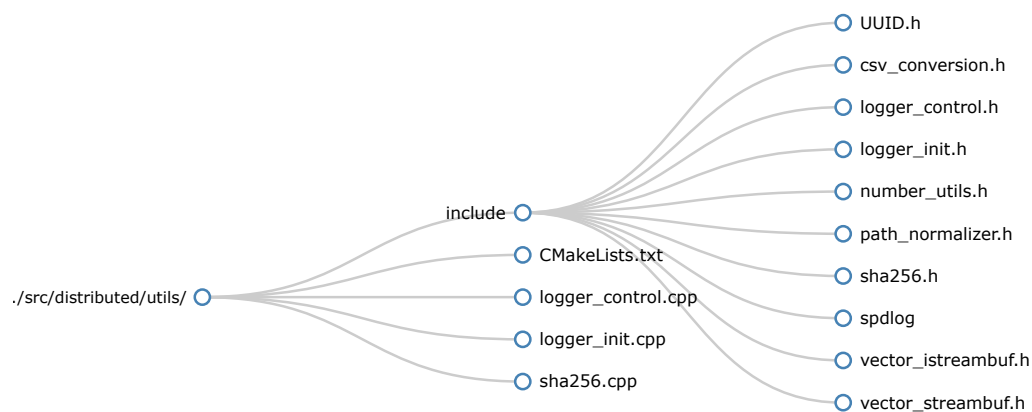


Figure 9.8: The utils subdirectory.

The utils subdirectory contains various files which thematically don't fit into any other subdirectories of this project. The list below provides a short summary about each of them:

- The library uses **spdlog** for logging, two helper files were created, first `logger_init` which either enables or disables the logger. The second is `logger_control`, a class which ties the logger's lifetime to any object which inherits from it.
- An implementation of the **sha256** hashing algorithm from the open source *Portable C++ hashing library* [60] was added, this algorithm is used by `udp_registry`.
- `UUID.h` provides a function for generating UUIDs, this is used to generate worker IDs.
- `vector_streambuf.h` and `vector_istreambuf.h` are classes which allow the usage of `std::vector<std::byte>` as buffers for Boost serialization.

- `path_normalizer.h` normalizes a filesystem path to remove any symbols which are reserved by Windows.
- `number_utils.h` contains abstract re-usable numerical functions which were moved here when refactoring other parts of the project.
- `csv_conversion.h` is used by the benchmark classes to convert CSV tables into HTML tables.

9.7 Benchmark

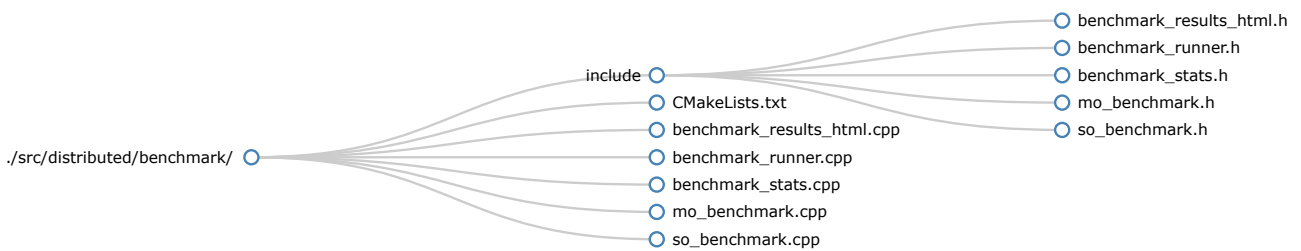


Figure 9.9: The benchmark subdirectory.

Classes in the benchmark subdirectory are used only for debugging or testing purposes. They allow us to measure the quality of different solver configurations or to run a set of standardized problems. The `./src/run_example/main.cpp` file is configured to run a benchmark if the `--benchmark` command line parameter is passed to it.

`benchmark_so.cpp` and `benchmark_mo.cpp` allow us to run single-objective and multi-objective benchmarks, the functions in these files internally use `benchmark_runner.cpp`.

`benchmark_stats.cpp` is a class for collecting statistics of each algorithm configuration that was executed by the solver, it then allows us to visualize them either by producing a CSV file, or a HTML file by utilizing `benchmark_results_html.h`. This can be useful if we for example want to test multiple configurations of the solver and then compare them against each other.

9.8 Unit Tests

Even though unit testing is beyond what was declared as the goals of this thesis, an attempt was made to generate unit tests using LLM-based AI tools, in the hopes of improving the future maintainability of this library.

Three different language models were used:

- **Grok** (Grok 4.3) [61]
- **DeepSeek** (DeepSeek-V4-Flash) [62]
- **GitHub Copilot** (Claude Haiku 4.5) [63]

Each of the language models was provided exactly the same prompt, consisting of the following sections:

- *Project info*
 - Basic information about the project.
- *Your task*
 - Technical details about how unit testing should be implemented.
- *Expected output*
 - Specifying that the LLM's output should be a script executable in a Linux terminal, creating files and inserting code as needed.
- *Terminal PWD*
 - Information about the environment where the output script will be executed.
- *Unit test implementation plan*
 - Each LLM was prompted to prepare an implementation plan, the best one was chosen and provided as a part of this prompt.
- *Catch2 documentation*
 - Documentation of the Catch2 unit testing framework.
- *Project code*
 - The code of all implementation (.cpp) and header (.h) files of this project. This included line numbers and filesystem locations of each file.

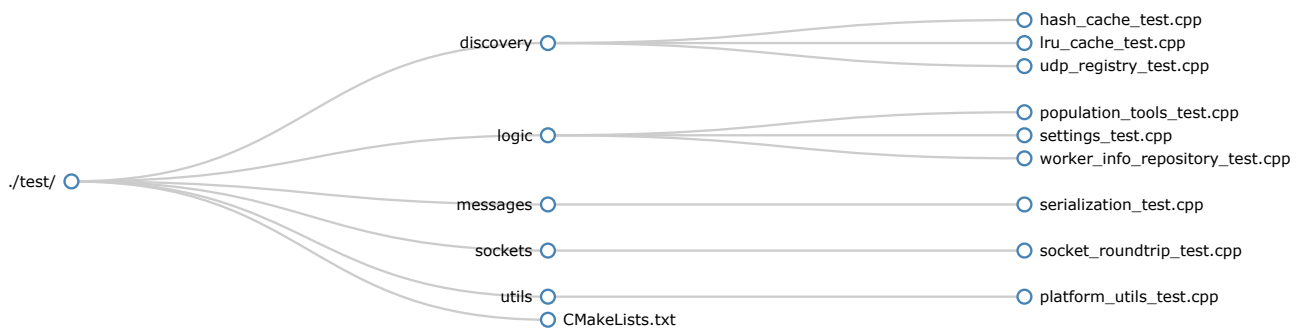
What each LLM produced is summarized in table 9.1, any tests involving `distributed_controller` or `distributed_worker` network operations were removed, as it's impossible to properly test the worker or controller without having the corresponding peer running somewhere on the network or in another process.

Tool	Model	Number of Tests	All Tests Pass
Grok	Grok 4.3	24	No
DeepSeek	DeepSeek-V4-Flash	13	No
GitHub Copilot	Claude Haiku 4.5	103	Yes

Table 9.1: Results of the generated unit tests by different LLMs.

Therefore the tests focused primarily on data structures, such as `lru_cache` or `settings`, and on various other logic which could be tested without having an active controller-worker connection.

As the data in table 9.1 shows, *GitHub Copilot* using the *Claude Haiku 4.5* model produced by far the best results, its implementation was thus merged into our library's repository. For an overview of which files and directories were created, see figure 9.10.

Figure 9.10: The `./test/` directory and its contents, as created by GitHub Copilot.

As we can see, the `/test/` directory is structured the same way as the `/src/distributed/` directory, grouping test files by what part of the library they correspond to.

A significant portion of tests is located in the `discovery` directory, here two data structures (`hash_cache` and `lru_cache`) are tested, the tests consist of adding items into the cache, clearing the cache, checking the cache's capacity, etc. The remaining file tests `udp_registry`'s filesystem related operations.

Next is the `logic` directory, again, two data structure classes are tested: `settings` and `worker_info_repository`. The tests here make sure that internal state of `worker_info_repository` is updated correctly depending on what member function is called and the filesystem functions of `settings` are verified. What remains are tests of various `population_tools` functions.

The remaining three directories each contain only a single test file, `serialization_test.cpp` checks if various message payload objects can be correctly serialized, `socket_roundtrip_test.cpp` tries to locally create two sockets and send a message between them, and lastly, `platform_utils_test.cpp` verifies that the correct OS-specific dynamic library extension is returned.

Implementation of the SCGMS Wrapper

10

The distributed metasolver that we've implemented is a **library**, which means that its intended purpose is to be used by other applications.

SmartCGMS, which was described in section 6.2, will be used in this chapter to showcase the possibilities of integrating our distributed metasolver into existing software, enabling the acceleration of an existing problem-solving architecture by making it distributed across many worker nodes.

To make the distributed metasolver available to SCGMS, it was necessary to create a **wrapper library**, adapting our solver so that it matches the SCGMS solver interface (see sec. 6.2).

All of the implementation files discussed in this chapter are available as an attachment of this thesis.

10.1 Overview

The wrapper (located in `/scgms-release/core/distributed4scgms`), consists of three different parts:

1. The core logic inside of `/distributed4scgms/src/` implementing the SCGMS solver interface. This compiles to a dynamic library file `distributed4scgms.dll`, which is placed inside of SCGMS `/filters/` directory and loaded at runtime. The most important file here is `distributed.h`.
2. The UDP subdirectory, we have already introduced the concept of UDPs in previous chapters, here we implement a **specific problem** from the SCGMS `PathFinder_test` submodule, making it available as a "User-Defined-Problem" to Pagmo algorithms.
3. The UDA (User-Defined-Algorithm) subdirectory was added to allow SCGMS algorithms to be used on Pagmo problems, this can be useful for debugging or when comparing the performance of SCGMS and Pagmo algorithms.

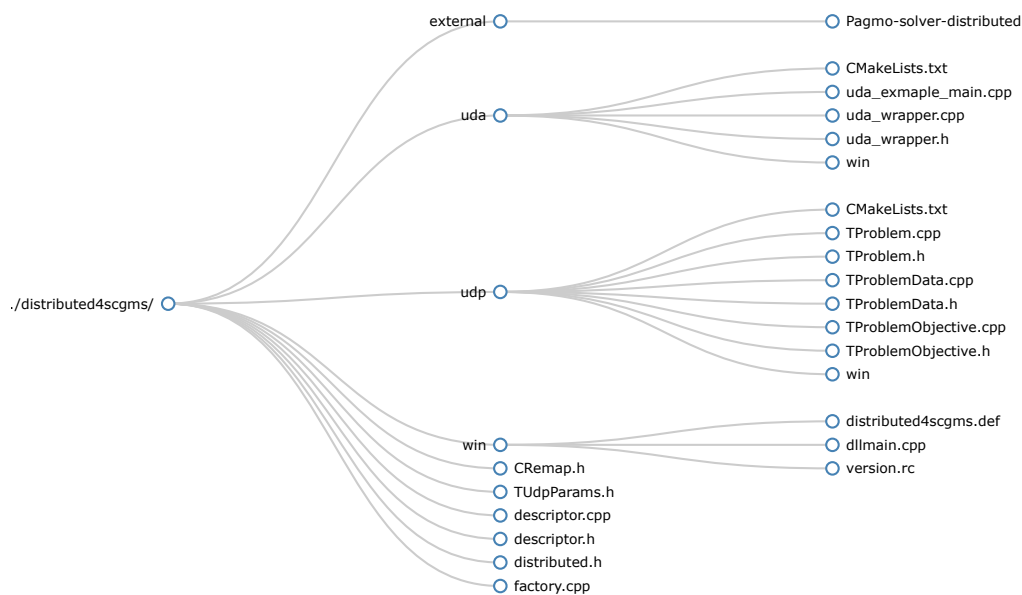


Figure 10.1: The distributed4scgms wrapper library, notice that the library is split into three parts. “solver-distributed” is the distributed metasolver library, it’s included as a git submodule.

10.2 Usage

In this section we’ll show how the **distributed4scgms.dll** wrapper can be used to solve a problem via SCGMS. As was already mentioned, we’ve used the `PathFinder_test`’s set of optimization problems and converted it into a Pagmo-compatible UDP.

To use this wrapper with any other SCGMS problem, a corresponding UDP must first be created. There is no way around this, as Pagmo requires the problem to have a specific interface, and we also need the problem to be serializable so it can be delivered to workers.

The steps below provide a high-level workflow of the problem-solving process:

1. `distributed4scgms` is compiled, the resulting DLL file is placed into `/build/compiled/filters`
 2. `tpproblem_udp` is compiled and placed into `/build/compiled`
 3. `pathfinder_test` is compiled and placed into `/build/compiled` as `pathfinder_test.exe`
-
4. User starts `pathfinder_test.exe`

5. `pathfinder_test.exe` internally loads `distributed4scgms.dll` from the `./filters` directory
6. An instance of `CDistributed_Solver` is constructed and `TSolver_Setup` is passed to it
7. `CDistributed_Solver::Solve()` is called by `pathfinder_test`, creating an instance of `distributed_solver`
8. The controller server is now running and waiting for workers to connect
9. User starts workers
10. Workers solve the problem
11. Controller collects results from workers
12. Results are returned back to the `pathfinder_test`

The **setup** structure mentioned in step 6 is crucial, it provides information about which UDP to use, the controller's address, etc. More about that in the next section.

10.3 Implementation Details

Solver Interface Wrapper

Everything in the `/src/` directory is centered around the `distributed.h` file, it contains a definition of the class `CDistributed_Solver`, its purpose is to accept `TSolver_Setup` in its constructor, and then operate the actual `distributed_solver` inside of its `Solve()` function. This is the usual pattern for SCGMS solvers.

`factory.cpp` is what allows SCGMS to create an instance of `CDistributed_Solver`, this is where the `do_solve_generic` SCGMS solver interface function is implemented. `descriptor.cpp` and `descriptor.h` implement the other solver interface function: `do_get_solver_descriptors`, which returns a GUID identifying this solver and a description of it.

We can see these functions being utilized in `pathfinder_test`:

```
solver::TSolver_Setup solver_setup{
    lower_bound.size(),
    1, // objectives count
    lower_bound.data(), upper_bound.data(),
    nullptr, 0, //no hints
    local_parameters.data(),
    &ds_data, nullptr, nullptr,
    max_generations, population_size, std::numeric_limits<double>::min(),
```

```
};
```

```
// This internally calls do_solve_generic  
solver::Solve_Generic(desc.id, solver_setup, solver_progress)
```

The two remaining files: `CRemap.h` and `TUdpParams.h` are used for converting SCGMS specific data formats to Pagmo-compatible structures. The `CRemap` class is responsible for converting the SCGMS solution vector format to a `pagmo::vector_double` by discarding fixed parameters. `TUdpParams.h` contains a `TUdpParamsSerializable` class, which converts the data from SCGMS `TSolver_Setup` into C++ vectors, this is necessary because we need to serialize in order to send it to our workers, the original `TSolver_Setup` contains C-style pointers which cannot be serialized.

UDP

The class `TProblem` inherits from `udp_base`, making it possible to be compiled as a DLL and then loaded via `lib_loader`. `TProblem` encapsulates instances of `CRemap` and `TUdpParamsSerializable`, additionally it also stores a shared pointer to `CCommon_Problem`.

`CCommon_Problem` is a class implemented in the `TProblemData.cpp` file, SCGMS passes a “data” object inside of `TSolver_Setup`, this data object is then supposed to be passed to the objective function each time it’s called. That is why we need to store this object inside of our UDP and make sure that it’s serializable so workers can operate upon it.

`TProblemObjective.cpp` contains the implementation of the problem’s objective function, this function is called inside of `TProblem::fitness()`, this way it’s possible to adapt the SCGMS-style objective function to be compatible with Pagmo.

UDA

The UDA subdirectory consists of `uda_wrapper` and an example main file which showcases how to use it.

The `uda_wrapper` class can convert any SCGMS solver into a Pagmo algorithm compatible with any Pagmo problem. The constructor of this class takes a UUID of an SCGMS solver as its parameter. Inside of the `evolve()` function the `TSolver_Setup` structure is prepared and filled with the correct values, `solver::Solve_Generic` is then called the same way as we saw in the pathfinder example above. The population is then evolved and results are returned back to the Pagmo problem.

This class therefore does the exact opposite of what was done in the `TProblem` class.

This chapter presents a number of experiments which were conducted to evaluate the performance characteristics of this distributed metasolver. The meaning of the measured data is later discussed in chapter 12.

In total three **nodes** connected over a Wireless LAN (WiFi) network were utilized in these experiments, their specifications are summarized in table 11.1.

Note that the **logical core** count of each CPU corresponds to how many islands is each node able to create locally when using worker's `ARCHIPELAGO_BASED` mode.

As was described in earlier chapters, the worker can run either in `SINGLE_THREADED` mode, where only one thread is utilized for the optimization algorithm, or in `ARCHIPELAGO_BASED` mode, where the worker internally uses `pagmo::archipelago` to spawn local `thread_islands` and split the population between them, utilizing all the available logical CPU cores.

Name	Microarch.	CPU model	C_p	C_L	Memory
Node A	Haswell	Intel Xeon E3-1231 v3	4	8	DDR3 16GB
Node B	Skylake-SP	Intel Xeon Gold 6138	20	40	DDR4 24GB
Node C	Skylake	Intel Core i5-6300U	2	4	DDR4 8GB

Table 11.1: Hardware specifications of the nodes used in this chapter. C_p stands for *Physical Cores*, whereas C_L is the number of *Logical Cores*.

11.1 Simple Distributed Scenario

Our first scenario involves creating the simplest possible distributed cluster consisting of two workers - one on the local machine, and the second on a remote node.

The exact same configuration (same problem, algorithm, population size, ...) is additionally also executed **only** on a single localhost worker, to see if distributing the optimization across multiple nodes brings any measurable benefit.

See figure 11.1 for an illustration, appendix B.1 shows source code of the controller program which was used to perform this experiment. The solver configuration specified within this program is summarized in table 11.2.

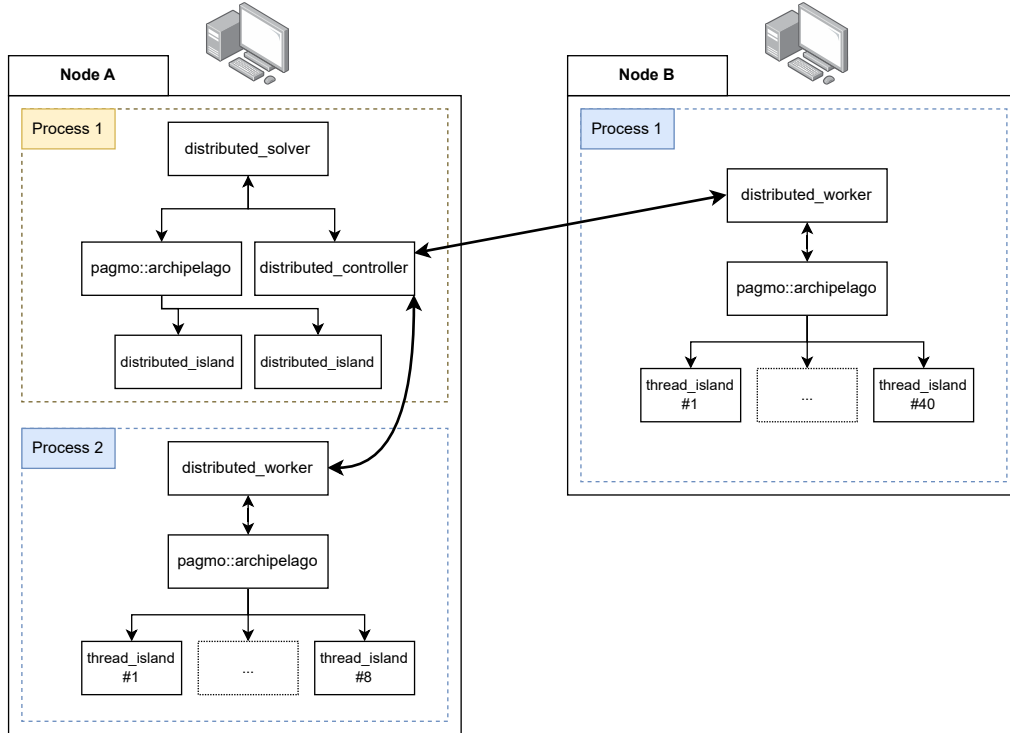


Figure 11.1: The simple distributed scenario. Notice that each node has the exact amount of `thread_islands` as its logical core count.

Table 11.2: Configuration of the simple distributed scenario's controller program.

Parameter	Value
<i>Problem</i>	
Problem class	Rastrigin
Dimensionality	30
<i>Algorithm</i>	
Algorithm type	Differential Evolution (DE)
Generations	2000
<i>First Step – Local</i>	
Workers	1 (local)
Total population	10 000
Population per worker	10 000
<i>Second Step – Distributed</i>	
Workers	2 (1 local + 1 remote)
Total population	10 000
Initial pop. per worker	5000 (dynamically adjusts)
<i>Output</i>	
Controller program	CSV statistics

Scenario Steps

The data presented in this section was obtained by performing the steps specified below. This should be reproducible if your hardware is similar enough to the two nodes (**Node A** and **Node B**) used in this scenario.

1. Compile the controller program (listing B.1) on **Node A** in **Release** mode.
2. Compile distributed-main-example.exe on both nodes in **Release** mode.
3. Run the controller on **Node A**. (EXE produced in step 1)
4. Run the local worker (port 5000) on **Node A**

```
.\distributed-main-example.exe --worker --address tcp://
localhost:5000 --mode "ARCHIPELAGO_BASED"
```

5. Run the distributed worker (port 5001) on **Node A**

```
.\distributed-main-example.exe --worker --address tcp://
localhost:5001 --mode "ARCHIPELAGO_BASED"
```

6. Run the distributed worker (port 5001) on **Node B**

```
.\distributed-main-example.exe --worker --address tcp://<IP-OF-NODE-A>:5001 --mode "ARCHIPELAGO_BASED"
```

7. Workers will now connect and perform their work.
8. Once both the “local step” and “distributed step” finish running, a CSV file with statistics will be saved to **Node A**’s local filesystem.
9. Run the controller program on **Node A** 9 more times, obtaining statistics from 10 consequent runs.

Scenario Results

The data obtained after executing the controller program 10 times is summarized in figures 11.2, 11.3, 11.4 and 11.5.

The *distributed step* (1 local worker + 1 remote worker) was on average 115% faster and the fitness of its solutions was about 24% better than that of the *local step* (single localhost worker).

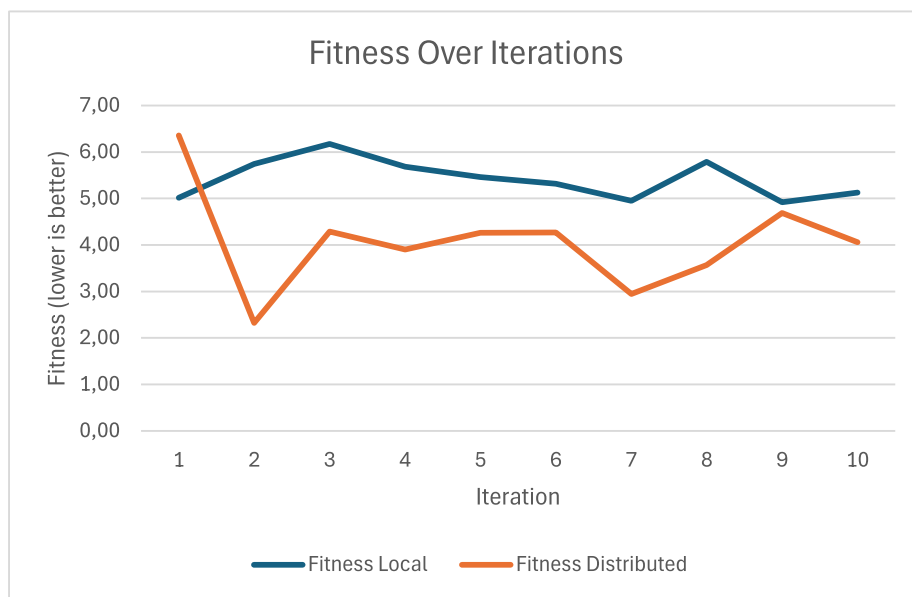


Figure 11.2: Solution fitness comparison between the “local step” and “distributed step”.

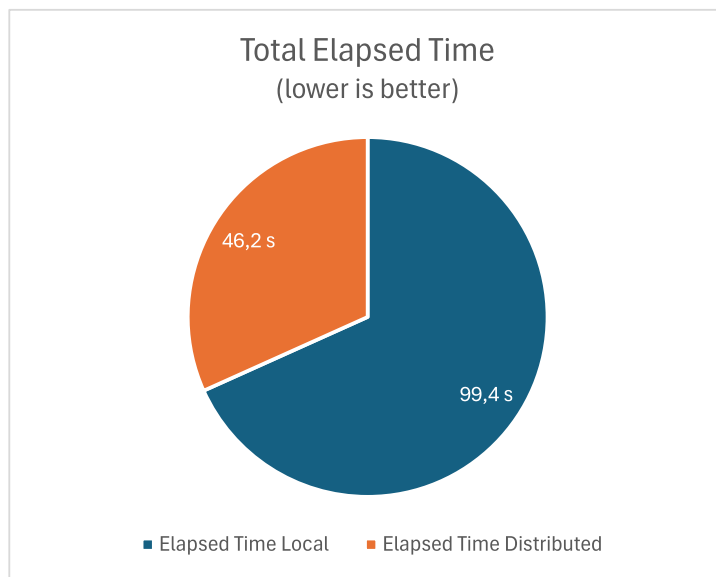


Figure 11.3: Pie chart showing the total time workers spent performing computations (over all 10 runs) for both the local and distributed variant. The “local step” took on average 9.94 seconds until results were returned to controller, in contrast to that the “distributed step” returned its results to controller on average in 4.64 seconds.

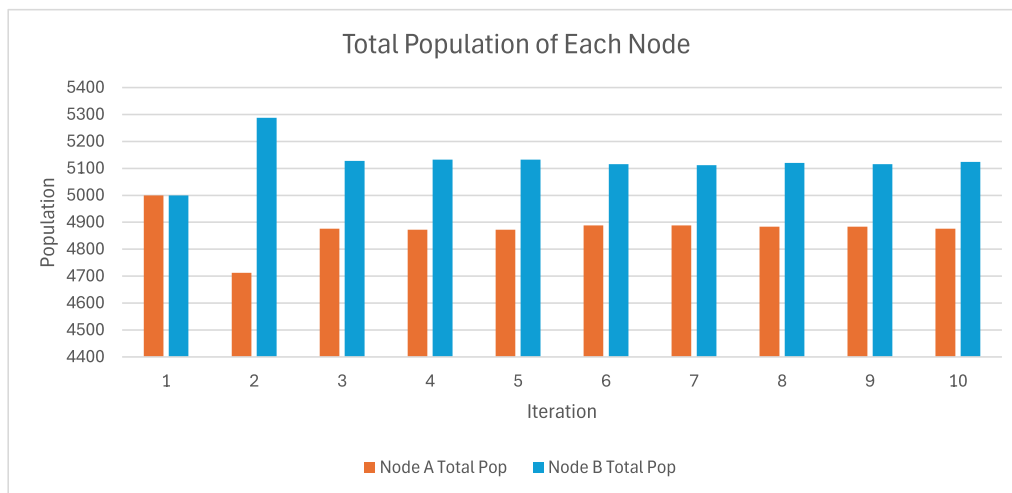


Figure 11.4: A chart showing how the solver’s load-balancing mechanism adjusts population sizes between two workers. During the first iteration population sizes for both workers are equal, as no data on their performance has been collected yet, this starts to change in subsequent iterations.

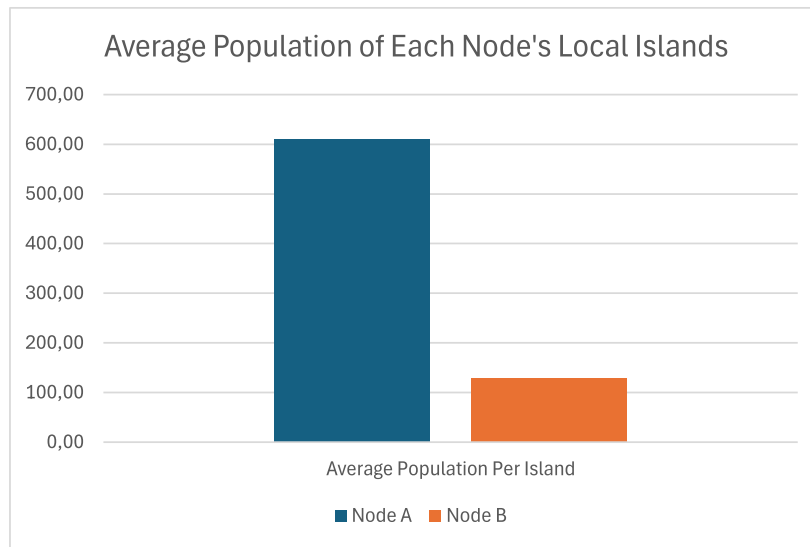


Figure 11.5: An `ARCHIPELAGO_BASED` worker splits the total input population into multiple parts, depending on the amount of local islands. There is one local island for every logical CPU core, meaning that **Node A** on average ends up with $\frac{4875}{8} = 609.375$ population members per island, as we can see **Node B** has a much lower per-island population due to its 40 logical cores: $\frac{5127}{40} = 128.17$.

11.2 Three-Node Distributed Scenario

This scenario can be seen as an extension of the “Simple Distributed Scenario” (sec. 11.1), only changes that were made are the addition of a third node: **Node C**.

The updated architecture can be seen in fig. 11.6. The controller program is exactly the same as in listing B.1, except that line 35 was changed to **3** workers.

So in each run of the controller program we still use the same configuration to collect data about two separate computations, which can later be compared:

1. Local computation performed only on a single localhost worker
2. Distributed computation performed on three workers (2x remote workers, 1x localhost worker)

Summary of the solver configuration for this scenario is provided in table 11.3.

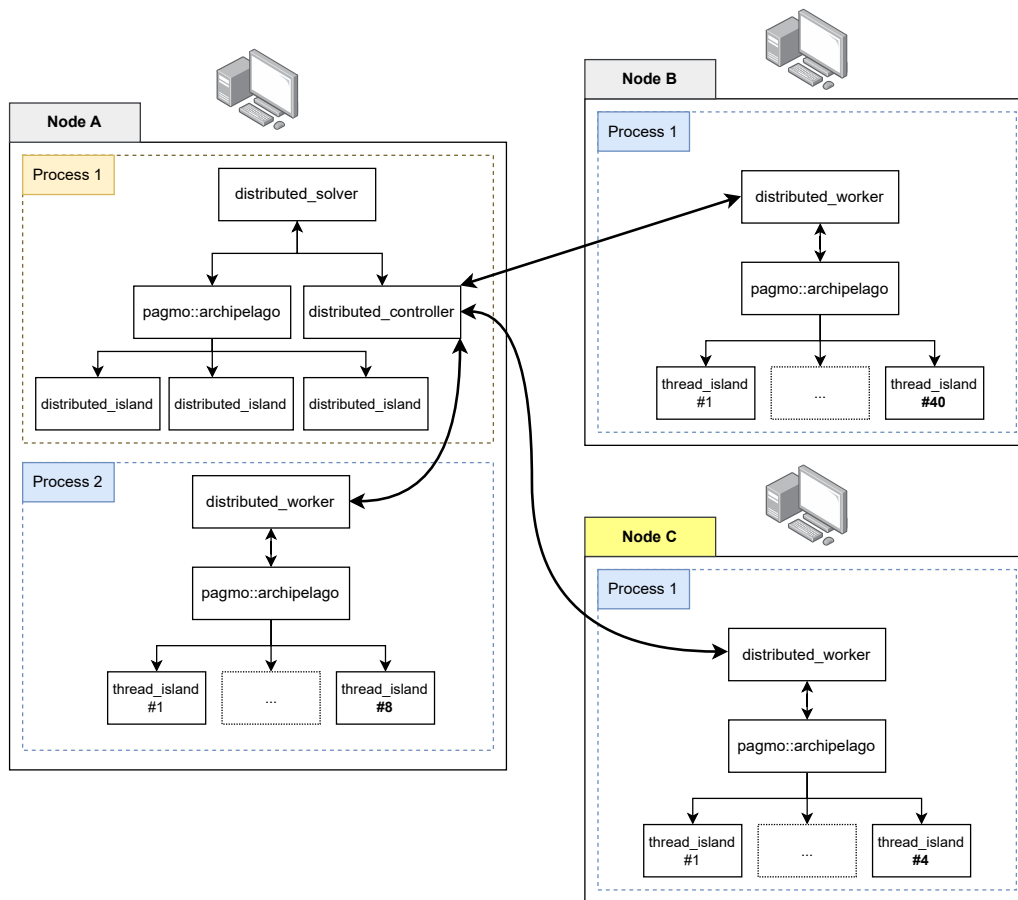


Figure 11.6: The Three-Node Distributed Scenario.

Scenario Steps

The steps of this scenario are the same as in section 11.1, with the addition of:

1. Compiling `distributed-main-example.exe` on **Node C** in **Release** mode.
2. Running the distributed worker (port 5001) on **Node C**

```
.\distributed-main-example.exe --worker --address tcp://<NODE-A-IP>:5001 --mode "ARCHIPELAGO_BASED"
```

Scenario Results

Figures 11.7, 11.8, 11.9, 11.10 and 11.11 summarize the data which was gathered by executing the controller program 10 times.

The *distributed step* is by 27.14% better in terms of average fitness than the *local step*, and better regarding the average time workers spent performing computations by about 113%.

So when compared to section 11.1, the average fitness of the *distributed step* slightly increased, whereas the time advantage is lower by 2%.

Table 11.3: Configuration of the three-node distributed scenario’s controller program.

Parameter	Value
<i>Problem</i>	
Problem class	Rastrigin
Dimensionality	30
<i>Algorithm</i>	
Algorithm type	Differential Evolution (DE)
Generations	2000
<i>First Step – Local</i>	
Workers	1 (local)
Total population	10 000
Population per worker	10 000
<i>Second Step – Distributed</i>	
Workers	3 (1 local + 2 remote)
Total population	10 000
Initial pop. per worker	~3333 (dynamically adjusts)
<i>Output</i>	
Controller program	CSV statistics

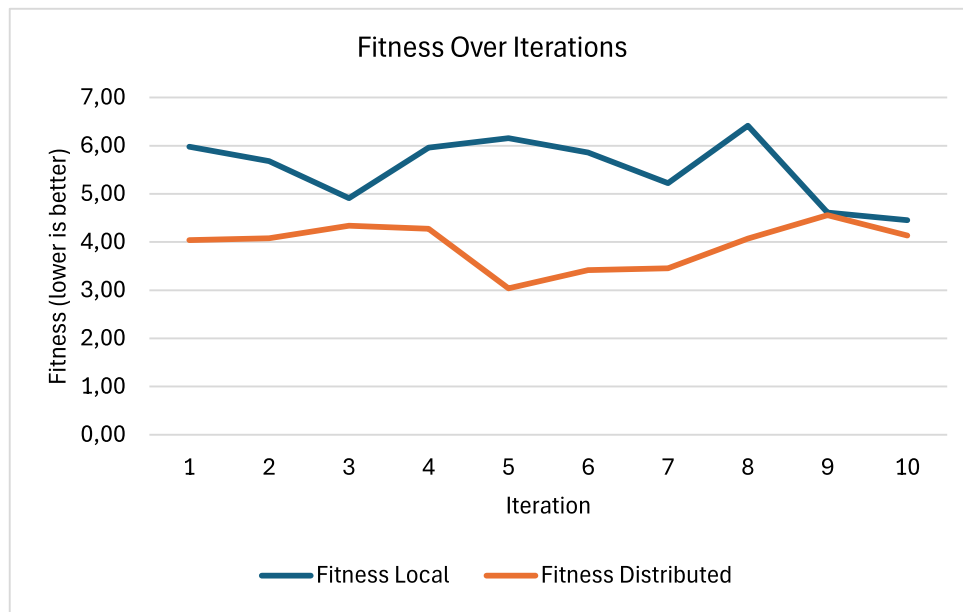


Figure 11.7: Fitness of the local variant (1 localhost worker) compared to the distributed variant (1 localhost worker + 2 remote workers).

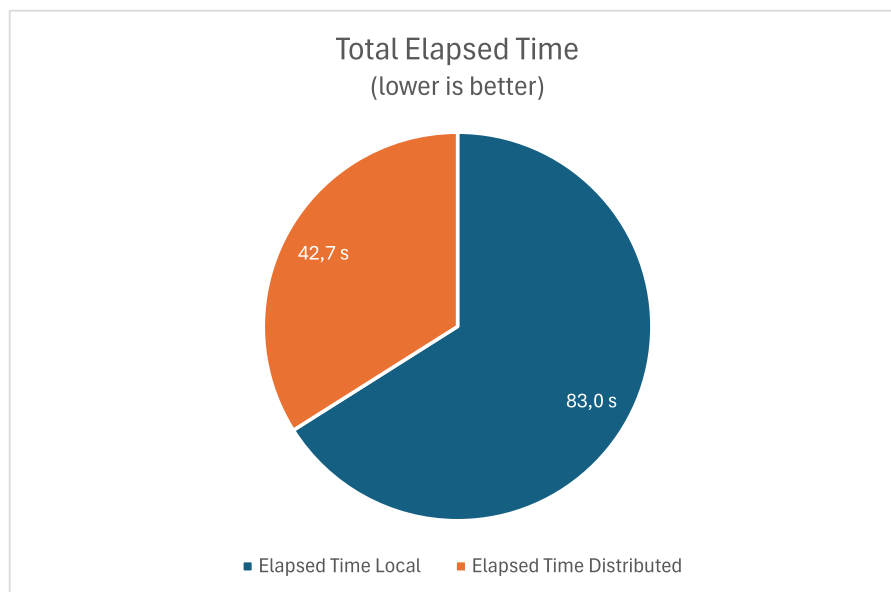


Figure 11.8: The total time workers spent performing computations over all 10 runs. When compared to fig. 11.3, elapsed time for the local variant decreased by about 15 seconds, this could've been caused by the non-determinism of the optimization algorithm and the influence of external factors such as the operating system's process scheduler. The distributed variant's time is also lower, the difference here is about 4 seconds.

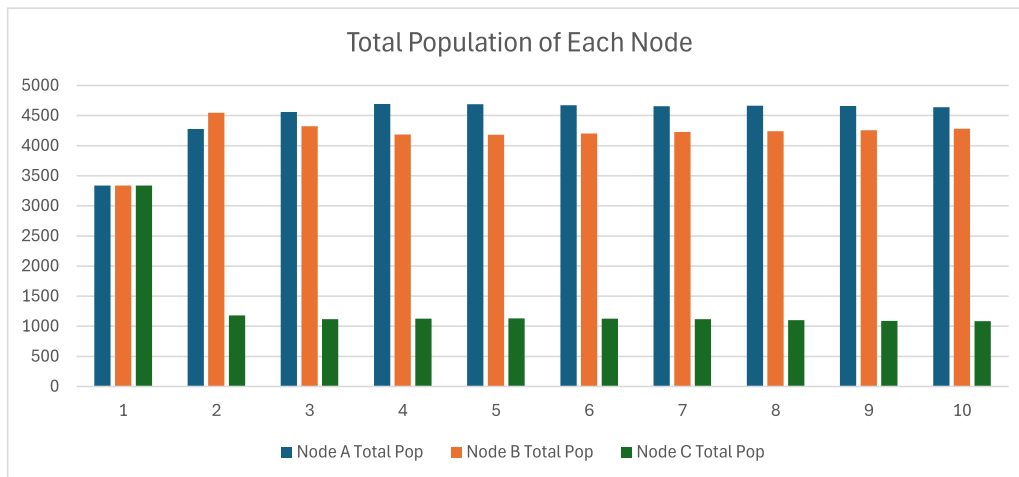


Figure 11.9: The total population assigned to each node. As was specified in table 11.3, we are splitting a “global” population of 10 000 among three workers. During the first run all workers receive the same portion because no data about their performance is known yet, the load balancing mechanism then adjusts the population in subsequent runs.

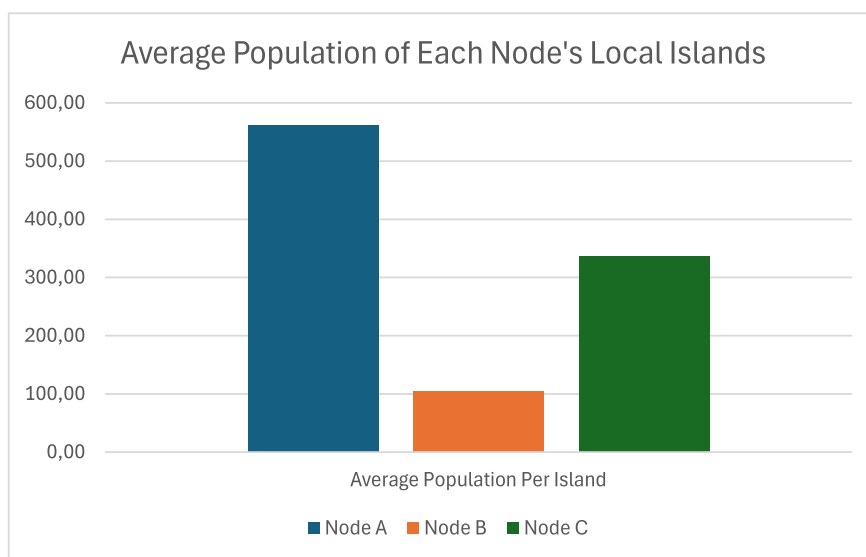


Figure 11.10: Average size of each node’s local island. **Node C** has the lowest average total population assigned to it, however it ends up with the second largest per-island population, because it has to split the input population only between 4 local islands.

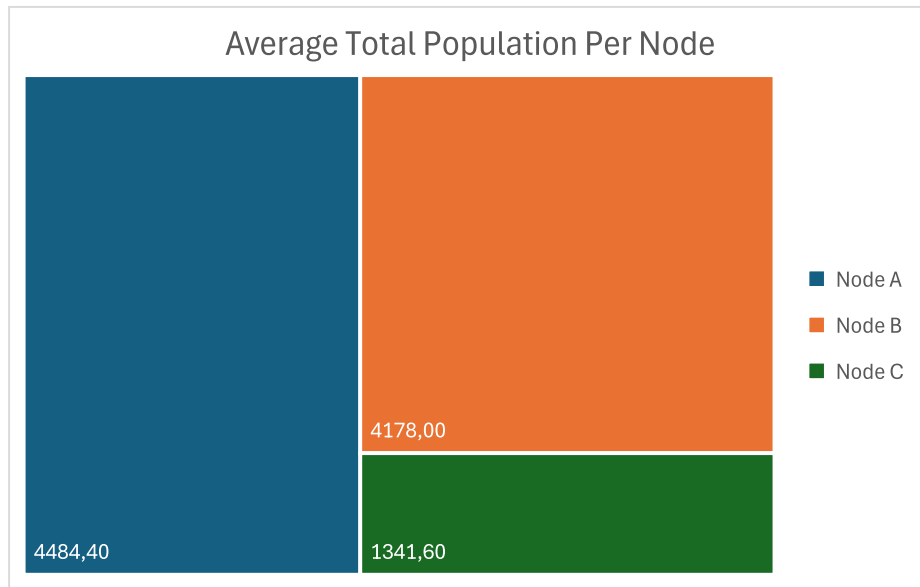


Figure 11.11: Average total population assigned to each node. Naturally **Node C** has the smallest population due to it having the least amount of logical CPU cores.

11.3 Distributed Metasolving Scenario

Sections 11.1 and 11.2 focused purely on the **distributed** and **load balancing** aspects of this solver, because only a single algorithm (Differential Evolution) was used on all worker nodes.

This section will compare how running only Differential Evolution on all three nodes compares to a scenario where each worker runs a different algorithm.

That is to say, we'll compare "standard" distributed solving against distributed metasolving.

As was described in earlier chapters of this thesis, island-based metasolving models greatly benefit from **migration** of individuals between islands. For this reason we'll be using the `distributed_solver`'s `cycleCount` parameter to perform 5 migration cycles during each run of the controller program. The controller program's source code is available in appendix B.2. With a summary of the configuration provided in table 11.4.

Our network architecture is the same as in figure 11.6, there are three workers connected to a controller on **Node A**.

To make sure that our results aren't specific to a single problem, we'll run the entire scenario two times, first on the **Rastrigin** function, and secondly on the **Rosenbrock** function.

Table 11.4: Configuration of the distributed metasolving scenario's controller program.

Parameter	Value
<i>Problem</i>	
Problem classes	Rastrigin, Rosenbrock
Dimensionality	60
<i>Algorithm</i>	
Generations	750
Migration cycles	5
<i>First Step - Normal</i>	
Algorithms	DE (all workers)
Workers	3 (1 local + 2 remote)
Total population	10 000
Initial pop. per worker	~3333 (dynamically adjusts)
<i>Second Step - Metasolver</i>	
Algorithms	DE, CMA-ES, PSO (one per worker)
Workers	3 (1 local + 2 remote)
Total population	10 000
Initial pop. per worker	~3333 (dynamically adjusts)
<i>Output</i>	
Controller program	CSV statistics

Scenario Steps

1. Compile the controller program (listing B.2) on **Node A** in **Release** mode.
2. Compile distributed-main-example.exe on all nodes in **Release** mode.
3. Run the controller on **Node A**. (EXE produced in step 1)
4. Run the distributed worker (port 5001) on **Node A**

```
.\distributed-main-example.exe --worker --address tcp://
localhost:5001 --mode "ARCHIPELAGO_BASED"
```

5. Run the distributed worker (port 5001) on **Node B**

```
.\distributed-main-example.exe --worker --address tcp://<IP-OF-
NODE-A>:5001 --mode "ARCHIPELAGO_BASED"
```

6. Run the distributed worker (port 5001) on **Node C**

```
.\distributed-main-example.exe --worker --address tcp://<IP-OF-NODE-A>:5001 --mode "ARCHIPELAGO_BASED"
```

7. Workers will now connect and perform their work.
8. Once both the “normal” distributed solver and distributed metasolver finish running, a CSV file with statistics will be saved to **Node A**’s local filesystem.
9. Run the controller program on **Node A** 9 more times, obtaining statistics of 10 consequent runs.
10. Change line 12 in listing B.2 to `pagmo::rosenbrock` and repeat all the steps above to obtain statistics for the Rosenbrock function

Scenario Results: Rastrigin

The figures 11.12, 11.13 and 11.14 show results of running the controller program from listing B.2 with problem `pagmo::rastrigin`. The controller program was executed 10 times in total.

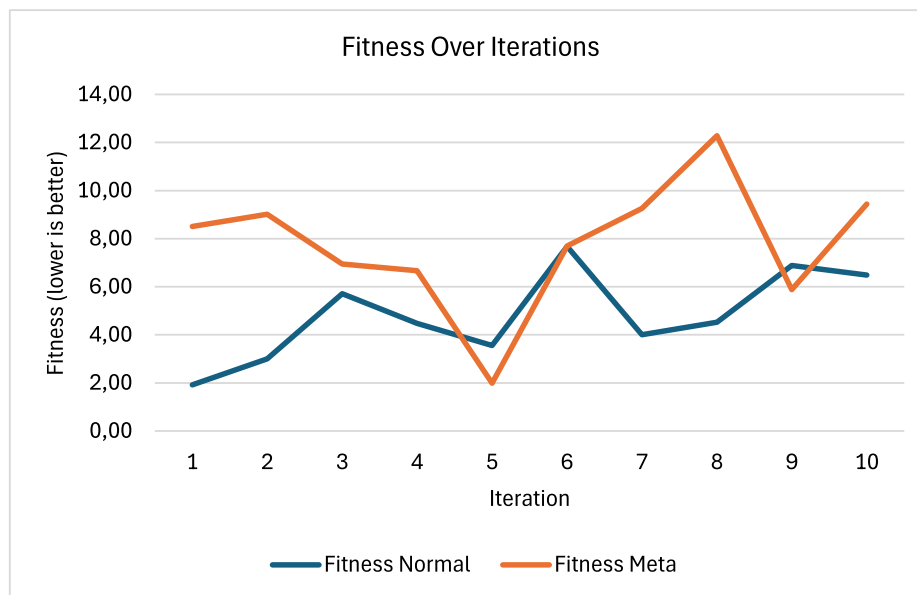


Figure 11.12: A line chart showing the difference in fitness between using a single algorithm solver (*Fitness Normal*) and metasolving (*Fitness Meta*). Both variants were using three worker nodes, as specified in table 11.4.

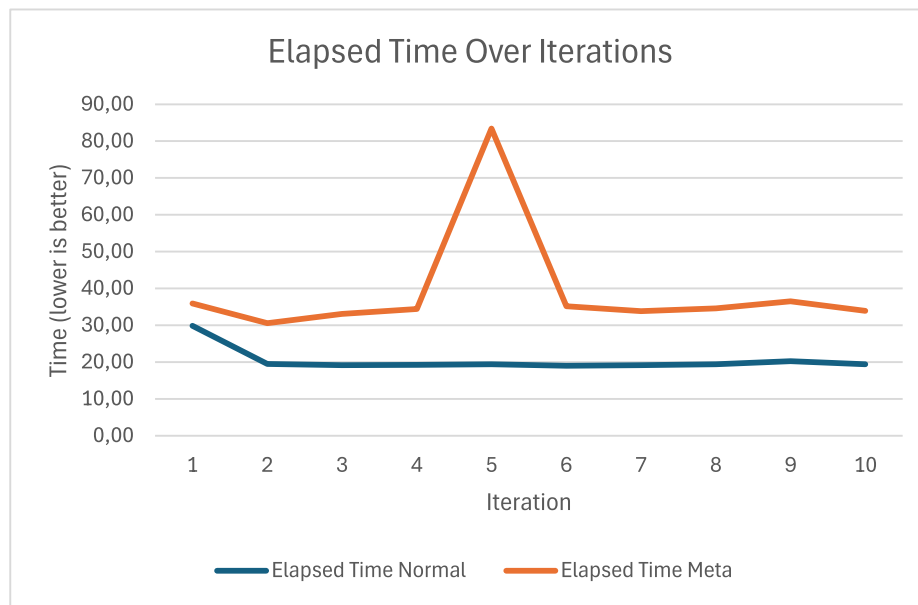


Figure 11.13: The time it took for worker nodes in each iteration to finish their work and return results to controller.

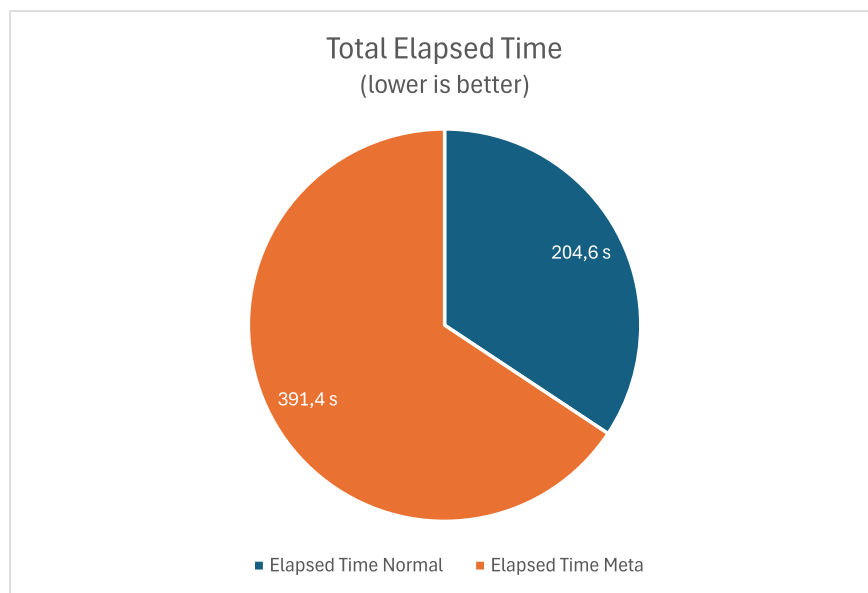


Figure 11.14: Total time worker nodes spent performing computations.

Scenario Results: Rosenbrock

The data that was collected after changing listing B.2 to run the `pagmo::rosenbrock` problem is shown in figures 11.15, 11.16 and 11.17. The controller program was executed 10 times, just like with the `pagmo::rastrigin` problem.

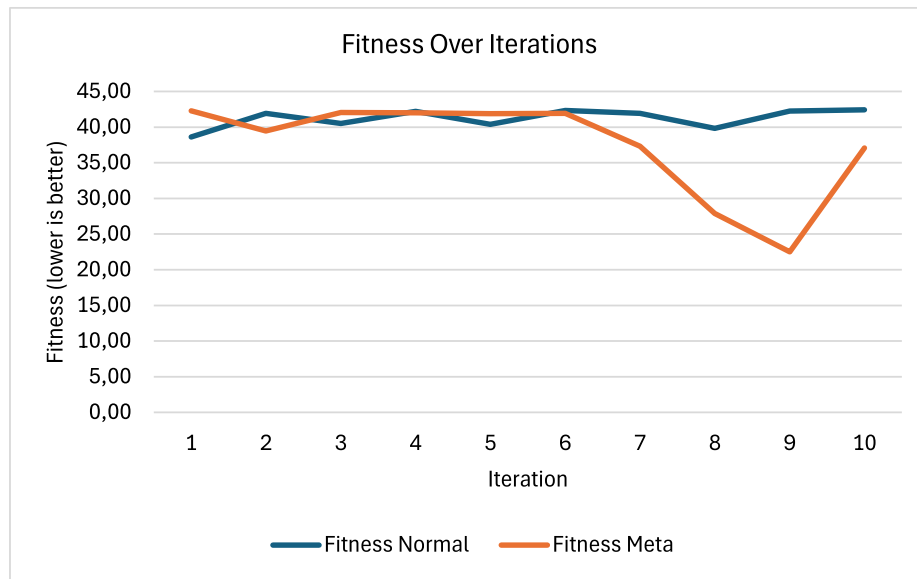


Figure 11.15: A line chart showing the difference in fitness between metasolving and a single-algorithm solver. Notice the slight improvement over figure 11.12.

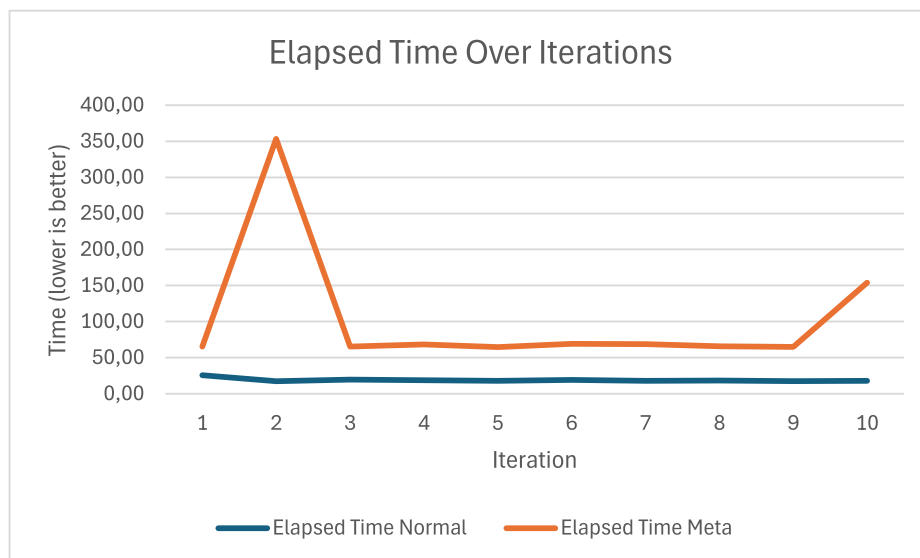


Figure 11.16: The time worker nodes spent performing computations in each iteration. Little to no improvement when compared with figure 11.13.

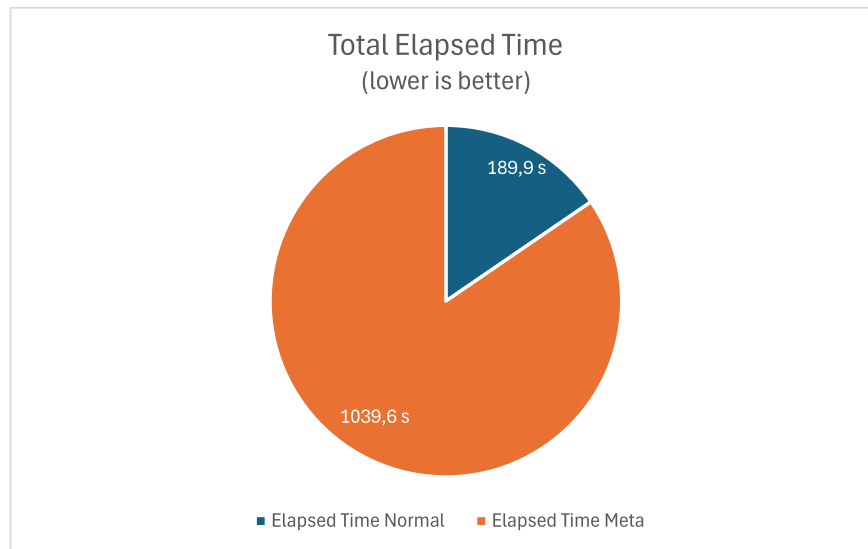


Figure 11.17: Metasolver's total elapsed time was significantly higher than in figure 11.14, due to the second iteration taking nearly 360 seconds (see fig. 11.16)

11.4 Local Metasolving Scenario

The goal of this scenario is to determine how the metasolver performs when running locally on a single node.

In section 11.3 we saw that the distributed metasolver was significantly slower than a single-algorithm distributed solver, which might've been caused by load-balancing and the different performance of each worker node.

This scenario eliminates the differences in performance by using localhost workers in the `SINGLE_THREADED` mode.

The controller program for this scenario is exactly the same as listing B.2, only difference is that in this scenario we're connecting three localhost workers.

As there are only three single-threaded workers, which aren't internally using `pagmo::archipelago` to split the input population in order to utilize every logical CPU core, the entire population of 10 000 will be split **only between three islands**.

Scenario Steps

1. Compile the controller program (listing B.2) on **Node A** and `distributed-main-example.exe` in **Release** mode.
2. Run the controller on **Node A**. (EXE produced in step 1)
3. Run the **first** local worker (port 5001) on **Node A**

```
.\distributed-main-example.exe --worker --address tcp://  
localhost:5001 --mode "SINGLE_THREADED"
```

4. Run the **second** local worker (port 5001) on **Node A**

```
.\distributed-main-example.exe --worker --address tcp://  
localhost:5001 --mode "SINGLE_THREADED"
```

5. Run **third** local worker (port 5001) on **Node A**

```
.\distributed-main-example.exe --worker --address tcp://  
localhost:5001 --mode "SINGLE_THREADED"
```

6. Workers will now connect and perform their work.
7. Once both the “normal” solver and metasolver finish running, a CSV file with statistics will be saved to **Node A**’s local filesystem.
8. Run the controller program on **Node A** 9 more times, obtaining statistics of 10 consequent runs.
9. Change line 12 in listing B.2 to `pagmo::rosenbrock` and repeat all the steps above to obtain statistics for the Rosenbrock function

Scenario Results: Rosenbrock

The results of the running controller program from listing B.2 with three connected localhost workers and the `pagmo::rosenbrock` problem are shown in figures 11.18, 11.19 and 11.20. Additionally, figures 11.21 and 11.22 were added to illustrate the poor performance of the metasolver in this scenario.

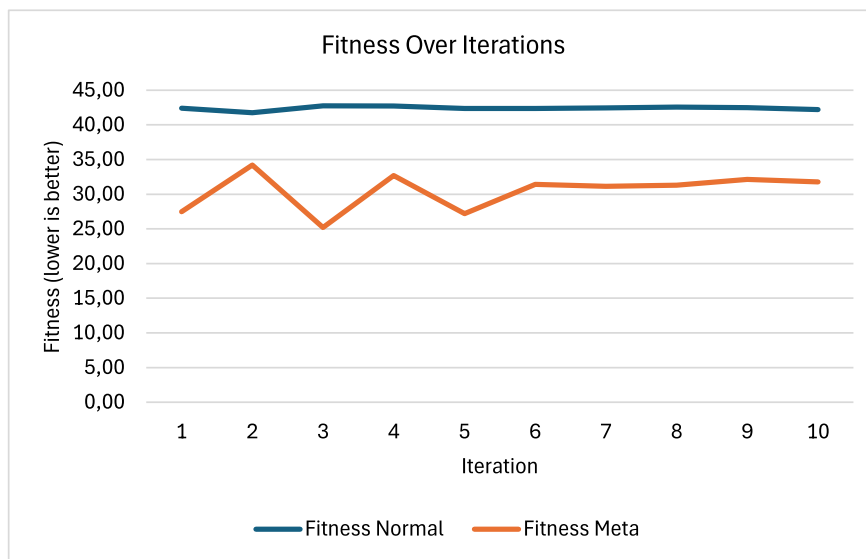


Figure 11.18: Comparison between a single-algorithm solver (*Fitness Normal*) and a metasolver (*Fitness Meta*).

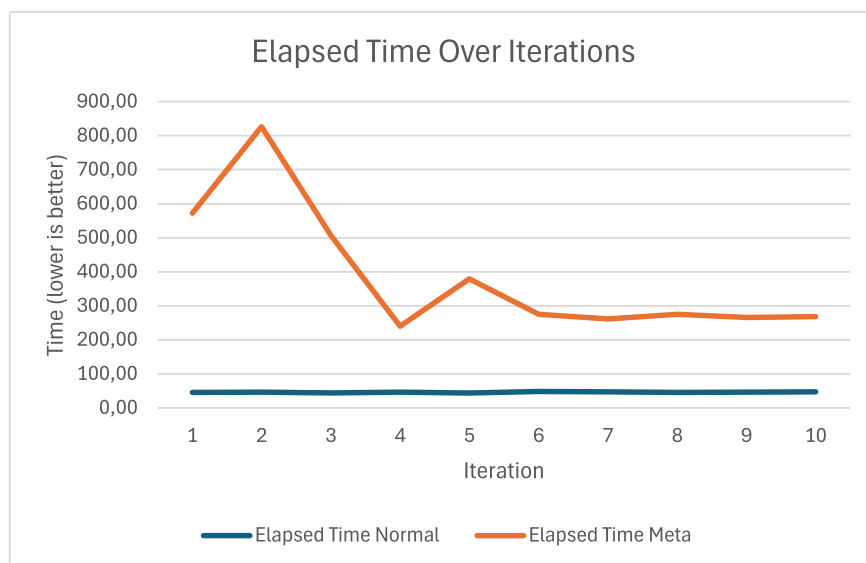


Figure 11.19: Time spent by worker nodes performing computations in each iteration. The metasolver was often slower by as much as ten times.

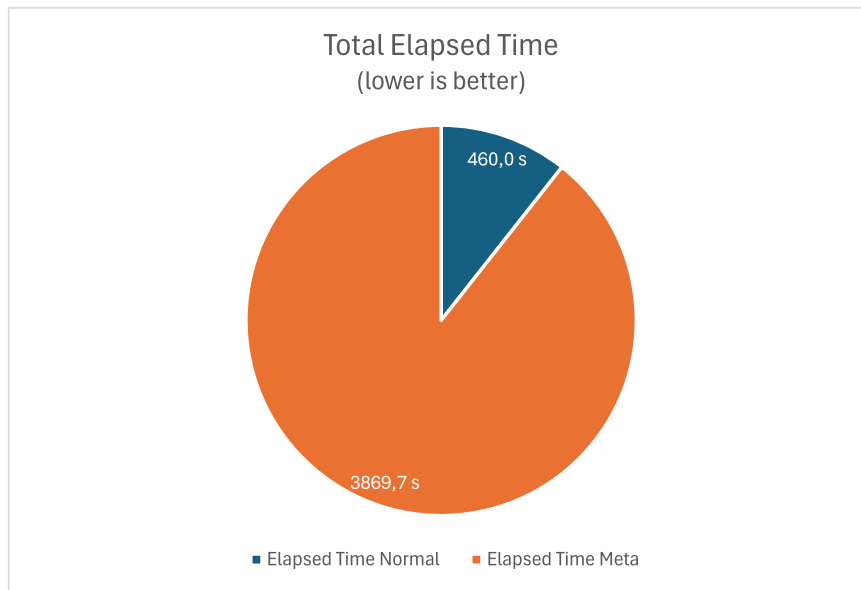


Figure 11.20: Total time spent by workers performing computations. Showing the significant advantage of the single-algorithm solver.

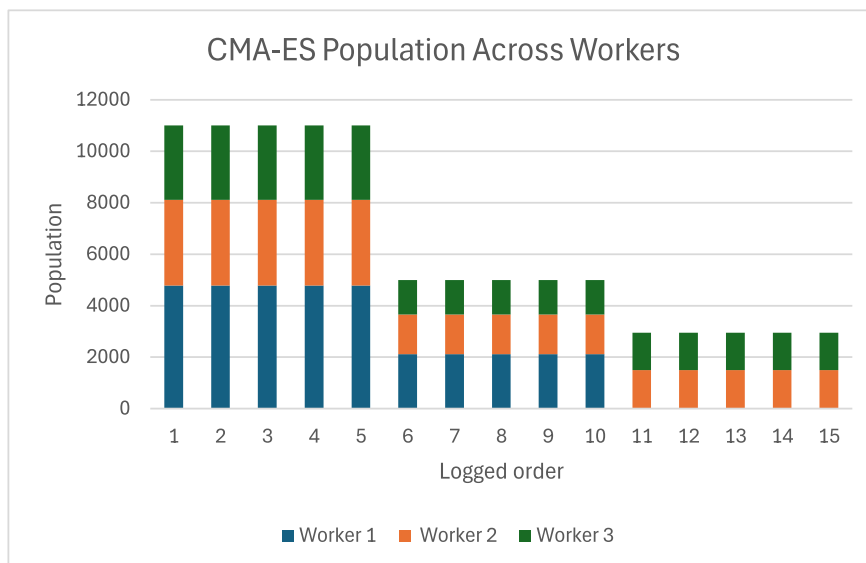


Figure 11.21: A bar chart showing extracted data from the log files of our workers, each worker logs the algorithm type and the total population which was assigned to it, this is displayed chronologically on the x axis. As we can see, the CMA-ES algorithm's population was slowly decreased by the load-balancing algorithm. This is because CMA-ES was by far the slowest, which meant that it was slowing down the entire metasolver.

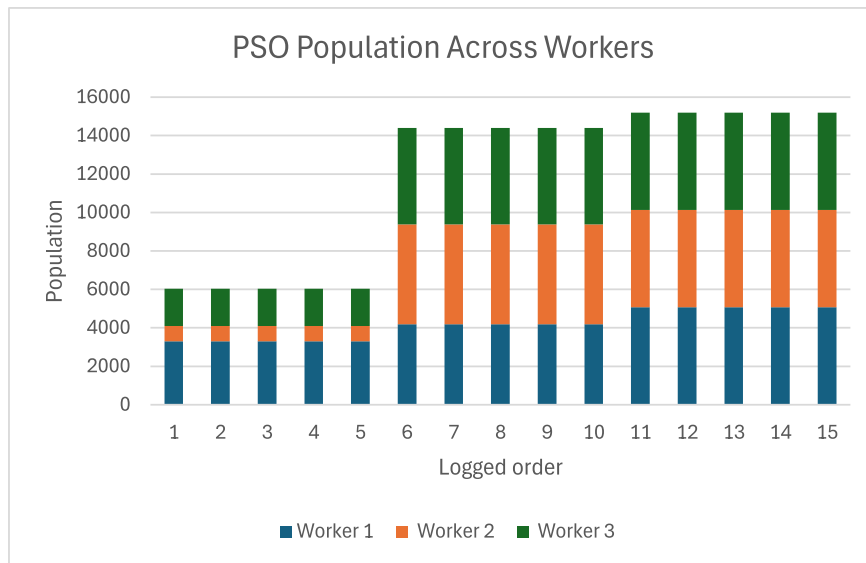


Figure 11.22: The load-balancing mechanism as a reaction to the inefficiency of CMA-ES increased population assigned to the PSO algorithm. This was likely the cause of the improvement seen around iteration 4 in figure 11.19.

Scenario Results: Rastrigin

The results of running the controller program with `pagmo::rastrigin` are shown in figures 11.23, 11.24 and 11.25. It can be seen that the metasolver performed significantly better than in the `pagmo::rosenbrock` case.

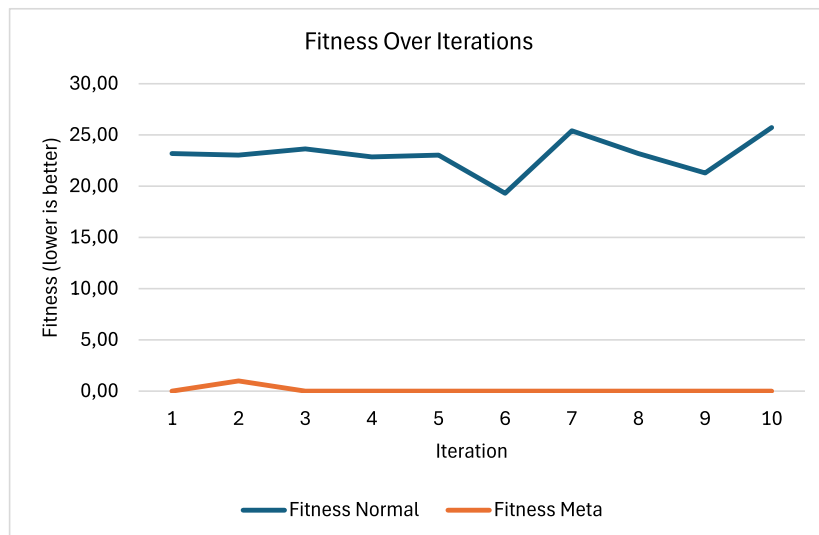


Figure 11.23: The metasolver was able to reach fitness of 0 (ideal solution) in almost every iteration, massively outperforming the single-algorithm solver.

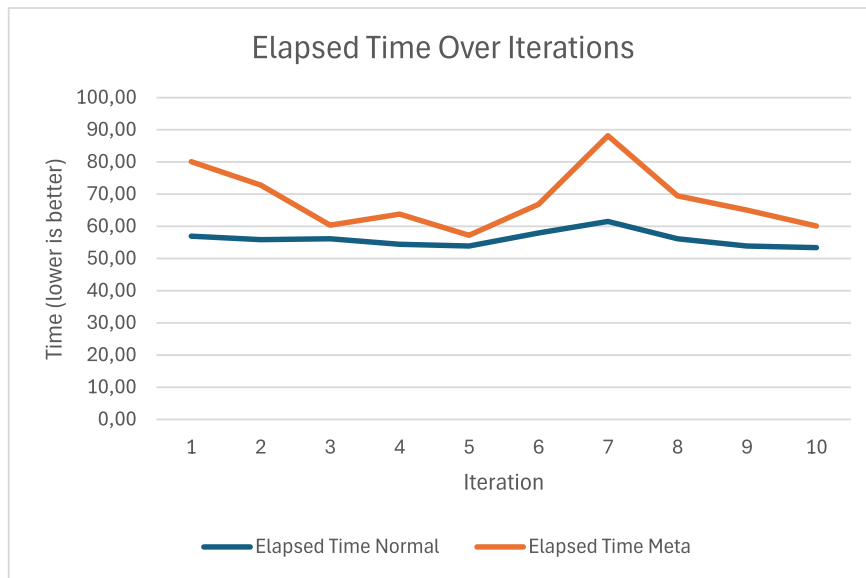


Figure 11.24: The metasolver was slightly slower, however this is still a very good result when compared to figure 11.19.

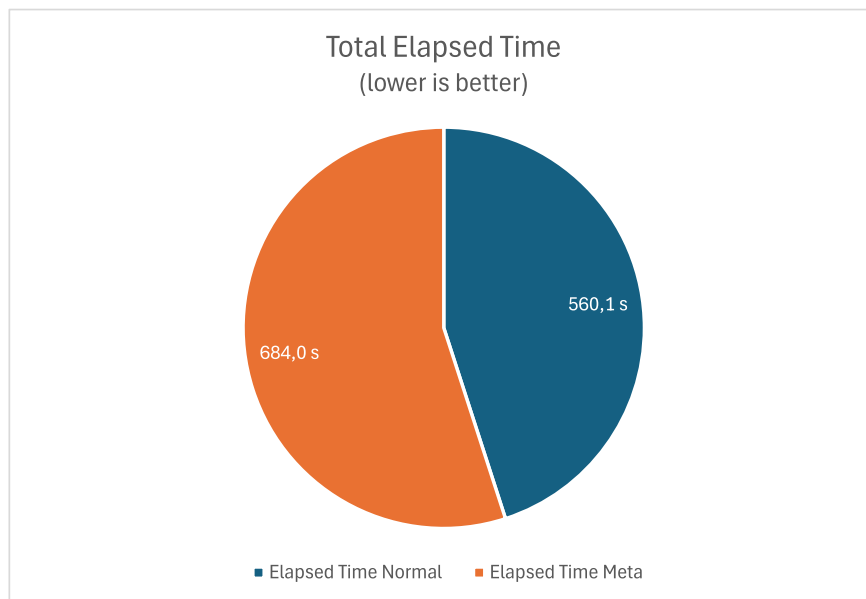


Figure 11.25: The time over all iterations is almost balanced, meaning that this is a good result for the metasolver.

11.5 Local Single-Objective Metasolving

In this scenario we'll compare how a metasolver using large amount of single-objective algorithms compares to each one of those algorithms when they're used

alone.

This scenario is included in the `src/run_example/main.cpp` file as a "single-objective benchmark", it can be executed by passing the `--benchmark-so` command line parameter to this EXE.

The detailed parameters of this scenario are specified in table 11.5. We're using 8 single-threaded localhost workers (one for each algorithm), the metasolver is then compared on 5 different problems against each of the algorithms alone.

Table 11.5: Configuration of the single-objective benchmark's controller program.

Parameter	Value
<i>Problems</i>	
Schwefel	dim. 3
Rosenbrock	dim. 40
Rastrigin	dim. 40
Ackley	dim. 40
Griewank	dim. 40
<i>Algorithms</i>	
Algorithm set	GACO, DE, SADE, DE1220, GWO, PSO, Bee Colony, SA
Generations	1000
<i>First Step – Metasolver</i>	
Workers	8 (one algorithm per worker)
Total population	3200
Migration cycles	10
<i>Second Step – Individual Algorithms</i>	
Workers	8 (same algorithm on all workers)
Runs	8 (one per algorithm)
Total population	3200
Migration cycles	10
<i>Output</i>	
Controller program	CSV statistics, HTML report

Scenario Steps

1. Compile `distributed-main-example.exe` in **Release** mode.
2. Run the benchmark controller.

```
.\distributed-main-example.exe --controller --benchmark-so
```

3. Run **8** single-threaded local workers.

```
.\distributed-main-example.exe --worker --address tcp://
localhost:5001 --mode "SINGLE_THREADED"
```

4. The single-objective benchmark will now start.
5. After the benchmark finishes, CSV and HTML files with statistics will be created in the current folder.

Scenario Results

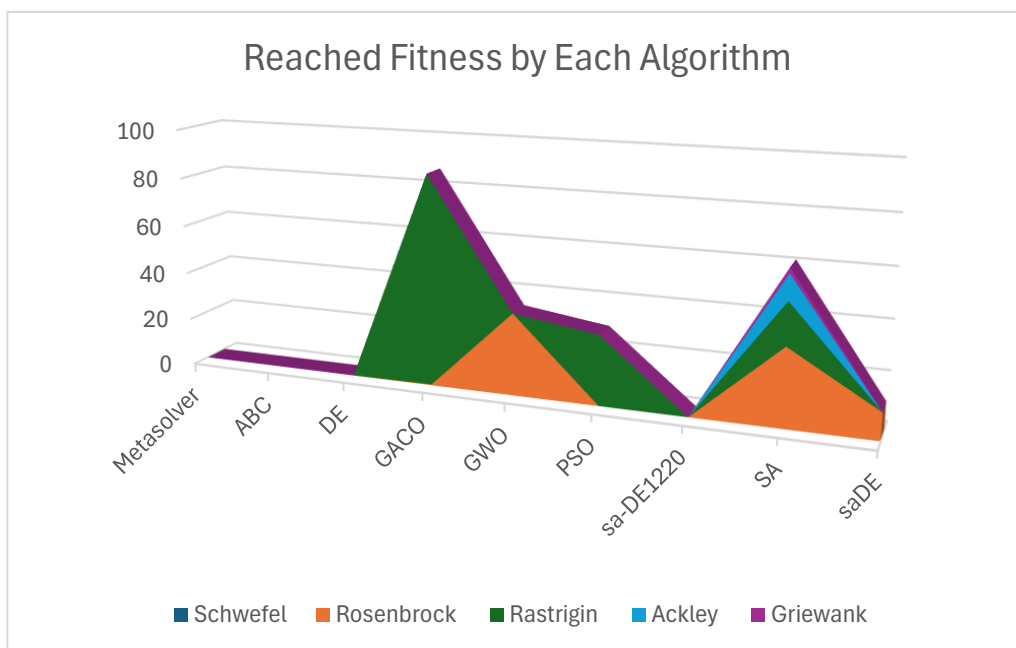


Figure 11.26: Fitness reached by each algorithm on every tested problem, lower fitness is better. Metasolver, DE, ABC and sa-DE1220 have reached fitness of 0 (perfect solution) on all five problems.

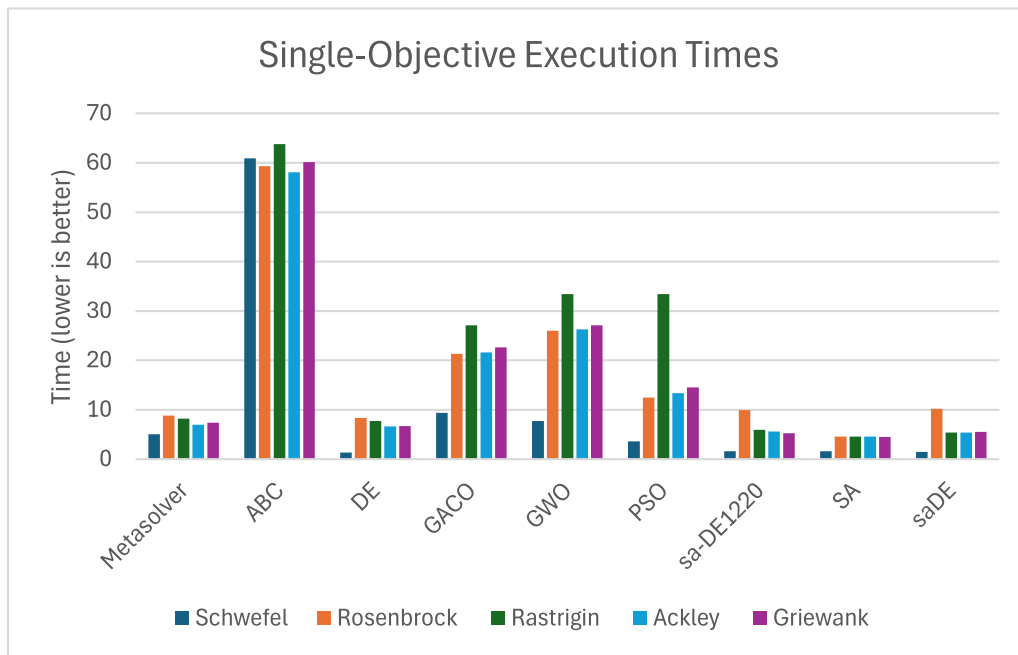


Figure 11.27: Execution time of each algorithm on every tested problem.

11.6 Local Multi-Objective Metasolving

This scenario is a multi-objective version of section 11.5, i.e. the goal is to compare a metasolver using large amount of multi-objective algorithms against each one of those algorithms individually.

This scenario is again included directly in the `src/run_example/main.cpp` file as a "multi-objective benchmark". It can be launched using the `--benchmark-mo` command line parameter.

The details of this scenario are summarized in table 11.6.

Scenario Steps

1. Compile `distributed-main-example.exe` in **Release** mode.
2. Run the benchmark controller.
3. Run 6 single-threaded local workers.

```
.\distributed-main-example.exe --controller --benchmark-mo
```

```
.\distributed-main-example.exe --worker --address tcp://
localhost:5001 --mode "SINGLE_THREADED"
```

4. The multi-objective benchmark will now start.
5. After the benchmark finishes, CSV and HTML files with statistics will be created in the current folder.

Table 11.6: Configuration of the multi-objective benchmark's controller program.

Parameter	Value
<i>Problems</i>	
ZDT1	dim. 30
ZDT2	dim. 30
ZDT3	dim. 30
ZDT4	dim. 30
<i>Algorithms</i>	
Algorithm set	NSGA2, MOEA/D, MOEA/D-GEN, MACO, NSPSO, IHS
Generations	1000
<i>First Step - Metasolver</i>	
Workers	6 (one algorithm per worker)
Total population	2400
Migration cycles	10
<i>Second Step - Individual Algorithms</i>	
Workers	6 (same algorithm on all workers)
Runs	6 (one per algorithm)
Total population	2400
Migration cycles	10
<i>Output</i>	
Controller program	CSV statistics, HTML report

Scenario Results

Figures 11.28, 11.29 and 11.30 show results of executing the metasolver and all of our algorithms on four multi-objective problems from the ZDT test suite. Because these problems are multi-objective it was necessary to provide two charts for each of their fitness functions.

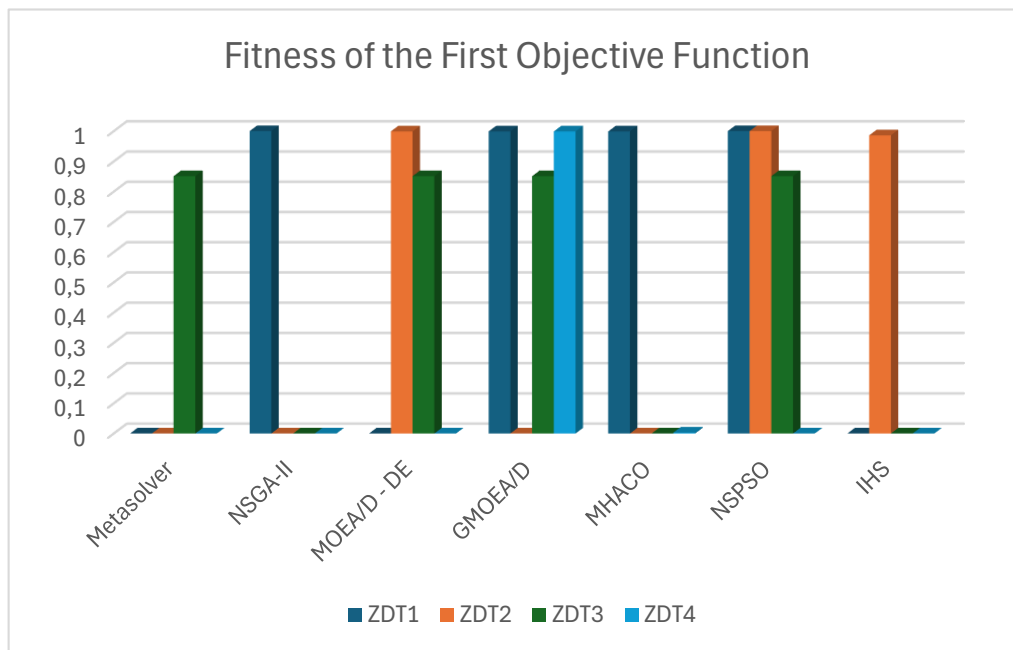


Figure 11.28: Fitness reached by each algorithm on the first objective function.

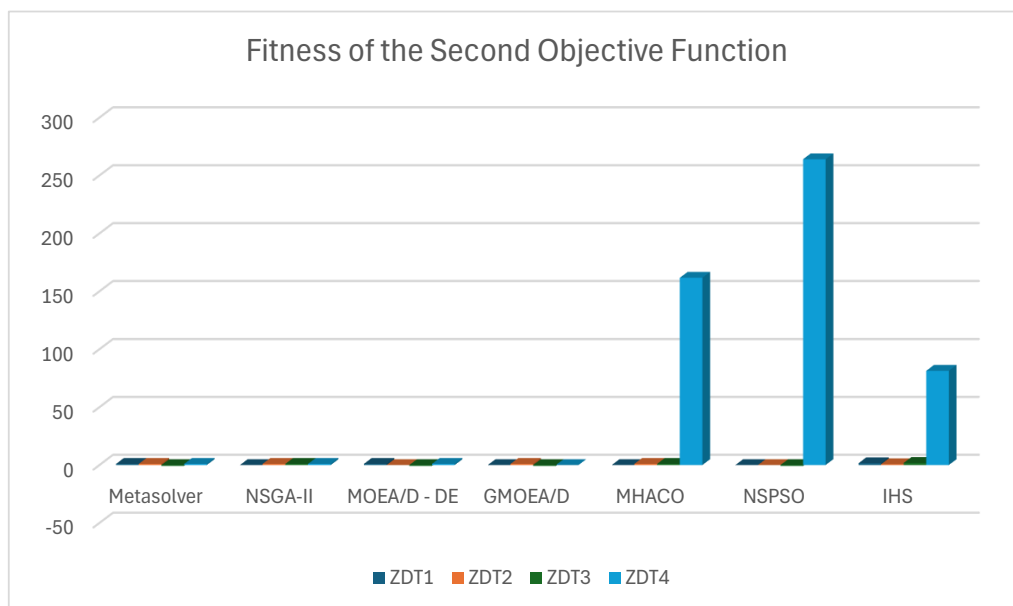


Figure 11.29: Fitness reached by each algorithm on the second objective function.

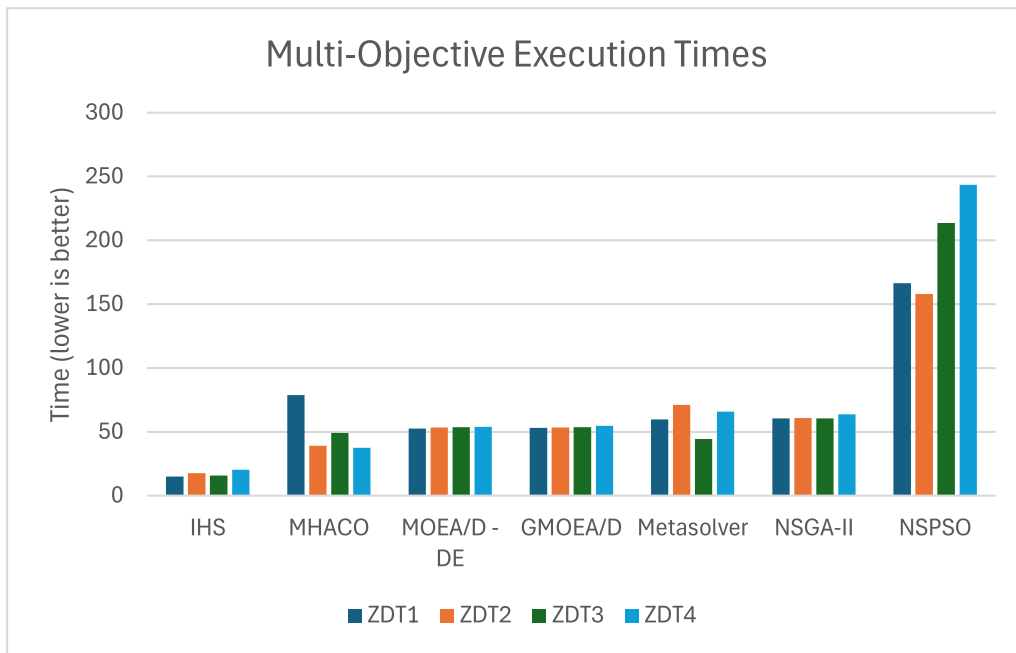


Figure 11.30: Execution time of each algorithm on every tested problem.

Before beginning with the discussion, it'd be appropriate to summarize what testing scenarios were performed in chapter 11, this will help us understand the context of what is discussed in this chapter:

1. *Simple Distributed Scenario* (sec. 11.1)

- **Purpose:** Testing if using multiple distributed nodes is faster than a local computation, seeing the effects of load-balancing on single-algorithm distributed solvers.
- **Compared data:** Comparing a single localhost worker against two workers (1x localhost, 1x remote).
- **Specifications:** All nodes using only the DE algorithm with 2000 generations, the population was set to 10 000. No migration cycles were performed.
- **Problem type:** Rastrigin Function, 30 dimensions.

2. *Three-Node Distributed Scenario* (sec. 11.2)

- **Purpose:** Same as the previous scenario, but with three nodes instead of two. Shows how adding a third low-performance node effects the overall system.
- **Compared data:** Comparing a single localhost worker against three workers (1x localhost, 2x remote).
- **Specifications and problem type** are the same as in the previous scenario.

3. *Distributed Metasolver Scenario* (sec. 11.3)

- **Purpose:** Evaluating the performance of a distributed metasolver

- **Compared data:** Comparing a distributed metasolver (Algorithms: DE, PSO, CMA-ES) against a single-algorithm distributed solver (DE), both using 3 workers (1x localhost, 2x remote).
- **Specifications:** Population of 10 000, 750 generations, 5 migration cycles.
- **Problem types:** Rastrigin and Rosenbrock, 60 dimensions.

4. *Local Metasolving Scenario* (sec. 11.4)

- **Purpose:** Evaluating the performance of a *local* metasolver, seeing how this differs from a distributed one.
- **Compared data:** Comparing a local metasolver (DE, PSO, CMA-ES) against a local single-algorithm solver (DE), both using three localhost `SINGLE_THREADED` workers.
- **Specifications** and **problem type** are the same as in the previous scenario.

5. *Local Single-Objective Metasolving* (sec. 11.5)

- **Purpose:** Evaluating the performance of a single-objective metasolver.
- **Compared data:** Comparing a local metasolver (GACO, DE, SADE, DE1220, GWO, PSO, Bee Colony, SA) against each of the algorithms on its own. 8 local `SINGLE_THREADED` workers were used for both cases.
- **Specifications:** Population of 3200 (each worker gets 400), 1000 generations, 10 migration cycles.
- **Problem types:** Schwefel (dim. 3), Rosenbrock (dim. 40), Rastrigin (dim. 40), Ackley (dim. 40), Griewank (dim. 40).

6. *Local Multi-Objective Metasolving* (sec. 11.6)

- **Purpose:** Evaluating the performance of a multi-objective metasolver.
- **Compared data:** Comparing a local metasolver (NSGA2, MOEA/D, MOEA/D-GEN, MACO, NSPSO, IHS) against each of the algorithms on its own. 6 local `SINGLE_THREADED` workers were used for both cases.
- **Specifications:** Population of 2400 (each worker gets 400), 1000 generations, 10 migration cycles.
- **Problem types:** ZDT1 to ZDT4, 30 dimensional.

Discussion of Key Findings

Results show, that using this library as a distributed single-algorithm solver works very well, significantly outperforming a local single-algorithm solver with the same configuration. The distributed part of this library can therefore be considered a **success**, the controller-worker model together with its work distribution mechanisms proved to be efficient and reliable for single-algorithm solvers.

Section 11.1 showed that splitting a single population among two worker nodes using the same algorithm can bring up to 115% faster work completion time than a local-only single-algorithm solver, while at the same time improving the average fitness by 24%.

Section 11.2 proved that the load-balancing mechanism works well enough even when three heterogeneous worker nodes of significantly different performance are used. The third “low performance” node which was added to the two nodes from section 11.1 did **not** slow down the entire distributed system, in fact, it slightly improved its total performance.

However, the situation changes somewhat when **metasolving** is introduced, using this library as a distributed metasolver often led to much worse work completion time than when a single-algorithm distributed solver was used.

This effect was first seen in section 11.3, the distributed metasolver achieved a significantly worse work completion time on both of the tested problems. The average fitness achieved in this scenario on the `pagmo::rosenbrock` problem was slightly better than the single-algorithm solver, however the poor work completion time outweighed any benefits.

Section 11.4 expanded on what was discovered in section 11.3. This section switched to using localhost single-threaded workers instead of distributed workers, which has revealed that the metasolver was significantly underperforming due to one of its algorithms (CMA-ES) slowing down the entire distributed system when solving the `pagmo::rosenbrock` problem. The load-balancing mechanism tried to counter this by reducing the amount of population assigned to workers running the CMA-ES algorithm, however this did not provide a sufficient improvement. On the other hand, the metasolver performed significantly better with this exact same configuration when solving the `pagmo::rastrigin` problem, presumably because the CMA-ES algorithm wasn't as slow in this type of a problem, which resulted in **massively better** fitness for the metasolver when compared to the single-algorithm solver at roughly the same work completion times.

Sections 11.5 and 11.6 further explored the possibilities of metasolving, this time a much larger amount of algorithms was used to form the metasolver (8 for single-objective and 6 for multi-objective), this metasolver was then compared against each of these algorithms on their own. In both of these scenarios, the metasolver

managed to reach a good fitness and work completion time comparable to the best algorithms. What could've possibly caused this was the larger amount of migration cycles, which "migrated" the good solutions to islands which were running "slow" algorithms.

So, the metasolver was able in some situations to achieve results which were close to the best performing algorithms on the specific problem, nonetheless it wasn't able to reach a significantly better fitness or work completion time either.

Strengths and Limitations of the Library

What turned out to work well was the "distributed" part of this library, the workers are able to connect to our controller via a single IP address + port combination. Reliability of the system is also exceptional, workers will automatically reconnect if the controller crashes, the controller can also handle a situation where any of its workers disconnect, by re-assigning the work to another worker.

Only disadvantage to this client-server approach is that the worker periodically tries to send TCP SYN packets to the controller when attempting to re-establish connection, this could possibly lead to useless network traffic being generated if the controller stays offline for a long time period. A viable solution to this would be to remotely shut down or start the worker program via some remote command execution protocol such as SSH, or using a different communication model than client-server.

The load-balancing capabilities wound up being a major weak point for some distributed metasolving situations. The load-balancing mechanism should be way more aggressive when penalizing slow algorithms, perhaps other things such as the generation count should be adjusted too, rather than only adjusting the population size alone.

To implement a highly performant metasolver would in the end require significant modifications to the Pagmo library, or replacing it with an entirely custom implementation. This would allow us to try different possibly beneficial approaches such as limiting an algorithm by the computational time it's allowed to use, instead of by the number of generations it's allowed to evolve.

The primary objective of this thesis was to create and implement a distributed metasolver, together with the evaluation of its performance on a set of standardized optimization problems.

First three chapters of this thesis explored the theoretical foundation upon which a distributed metasolver operates, starting from the most fundamental theory such as the definition of an optimization problem or the description of various population-based optimization algorithms, to more high-level concepts such as the island model or different approaches related to distributed computing.

This knowledge was later utilized to build a theoretical design of the distributed metasolver, along with investigating potentially useful software libraries for the implementation stage.

Finally, based on these requirements a C++ software library was implemented. The most important parts of this library are the **Worker** and **Controller** classes, representing the two different node types of our distributed system. The controller is able to provide workers with a population and an optimization problem distributed as a DLL (dynamic library), the workers then use an optimization algorithm to evolve this population, returning their results back to the controller.

Furthermore, an **SCGMS** wrapper library around the distributed metasolver was created, showcasing that it can be used as an application solver embedded inside of existing software frameworks.

Based on the results of various evaluation scenarios, it was discovered that the implemented library achieves impressive results when used as a single-algorithm distributed solver, providing superior fitness and work completion time when compared to the non-distributed variant.

On the other hand, the library underperformed in several scenarios when used as a distributed metasolver, due to the difficulty of load-balancing between algorithms of different computational complexity.

Future work could include for example modifying the Pagmo library to improve its metasolving capabilities, or implementing a more robust load-balancing system.

Build Instructions

A

To successfully build this library, the following dependencies need to be manually installed:

- Boost [57]
- Intel TBB [58]

Installing dependencies and compiling the library is significantly easier on Linux-based systems than on Windows, we'll therefore start with the Linux build instructions first.

A.1 GNU Linux

The following steps were tested on a clean installation of **Ubuntu 26.04 LTS**, these steps may or may not differ depending on the Linux distribution of your choice.

1. Install Intel TBB.

```
sudo apt install libtbb-dev
```

2. Install the Boost libraries.

```
sudo apt install libboost-all-dev
```

3. Install the build tools (if they're not already installed).

```
sudo apt install cmake ninja-build g++ git
```

4. **Important:** the versions of the packages you installed should ideally be greater than or equal to the versions specified below. The compilation or CMake configuration might fail if any of these tools are outdated.

```
cmake --version  
> cmake version 4.0.2  
g++ --version
```

```
> 15.2.0
cat /usr/include/boost/version.hpp | grep "BOOST_LIB_VERSION"
> BOOST_LIB_VERSION "1_90"
```

5. Clone the Distributed Metasolver repository (or use the source files provided as an attachment to this thesis).

```
git clone https://github.com/jonas-s-s-s/Pagmo-solver-
distributed
```

6. Navigate to the root of this project.

```
cd ./Pagmo-solver-distributed
```

7. Initialize git submodules.

```
git submodule update --init --recursive
```

8. Configure and build using the provided CMakePresets.json file.

```
cmake --preset dist_solver-Debug
cmake --build --preset "dist_solver-build: Debug"
```

9. Navigate to the compiled binaries directory.

```
cd ./build_Debug/bin/
```

10. Execute the unit tests binary, all tests should pass.

```
./unit_tests_distributed_solver
=====
All tests passed (113 assertions in 34 test cases)
```

11. Start a controller.

```
./distributed-main-example --controller --cache-dir .
[2026-05-04 12:43:27.618] [info] Waiting until 2 workers
connect
```

12. Start two workers in separate terminal windows.

```
./distributed-main-example --worker --mode "SINGLE_THREADED"
```

13. The workers will connect to the controller over localhost, they'll then run optimization on the default example scenario of `schwefel_udp`. The controller's output should look similar to what is shown below.

```
[2026-05-04 12:48:41.204] [debug] Best individual: [420.968755,
420.968745, ]
```


A.2 MS Windows

Microsoft Visual Studio and the MSVC compiler are the recommended tools for building this library on Windows. If you're using any other IDE or toolchain, it's your own responsibility to adjust the steps below accordingly.

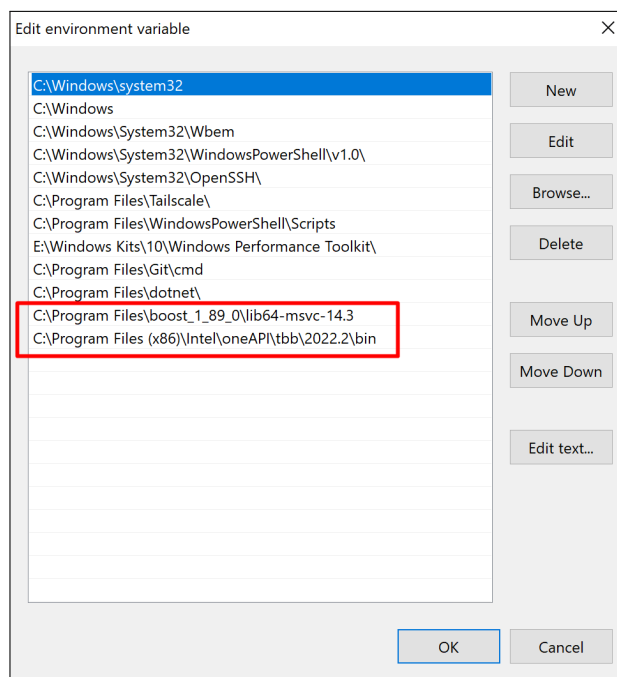
1. Download and install Boost from the provided URL, successful compilation of this library was confirmed with Boost 1.89.0.

<https://www.boost.org/releases/latest/>

2. Download and install Intel TBB.

<https://www.intel.com/content/www/us/en/developer/tools/oneapi/onetbb-download.html>

3. Make sure to include the Boost and Intel TBB binary directories in your system's **path** environment variable, if this isn't done, the compiled application might not start due to missing DLL files. Examples of these paths are shown below.



4. Clone the Distributed Metasolver repository (or use the source files provided as an attachment to this thesis).

```
git clone https://github.com/jonas-s-s-s/Pagmo-solver-distributed
```

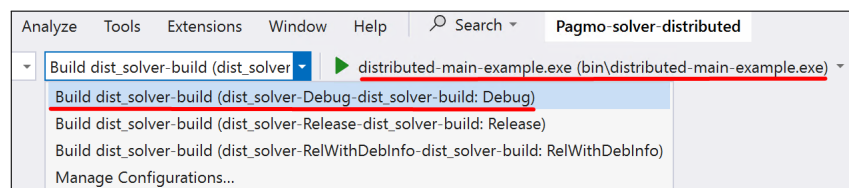
5. Navigate to the root of this project.

```
cd ./Pagmo-solver-distributed
```

6. Initialize git submodules.

```
git submodule update --init --recursive
```

7. Open Microsoft Visual Studio, click on "Open a local folder", choose the root folder of this library.
8. Look at the main toolbar, choose the CMake Debug preset and distributed-main-example.exe.



9. Open the CMakePresets.json file, change every occurrence of "BOOST_PATH" to match your Boost install location. CMake might fail to find Boost if this isn't done.

```
"environment": {
  "BOOST_PATH": "C:/Program Files/boost_1_89_0/lib64-msvc-
14.3/cmake/Boost-1.89.0"
}
```

10. Start the build process by clicking on:
Build → Build distributed-main-example.exe
11. Next build the schwefel_udp.dll, so we can use it to run the example scenario in main.
12. Open a terminal window in the ./build_Debug/bin/ directory
13. Use the EXE to start a controller.

```
./distributed-main-example.exe --controller --cache-dir .
[2026-05-04 12:43:27.618] [info] Waiting until 2 workers
connect
```

14. Start two workers in separate terminal windows.

```
./distributed-main-example.exe --worker --mode "SINGLE_THREADED
"
```

15. The workers will connect to the controller over localhost, they'll then run optimization on the default example scenario of `schwefel_udp`. The controller's output should look similar to what is shown below.

```
[2026-05-04 12:48:41.204] [debug] Best individual: [420.968755,  
420.968745, ]
```

16. Use `./distributed-main-example.exe --help` to see a list of all available command line parameters.

Controller Programs

B

The following sections contain controller programs used for collection of data regarding different solver configurations in chapter 11.

B.1 Simple Distributed Scenario

Source code B.1: Controller program used for the simple distributed scenario.

```
1 #include "distributed_solver.h"
2 #include "distributed_worker.h"
3 #include "pagmo/algorithms/de.hpp"
4 #include <string>
5 #include "benchmark_stats.h"
6 #include "so_benchmark.h"
7
8 int main()
9 {
10     pagmo::problem problem{pagmo::rastrigin{30}};
11     benchmark_stats bench{problem};
12     pagmo::algorithm algo{pagmo::de(2000)};
13     // 1) Running evolution on 1 local worker (LOCAL ONLY RUN)
14     {
15         distributed_solver solver{
16             "tcp://0.0.0.0:5000",
17             1,
18         };
19         solver.enable_logging();
20         solver.wait_until_workers_connect();
21         bench.start_timer();
22         // Total population size of 10 000
23         solver.evolve(problem, {algo}, 10000);
24         solver.wait_until_completion();
25         bench.add_data_point(
26             solver.get_best_individual(),
27             "LOCAL_" + algo.get_name(),
28             bench.stop_timer()
```

```
29     );
30 }
31 // 2) Running evolution on 1 local worker and 1 remote worker (DISTRIBUTED RUN)
32 {
33     distributed_solver solver{
34         "tcp://0.0.0.0:5001",
35         2,
36     };
37     solver.enable_logging();
38     solver.wait_until_workers_connect();
39     bench.start_timer();
40     // Total population size of 10 000, split between 2 workers
41     solver.evolve(problem, {algo}, 10000);
42     solver.wait_until_completion();
43     bench.add_data_point(
44         solver.get_best_individual(),
45         "DISTRIBUTED_" + algo.get_name(),
46         bench.stop_timer()
47     );
48 }
49 // 3) Saving collected statistics
50 bench.end_current_measurement();
51 bench.save_all_results_as_csv();
52 return 0;
53 }
```

B.2 Distributed Metasolving Scenario

Source code B.2: Controller program used for the distributed metasolving scenario.

```
1 #include "distributed_solver.h"
2 #include "distributed_worker.h"
3 #include "pagmo/algorithms/de.hpp"
4 #include <string>
5 #include "benchmark_stats.h"
6 #include "so_benchmark.h"
7 #include "pagmo/algorithms/cmaes.hpp"
8
9 int main()
10 {
11     // Change to pagmo::rosenbrock here for the second phase
12     pagmo::problem problem{pagmo::rastrigin{60}};
13     benchmark_stats bench{problem};
14     int gens = 750;
15     pagmo::algorithm algo{pagmo::de(gens)};
```

```
16 // 1) Running DE on 3 worker nodes
17 {
18     distributed_solver solver{
19         "tcp://0.0.0.0:5001",
20         3,
21     };
22     solver.enable_logging();
23     solver.wait_until_workers_connect();
24     bench.start_timer();
25     // Total population size of 10 000, split between 3 workers, 5 migration cycles
26     solver.evolve(problem, {algo}, 10000, 5);
27     solver.wait_until_completion();
28     bench.add_data_point(
29         solver.get_best_individual(),
30         "NORMAL_" + algo.get_name(),
31         bench.stop_timer()
32     );
33 }
34 // 2) Running {DE, CMA-ES, PSO} on 3 worker nodes
35 pagmo::algorithm de{pagmo::de(gens)};
36 pagmo::algorithm cmaes{pagmo::cmaes(gens)};
37 pagmo::algorithm pso{pagmo::pso(gens)};
38 auto algs = std::vector{de, cmaes, pso};
39 {
40     distributed_solver solver{
41         "tcp://0.0.0.0:5001",
42         3
43     };
44     solver.enable_logging();
45     solver.wait_until_workers_connect();
46     bench.start_timer();
47     // Total population size of 10 000, split between 3 workers, 5 migration cycles
48     solver.evolve(problem, {algs}, 10000, 5);
49     solver.wait_until_completion();
50     bench.add_data_point(
51         solver.get_best_individual(),
52         "METASOLVER",
53         bench.stop_timer()
54     );
55 }
56 // 3) Saving collected statistics
57 bench.end_current_measurement();
58 bench.save_all_results_as_csv();
59 return 0;
60 }
```

Bibliography

1. COCKE, John; SLOTNICK, Daniel. *The Use of Parallelism in Numerical Calculations*. Yorktown Heights, NY, 1958. Research Memorandum, RC-55. IBM Research. Available also from: <https://www.computerhistory.org/collections/catalog/102635119/>.
2. ADEFEMI, Temitayo. *What Every Computer Scientist Needs To Know About Parallelization*. 2025. Available from arXiv: 2504.03647 [cs.DC].
3. KOUTNY, Tomas; UBL, Martin. SmartCGMS as a Testbed for a Blood-Glucose Level Prediction and/or Control Challenge with (an FDA-Accepted) Diabetic Patient Simulation. *Procedia Computer Science*. 2020, vol. 177, pp. 354–362. ISSN 1877-0509. Available from DOI: <https://doi.org/10.1016/j.procs.2020.10.048>. The 11th International Conference on Emerging Ubiquitous Systems and Pervasive Networks (EUSPN 2020) / The 10th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare (ICTH 2020) / Affiliated Workshops.
4. NOCEDAL, Jorge; WRIGHT, Stephen J. *Numerical Optimization*. 2nd ed. New York: Springer, 2006. Springer Series in Operations Research and Financial Engineering. ISBN 978-0387303031.
5. LYNCH, Peter. 100 Years of L. F. Richardson's Weather Prediction by Numerical Process. *Monthly Weather Review*. 2022, vol. 150, no. 4, pp. 693–703. Available from DOI: <https://doi.org/10.1175/MWR-D-22-0068.1>.
6. EDUCATIVE, INC. *What is Gradient Descent?* [<https://www.educative.io/answers/what-is-gradient-descent>]. [N.d.]. Accessed: 2026-04-20.
7. MIRJALILI, Seyedali; MIRJALILI, Seyed Mohammad; LEWIS, Andrew. Grey Wolf Optimizer. *Advances in Engineering Software*. 2014, vol. 69, pp. 46–61. ISSN 0965-9978. Available from DOI: <https://doi.org/10.1016/j.advengsoft.2013.12.007>.

8. DORIGO, M.; GAMBARDILLA, L.M. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*. 1997, vol. 1, no. 1, pp. 53–66. Available from DOI: 10.1109/4235.585892.
9. KARABOGA, Dervis; BASTURK, Bahriye. A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *Journal of Global Optimization*. 2007, vol. 39, no. 3, pp. 459–471. Available from DOI: 10.1007/s10898-007-9149-x.
10. EIBEN, Agoston E.; SMITH, James E. *Introduction to Evolutionary Computing*. 2nd. Springer Publishing Company, Incorporated, 2015. Natural Computing Series. ISBN 978-3-662-44873-1. Available from DOI: 10.1007/978-3-662-44874-8.
11. RAJWAR, Kanchan; DEEP, Kusum; DAS, Swagatam. An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges. *Artificial Intelligence Review*. 2023, vol. 56, no. 11, pp. 13187–13257. Available from DOI: 10.1007/s10462-023-10470-y.
12. LUKE, Sean. *Essentials of Metaheuristics*. Lulu, 2009. Available for free at <http://cs.gmu.edu/~sean/book/metaheuristics/>.
13. TALBI, El-Ghazali. *Metaheuristics: From Design to Implementation*. Hoboken, NJ: John Wiley & Sons, 2009. ISBN 978-0-470-27858-1. Available from DOI: 10.1002/9780470496916.
14. HOLLAND, John H. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. Ann Arbor, MI: University of Michigan Press, 1975.
15. SCHWEFEL, Hans-Paul. *Evolution and Optimum Seeking: The Sixth Generation*. New York: Wiley, 1995. ISBN 0-471-57148-2. Available also from: https://www.researchgate.net/publication/234811153_Evolution_and_Optimum_Seeking_The_Sixth_Generation.
16. HANSEN, Nikolaus; OSTERMEIER, Andreas. Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation. In: *Proceedings of IEEE International Conference on Evolutionary Computation*. IEEE, 1996, pp. 312–317.
17. STORN, Rainer; PRICE, Kenneth. *Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces*. 1995. Tech. rep., TR-95-012. International Computer Science Institute.

18. STORN, Rainer; PRICE, Kenneth. Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. *Journal of Global Optimization*. 1997, vol. 11, no. 4, pp. 341–359. Available from doi: 10.1023/A:1008202821328.
19. KOZA, John R. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA: MIT Press, 1992. ISBN 978-0262111706.
20. AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2nd. Boston, MA, USA: Addison-Wesley, 2006. ISBN 978-0321486813.
21. DEDERA, Lubomír. Domain-Specific Languages for Command and Control Systems. *Science & Military*. 2010, vol. 5, no. 1, pp. 40–45. Available also from: https://www.researchgate.net/publication/265218184_DOMAIN-SPECIFIC_LANGUAGES_FOR_COMMAND_AND_CONTROL_SYSTEMS.
22. INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *Programming languages — C*. Second. Geneva, Switzerland: International Organization for Standardization, 1999. Available also from: https://www.dii.uchile.cl/~daespino/files/Iso_C_1999_definition.pdf. ISO/IEC 9899:1999.
23. KENNEDY, James; EBERHART, Russell. Particle swarm optimization. In: *Proceedings of ICNN'95 - International Conference on Neural Networks*. IEEE, 1995, vol. 4, pp. 1942–1948. Available from doi: 10.1109/ICNN.1995.488968.
24. DORIGO, M.; MANIEZZO, V.; COLORNI, A. Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*. 1996, vol. 26, no. 1, pp. 29–41. Available from doi: 10.1109/3477.484436.
25. DEB, Kalyanmoy; PRATAP, Amrit; AGARWAL, Sameer; MEYARIVAN, Tamizhi. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*. 2002, vol. 6, no. 2, pp. 182–197. Available from doi: 10.1109/4235.996017.
26. MENG, Qinxue; WU, Jia; ELLIS, John; KENNEDY, Paul. Dynamic Island Model based on Spectral Clustering in Genetic Algorithm. In: 2017, pp. 1724–1731. Available from doi: 10.1109/IJCNN.2017.7966059.
27. PENG, Fei; TANG, Ke; CHEN, Guoliang; YAO, Xin. Population-Based Algorithm Portfolios for Numerical Optimization. *IEEE Transactions on Evolutionary Computation*. 2010, vol. 14, no. 5, pp. 782–805. Available from doi: 10.1109/TEVC.2010.2040183.

28. SILVEIRA, Lucas A. da; LIMA, Thaynara A. de; AYALA-RINCÓN, Mauricio. Reconfigurable Heterogeneous Parallel Island Models. *arXiv preprint arXiv:2205.02916*. 2022. Available also from: <https://arxiv.org/abs/2205.02916>.
29. MANCHEK, Bob. *The History of PVM* [<https://www.netlib.org/pvm3/faq.html/node4.html#SECTION00022000000000000000>]. 1995. Accessed: 2026-04-26.
30. INTEL CORPORATION. *Microprocessor Quick Reference Guide* [<https://www.intel.com/pressroom/kits/quickreffam.htm>]. 2008. Historical Reference. Accessed: 2026-04-26.
31. GEIST, Al et al. *PVM: Parallel Virtual Machine: A Users' Guide and Tutorial for Networked Parallel Computing*. Cambridge, MA, USA: MIT Press, 1994. ISBN 978-0262571081. Available also from: <https://cecs.wright.edu/people/faculty/schung/ceg820/pvm-book.pdf>.
32. WANG, Paul. Tools for parallel/distributed mathematical computation. In: 1997, pp. 188–195. Available from doi: 10.1145/266670.266727.
33. TENNENHOUSE, David L.; SMITH, Jonathan M.; SINCOSKIE, W. David; WETHERALL, David J.; MINDEN, Gary J. A survey of active network research. *IEEE Communications Magazine*. 1997, vol. 35, no. 1, pp. 80–86.
34. KOUTNY, Tomas. Smart Active Node: Security and Efficiency of Programmable Networking. In: *Proceedings of the 5th European Conference on Computer Systems (EuroSys '10)*. Paris, France: ACM, 2010.
35. TENNENHOUSE, David L.; WETHERALL, David J. Towards an active network architecture. *ACM SIGCOMM Computer Communication Review*. 1996, vol. 26, no. 2, pp. 5–17. Available from doi: 10.1145/231699.231701.
36. WETHERALL, David J.; GUTTAG, John V.; TENNENHOUSE, David L. ANTS: A toolkit for building and dynamically deploying network protocols. In: *Proceedings IEEE OPENARCH'98*. IEEE, 1998, pp. 117–129.
37. AUBRECHT, Vladimir; KOUTNY, Tomas. Revisiting the DARPA's Idea of a Programmable Network. In: *Military Communications and Information Systems Conference*. 2012.
38. RAI, Anurag; SINGH, Rahul; MODIANO, Eytan. *A Distributed Algorithm for Throughput Optimal Routing in Overlay Networks*. 2016. Available from arXiv: 1612.05537 [cs.NI].

39. IEEE/Open Group Standard for Information Technology–Portable Operating System Interface (POSIX™) Base Specifications, Issue 8. *IEEE/Open Group Std 1003.1-2024 (Revision of IEEE Std 1003.1-2017)*. 2024, pp. 1–4107. Available from DOI: 10.1109/IEEESTD.2024.10555529.
40. LINUX MAN-PAGES PROJECT. *epoll(7) — I/O event notification facility* [online]. 2023. [visited on 2026-04-28]. Available from: <https://man7.org/linux/man-pages/man7/epoll.7.html>. Section 7.
41. KOHLHOFF, Christopher M. *Asio C++ Library* [<https://think-async.com/Asio/>]. 2025. Accessed: 2026-04-29.
42. KOHLHOFF, Christopher M. *Boost.Asio Rationale* [https://www.boost.org/doc/libs/latest/doc/html/boost_asio/overview/rationale.html]. Boost C++ Libraries, 2026. Accessed: 2026-04-29.
43. KOHLHOFF, Christopher M. *Basic Boost.Asio Anatomy* [https://www.boost.org/doc/libs/latest/doc/html/boost_asio/overview/basics.html]. Boost C++ Libraries, 2026. Accessed: 2026-04-29.
44. ZEROMQ AUTHORS. *ZeroMQ: Get Started* [<https://zeromq.org/get-started/>]. 2026. Accessed: 2026-04-29.
45. HINTJENS, Pieter; HURTON, Martin; BARBER, Ian. *ZeroMQ Message Transport Protocol (ZMTP) 3.0* [<https://rfc.zeromq.org/spec/23/>]. iMatix Corporation and Contributors, 2014. ZeroMQ RFC 23, status: stable.
46. BISCANI, Francesco; IZZO, Dario. A parallel global multiobjective framework for optimization: pagmo. *Journal of Open Source Software*. 2020, vol. 5, no. 53, p. 2338. Available from DOI: 10.21105/joss.02338.
47. PAGMO DEVELOPMENT TEAM. *Evolving a population*. 2024. Available also from: https://esa.github.io/pagmo2/docs/cpp/tutorials/evolving_a_population.html. pagmo 2.19.1 documentation, accessed 2026-04-30.
48. PAGMO DEVELOPMENT TEAM. *Quick start*. 2024. Available also from: <https://esa.github.io/pagmo2/quickstart.html>. pagmo 2.19.1 documentation, accessed 2026-04-30.
49. DUFEK, Jonáš. *Possible 50% performance improvement for NSGA-II on MSVC*. 2026. Available also from: <https://github.com/esa/pagmo2/pull/616>. GitHub pull request #616, pagmo2 repository, accessed 2026-04-30.
50. DUFEK, Jonáš. *Pagmo-NSGA2-Improve-Test*. 2026. Available also from: <https://github.com/jonas-s-s-s/Pagmo-NSGA2-Improve-Test>. GitHub repository, accessed 2026-04-30.

51. MICROSOFT. *Floating-point primitives (C runtime library reference)*. 2025. Available also from: <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/floating-point-primitives?view=msvc-170>. Last updated 2025-09-30, accessed 2026-04-30.
52. KOUTNY, Tomas. *SmartCGMS: Next-Generation Continuous Glucose Monitoring System* [<https://diabetes.zcu.cz/smartcgms/>]. n.d. Accessed: 2026-05-02.
53. ZEROMQ CONTRIBUTORS. *ZeroMQ: High-performance asynchronous messaging library* [<https://zeromq.org/>]. 2026. Accessed: 2026-05-03.
54. ZEROMQ CONTRIBUTORS. *cppzmq: Header-only C++ binding for libzmq* [<https://github.com/zeromq/cppzmq>]. 2025. Version 4.11.0, accessed 2026-05-03.
55. CATCH2 PROJECT. *Catch2: A modern, C++-native test framework for unit tests, TDD and BDD* [<https://github.com/catchorg/Catch2>]. 2025. Version 3.11.0, accessed 2026-05-03.
56. EIGEN CONTRIBUTORS. *Eigen: A C++ template library for matrices, vectors, numerical solvers, and related algorithms* [<https://gitlab.com/libeigen/eigen>]. 2019. Accessed: 2026-05-03.
57. BOOST.ORG. *Boost C++ Libraries* [<https://www.boost.org/>]. 2026. Accessed: 2026-05-03.
58. INTEL CORPORATION. *Intel oneAPI Threading Building Blocks (oneTBB)* [<https://www.intel.com/content/www/us/en/developer/tools/oneapi/onetbb.html>]. 2024. Accessed: 2026-05-03.
59. ZEROMQ CONTRIBUTORS. *Zero-copy and Multi-part Messages* [<http://wiki.zeromq.org/blog:zero-copy>]. 2012. Available also from: <http://wiki.zeromq.org/blog:zero-copy>. Originally written 2010-05-08, revised 2012-08-01.
60. BRUMME, Stephan. *Portable C++ Hashing Library* [<https://github.com/stbrumme/hash-library>]. 2026. GitHub repository, accessed 2026-05-07.
61. XAI GROK. *Implementing Unit Tests for Distributed Solver* [Online. In: Grok]. 2026. Available also from: https://grok.com/share/c2hhcmQtMw_cf4f0d90-2075-4bda-b145-e81951339756. [Accessed 2026-05-07].
62. DEEPSEEK. *Implement unit testing with Catch2 for distributed solver* [Online. In: DeepSeek Chat]. 2026. Available also from: <https://chat.deepseek.com/share/v85yb7yuk0v6h5io4l>. [Accessed 2026-05-07].

63. GITHUB COPILOT. *Unit testing implementation for distributed solver* [Online. In: GitHub Copilot]. 2026. Available also from: [https://github.com/copilot/share/424e131a - 4200 - 8081 - 9143 - b641c01e092d](https://github.com/copilot/share/424e131a-4200-8081-9143-b641c01e092d). [Accessed 2026-05-07].

List of Figures

2.1	An illustration of gradient descent [6], a classical iterative optimization algorithm commonly used in Machine Learning. Notice that at each iteration, the derivative of the objective function defines a descent direction, only a single “candidate solution” is maintained. Population-based methods don’t follow this concept, instead they operate on a “population” of many different “candidate solutions”.	7
2.2	Example of a population-based solver, the solver has two types of inputs: the problem containing both the objective function and the constraints, and an arguments structure, containing the generations count and population size. The arguments structure can possibly include other fields, such as for example initial population members. After running the solver, the best population member is chosen and returned back to the user.	10
2.3	Illustration of a one-point crossover, the SGA performs this crossover with the probability p_c , otherwise the children stay the same as their parents. The crossover point is chosen randomly using a uniform RNG.	12
2.4	A mutation performed on two bits of the parent chromosome. SGA mutates each bit with the probability p_m	13
2.5	An (1 + 1) ES, parents are designated by gray background, offspring by yellow background. Individual with the highest fitness in each generation is highlighted in red.	14
2.6	An (2 + 4) ES, so a variant of $(\mu + \lambda)$ ES. Notice the increased rate of convergence as opposed to the (1 + 1) ES example. It can also be seen that this type of algorithm is highly parallelizable, both the mutation step and the selection step can be parallelized across many GPGPU or CPU cores.	14
2.7	Example parse tree of a computer program. Source: [21]	18

2.8	The translation phases of a typical C language compiler, in accordance with ISO/IEC 9899 (C99), for a detailed explanation of these phases see: [22, p. 9]. The image highlights at which stage of the translation process is genetic programming situated, as opposed to large language models.	18
2.9	An example of a fully-connected island model, different shapes represent individuals migrating from one island to another. Source: [26]	23
3.1	PVM architecture depicted as a number of <i>processing elements</i> (“pe”) interconnected over a network. The master PVMD has the ability to spawn slave PVMDs on any of the “processing elements”. Source: [32]	27
3.2	Header of the PVMD UDP protocol, using Task IDs for identification and additional fields to ensure the reliable transfer of messages. Source: [31, p. 113]	27
3.3	A typical pattern of code distribution in active networks, each node maintains a “code group” (cache), if the node doesn’t have the required code to process this capsule, it can request it from the previous node. Source: [36, p. 4]	32
3.4	Example structure of an active networks capsule. Source: [34]	32
4.1	Overlay network example. Source: [38]	34
4.2	A computational cluster where each worker node uses a population-based optimization algorithm to evolve a population of candidate solutions to an optimization problem received from the controller, results sent back by workers accumulate at the controller. It can be seen that in this schema workers don’t connect to each other, instead they only connect to the controller, this is intentional as it allows us to centralize all decision making logic (such as migration of individuals between islands) to the controller, thus this is where logic of the “overlay network” described above will be implemented.	35
5.1	Boost.Asio callback style asynchronous model, the numbers signify: 1) User program initiates a connect operation by calling the I/O object (socket), 2) The I/O object forwards this request to the execution context (io_context), 3) Execution context signals the underlying OS API to start an asynchronous connect operation. The function “Your Completion Handler” is then notified once an event occurs on the socket. (Inspired by: [43])	42
6.1	Internal details of how the “Archipelago-Island-UDI” model works. Left portion of the figure shows the program flow (see listing 6.2), right portion of the figure shows the internals of each function.	51

6.2	Initial flame graph (before any optimizations)	52
6.3	MSVC profiler showing the CPU time usage of function <code>greater_than_f</code>	52
6.4	Flame graph after optimizing the <code>greater_than_f</code> function	54
6.5	Flame graph after fixing <code>greater_than_f</code> and inlining <code>pareto_dominance</code>	55
6.6	Final flame graph after applying all fixes	55
6.7	Before and after flame graph of the same application compiled with Clang instead of MSVC	56
6.8	Closed-loop glucose control workflow	56
6.9	SmartCGMS workflow	57
7.1	Architecture of the metasolver distributed system, showcasing three connected workers.	61
8.1	Structure of the protocol's messages. The Payload field can contain an arbitrary serialized object, the figure shows how the payload would look like with a <code>work_container</code> object inside of it.	65
8.2	Sequence diagram showing the typical interaction between the con- troller and worker. The <code>GET_DLL</code> and <code>DLL_BINARY</code> steps are op- tional, they only happen if worker doesn't have the DLL cached.	66
9.1	Subdirectory tree of the library, with root at the <code>./src/</code> directory. . .	67
9.2	The logic subdirectory.	68
9.3	The messages subdirectory.	78
9.4	The sockets subdirectory.	79
9.5	OOP inheritance hierarchy of the socket classes.	80
9.6	The dynamic subdirectory.	81
9.7	The discovery subdirectory.	83
9.8	The utils subdirectory.	86
9.9	The benchmark subdirectory.	87
9.10	The <code>./test/</code> directory and its contents, as created by GitHub Copilot.	89
10.1	The <code>distributed4scgms</code> wrapper library, notice that the library is split into three parts. "solver-distributed" is the distributed metasolver li- brary, it's included as a git submodule.	92
11.1	The simple distributed scenario. Notice that each node has the exact amount of <code>thread_islands</code> as its logical core count.	96
11.2	Solution fitness comparison between the "local step" and "distributed step".	98

11.3	Pie chart showing the total time workers spent performing computations (over all 10 runs) for both the local and distributed variant. The “local step” took on average 9.94 seconds until results were returned to controller, in contrast to that the “distributed step” returned its results to controller on average in 4.64 seconds.	99
11.4	A chart showing how the solver’s load-balancing mechanism adjusts population sizes between two workers. During the first iteration population sizes for both workers are equal, as no data on their performance has been collected yet, this starts to change in subsequent iterations.	99
11.5	An ARCHIPELAGO_BASED worker splits the total input population into multiple parts, depending on the amount of local islands. There is one local island for every logical CPU core, meaning that Node A on average ends up with $\frac{4875}{8} = 609.375$ population members per island, as we can see Node B has a much lower per-island population due to its 40 logical cores: $\frac{5127}{40} = 128.17$	100
11.6	The Three-Node Distributed Scenario.	101
11.7	Fitness of the local variant (1 localhost worker) compared to the distributed variant (1 localhost worker + 2 remote workers).	103
11.8	The total time workers spent performing computations over all 10 runs. When compared to fig. 11.3, elapsed time for the local variant decreased by about 15 seconds, this could’ve been caused by the non-determinism of the optimization algorithm and the influence of external factors such as the operating system’s process scheduler. The distributed variant’s time is also lower, the difference here is about 4 seconds.	103
11.9	The total population assigned to each node. As was specified in table 11.3, we are splitting a “global” population of 10 000 among three workers. During the first run all workers receive the same portion because no data about their performance is known yet, the load balancing mechanism then adjusts the population in subsequent runs.	104
11.10	Average size of each node’s local island. Node C has the lowest average total population assigned to it, however it ends up with the second largest per-island population, because it has to split the input population only between 4 local islands.	104
11.11	Average total population assigned to each node. Naturally Node C has the smallest population due to it having the least amount of logical CPU cores.	105
11.12	A line chart showing the difference in fitness between using a single algorithm solver (<i>Fitness Normal</i>) and metasolving (<i>Fitness Meta</i>). Both variants were using three worker nodes, as specified in table 11.4.	107

11.13	The time it took for worker nodes in each iteration to finish their work and return results to controller.	108
11.14	Total time worker nodes spent performing computations.	108
11.15	A line chart showing the difference in fitness between metasolving and a single-algorithm solver. Notice the slight improvement over figure 11.12.	109
11.16	The time worker nodes spent performing computations in each iteration. Little to no improvement when compared with figure 11.13. . . .	109
11.17	Metasolver's total elapsed time was significantly higher than in figure 11.14, due to the second iteration taking nearly 360 seconds (see fig. 11.16)	110
11.18	Comparison between a single-algorithm solver (<i>Fitness Normal</i>) and a metasolver (<i>Fitness Meta</i>).	112
11.19	Time spent by worker nodes performing computations in each iteration. The metasolver was often slower by as much as ten times.	112
11.20	Total time spent by workers performing computations. Showing the significant advantage of the single-algorithm solver.	113
11.21	A bar chart showing extracted data from the log files of our workers, each worker logs the algorithm type and the total population which was assigned to it, this is displayed chronologically on the x axis. As we can see, the CMA-ES algorithm's population was slowly decreased by the load-balancing algorithm. This is because CMA-ES was by far the slowest, which meant that it was slowing down the entire metasolver. .	113
11.22	The load-balancing mechanism as a reaction to the inefficiency of CMA-ES increased population assigned to the PSO algorithm. This was likely the cause of the improvement seen around iteration 4 in figure 11.19. .	114
11.23	The metasolver was able to reach fitness of 0 (ideal solution) in almost every iteration, massively outperforming the single-algorithm solver. .	114
11.24	The metasolver was slightly slower, however this is still a very good result when compared to figure 11.19.	115
11.25	The time over all iterations is almost balanced, meaning that this is a good result for the metasolver.	115
11.26	Fitness reached by each algorithm on every tested problem, lower fitness is better. Metasolver, DE, ABC and sa-DE1220 have reached fitness of 0 (perfect solution) on all five problems.	117
11.27	Execution time of each algorithm on every tested problem.	118
11.28	Fitness reached by each algorithm on the first objective function. . . .	120
11.29	Fitness reached by each algorithm on the second objective function. .	120
11.30	Execution time of each algorithm on every tested problem.	121

List of Tables

2.1	Basic terminology of population-based algorithms	9
2.2	Overview of Evolution Strategy types	15
5.1	Overview of ZeroMQ socket patterns	46
8.1	Overview of the different messages sent between a worker (W) and the controller (C). The <code>WORKER_LEAVE</code> message is not actually sent by the worker itself, rather it's internally generated by ZeroMQ when the worker's connection is determined to be closed (no heartbeat signal received for a certain period of time).	65
9.1	Results of the generated unit tests by different LLMs.	89
11.1	Hardware specifications of the nodes used in this chapter. C_P stands for <i>Physical Cores</i> , whereas C_L is the number of <i>Logical Cores</i>	95
11.2	Configuration of the simple distributed scenario's controller program.	97
11.3	Configuration of the three-node distributed scenario's controller program.	102
11.4	Configuration of the distributed metasolving scenario's controller program.	106
11.5	Configuration of the single-objective benchmark's controller program.	116
11.6	Configuration of the multi-objective benchmark's controller program.	119

List of Listings

2.1	A generalized population-based algorithm	8
2.2	The Simple Genetic Algorithm (SGA)	12
3.1	Example of a PVM master, which starts a single slave and then blocks until a “hello” message is received	28
3.2	A PVM slave, which receives its parent TID, constructs a message, and sends it back to the master	29
5.1	BSD Sockets client in C++, see listing 5.2 for the corresponding server code.	39
5.2	BSD Sockets server in C++	40
5.3	Boost.Asio TCP client	43
5.4	Boost.Asio “hello world” server	43
5.5	ZeroMQ ROUTER server example	47
5.6	ZeroMQ DEALER client example	47
6.1	Running an optimization algorithm in Pagmo (no islands) [47] . .	48
6.2	Basic example of archipelago in Pagmo [48]	50
6.3	Function <code>greater_than_f</code> inside of <code>pagmo/detail/custom_comparisons.hpp</code>	53
6.4	Optimized body of <code>greater_than_f</code>	53
7.1	A minimal controller node	62
7.2	A minimal worker node	63
B.1	Controller program used for the simple distributed scenario. . . .	132
B.2	Controller program used for the distributed metasolving scenario.	133

1101001
101011000011100010 1100001
101011010101 10



11010011101101001
01100001 101
111000101011 101